QUEUE2PLAY

Complete System Overview & Documentation

Java console game-rental system
Study guide, technical reference, full user manual and DSA defense reviewer

DOCUMENT BASIS	VERIFIED SCOPE
Source snapshot	12 October 2026; all 16 current application Java files and the current regression suite.
Application entry point	Main.main() starts Queue2Play.
Verification	Compiled with JDK 24; complete BehaviorRegressionTest rerun passed 63,287 checks using isolated data.
Change policy	Documentation only. Original application and test source files were not edited.
Reference style	Filename:starting-ending line. Every method reference is generated from the current source snapshot.

The current code is the authority. Earlier plans and screenshots can contain older keys or rules; this document describes what is implemented now. No private customer data is reproduced.

Contents

Contents 2

How to use this documentation . . . 5

1. Project overview 6

1.1 Architecture and object connections . . . 7

1.2 Rules that define the current system . . . 7

2. Java files and classes 8

2.1 Domain records and shared identity . . . 9

2.2 Test Java file and support fixtures . . . 9

3. DSA analysis 10

3.1 Complexity and assumptions . . . 10

3.2 Catalog search and stable sorting . . . 11

3.3 Queue node operations and fairness are different concerns . . . 12

3.4 Heap, stack and hash defense points . . . 12

3.5 DSA in every module . . . 13

4. Important methods and transaction connections 14

5. Complete user manual: start, login and registration 17

5.1 Starting the project . . . 17

5.2 Built-in demonstration accounts . . . 17

5.3 Login from start to finish . . . 18

5.4 Registration from start to finish . . . 18

6. Complete user manual: Regular and VIP customers 19

6.1 Browse, search, sort and pages . . . 19

6.2 Place an order from start to finish . . . 19

6.3 Cost examples and what is charged . . . 20

6.4 My Queue and cancellation . . . 21

6.5 Rental history . . . 21

6.6 Upgrade to VIP . . . 21

6.7 Back, discard and error recovery . . . 21

7. Complete user manual: administrator 22

7.1 Checkout queue and one request . . . 22

7.2 Batch checkout . . . 23

7.3 Administrator queue cancellation . . . 23

7.4 Receipt navigation . . . 23

7.5 Process one or several returns . . . 23
7.6 Inventory: browse and add a game . . . 24
7.7 Inventory: update stock . . . 24
7.8 Inventory: delete a game . . . 24
7.9 Active rentals, audit and logout . . . 25

8. Regular and VIP queue behavior **26**

8.1 Eligibility, position and the three-VIP rule . . . 26
8.2 Shared stock and recalculation . . . 26
8.3 VIP upgrade and different cancellation paths . . . 27

9. Validations and error handling **28**

9.1 Login and registration validation . . . 28
9.2 Rental, quantity and stock validation . . . 29
9.3 Exact common diagnostics . . . 29
login() messages - AccountManager.java:353-400 . . . 29
register() messages - AccountManager.java:207-337 . . . 30
validateRentalSelection() messages - Queue2Play.java:522-539 . . . 30
validateCheckout() messages - RentalManager.java:122-144 . . . 30
validateCart() messages - RentalManager.java:212-239 . . . 31
9.4 Error handling and safe retry . . . 31

10. File storage, updates and recovery **32**

10.1 File formats . . . 32
10.2 Locate and load data . . . 32
10.3 Backward compatibility . . . 33
10.4 Recoverable five-file save . . . 33
10.5 In-memory rollback and save boundaries . . . 34

11. Developer and operational guide **35**

11.1 Build and test commands . . . 35
11.2 Supported runtime settings . . . 35
11.3 How to trace a feature . . . 35
11.4 Core invariants for future maintenance . . . 36

12. Testing and DSA defense reviewer **37**

12.1 Tests actually executed for this report . . . 37
12.2 Verified regression coverage . . . 38
12.3 Practical demonstration scenarios . . . 39
12.4 Technical defense questions and answers . . . 40

13. Program analysis: issues and recommendations **43**

High, conditional: Post-commit read failure can disagree with the memory rollback . . . 43

Medium, source-based: Order Submitted remains unpaged . . . 43
Medium, source-based: Remove-line prompt adds output after a full cart page . . . 43
Medium, source-based: Checkout tape is fixed at 81 columns . . . 43
Medium, source-based: Cancellation history is not consistent across paths . . . 43
Medium, boundary-dependent: Return preview freezes return time before confirmation . . . 43
Low, source-based: Oversized record can exceed height on its own page . . . 44
Low, source-based: Viewport is fixed for the session . . . 44
Low, source-based: Character count is not terminal cell width . . . 44
Low, extreme boundary: All platform counters can block one platform . . . 44
13.1 Missing features versus bugs . . . 44
13.2 TUI recommendations . . . 45

Appendix A. Complete application method reference 46

Main.java - 1 declarations . . . 46
Queue2Play.java - 61 declarations . . . 46
AccountManager.java - 19 declarations . . . 52
GameCatalog.java - 14 declarations . . . 54
RentalManager.java - 18 declarations . . . 55
CheckoutQueue.java - 22 declarations . . . 57
RentalDataStore.java - 24 declarations . . . 59
AppStorage.java - 10 declarations . . . 61
PasswordSecurity.java - 6 declarations . . . 62
User.java - 3 declarations . . . 62
Game.java - 2 declarations . . . 63
RentalRequest.java - 1 declarations . . . 63
Rental.java - 2 declarations . . . 63
TerminalUI.java - 227 declarations . . . 64
StorageException.java - 2 declarations . . . 82
InputEndedException.java - 0 declarations . . . 83

Appendix B. Complete validation and diagnostic reference 84

AccountManager.java - 18 diagnostic/constraint sites . . . 85
AppStorage.java - 9 diagnostic/constraint sites . . . 87
CheckoutQueue.java - 1 diagnostic/constraint sites . . . 87
GameCatalog.java - 8 diagnostic/constraint sites . . . 88
PasswordSecurity.java - 4 diagnostic/constraint sites . . . 88
Queue2Play.java - 94 diagnostic/constraint sites . . . 89
RentalDataStore.java - 65 diagnostic/constraint sites . . . 97
RentalManager.java - 24 diagnostic/constraint sites . . . 102
TerminalUI.java - 25 diagnostic/constraint sites . . . 105
B.1 Boolean validation predicates and format checks . . . 107
B.2 Stored-record validation overview . . . 110

Appendix C. Source snapshot and reference manifest **111**

How to use this documentation

Start with the overview and user manual. For defense preparation, study the DSA chapter and queue scenarios. For implementation details, use the important-method tables and Appendix A. For an exact diagnostic or storage rule, use Appendix B and the file-storage chapter.

A method is a named function inside a Java class. A constructor creates an object and has no return type. A parameter is the input listed in a method declaration. A return value is what the caller receives. DSA means data structures and algorithms: the way records are stored, searched, ordered and processed.

Evidence labels: Executed means the check actually ran for this documentation. Source-based means the result follows from the inspected code. Conditional means a risk is possible under a specific failure or boundary, but was not reproduced in this run. An expected result is not automatically a test PASS.

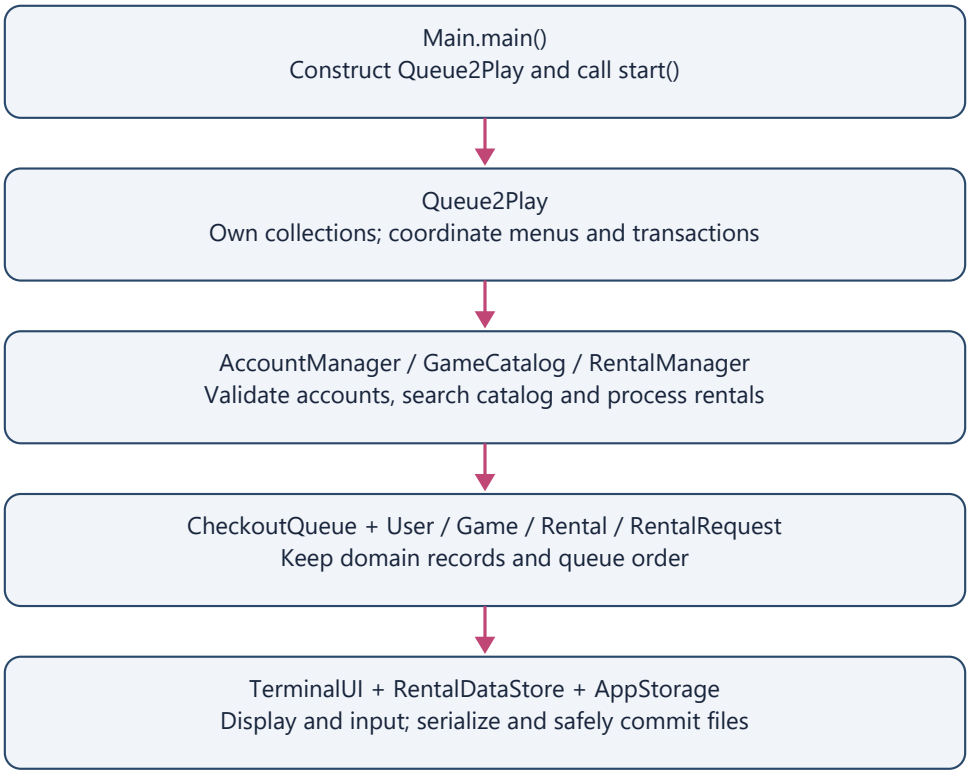
1. Project overview

QUEUE2PLAY is a local terminal application for renting PS5, Xbox and Nintendo games. Customers browse the catalog, prepare orders, join a checkout queue, see queue positions and view rental history. Administrators approve eligible requests, process returns, maintain game stock and inspect active rentals and audit records. Accounts, inventory, rentals, pending requests and logs are saved in text files.

AREA	CURRENT FUNCTIONALITY
Accounts	Username or email login; masked password; registration with confirmation and field editing; three failed attempts before a 30-second wait; Regular and VIP clients plus administrators.
Catalog	All games or platform filter; complete ID, partial title or complete platform search; five display orders; responsive pages with a maximum of nine games.
Orders	Cart merges repeated selections of a game. New queued quantity plus cart quantity is limited to three copies per game. Duration is 1, 3, 7 or a custom 1-30 days.
Queue	VIP arrival order, stock-aware processing and a regular turn after three successful VIP checkouts when a regular request fits stock. Predicted positions agree across admin and customer displays.
Checkout	One request or all predicted-ready requests; Yes/No confirmation; numbered result slips, session totals, queue viewer and paged receipt.
Returns	Earliest due first; select one or several current-page rental numbers; confirm the selection; restore full quantities; calculate and record late fees.
Inventory	Browse, add a game, change stock and delete a game if it has no active rental. Game titles, platforms and prices can be edited in a new-game draft before saving.
Storage	UTF-8 pipe-separated files; state validation; recoverable multi-file journal; stale-write detection and in-memory rollback. No permanent .txt.bak creation.

1.1 Architecture and object connections

Main startup and service wiring



Queue2Play uses composition: it has services, a queue, collections and a UI. The services hold references to the same records, so changing a game stock or rental status affects every view that reads that object. TerminalUI handles presentation; RentalManager handles rental rules; RentalDataStore turns records into file text; AppStorage handles physical file replacement and recovery. See Queue2Play.java:119-124, Queue2Play.java:144-187, RentalDataStore.java:112-158 and AppStorage.java:145-207.

1.2 Rules that define the current system

RULE	MEANING AND SOURCE
3 copies for new requests	The limit is per customer and per game, including existing queued copies and the current cart. It is not a limit of three different game titles. Queue2Play.java:12-24; RentalManager.java:212-239.
5 copies for saved records	Legacy queue and rental records may contain up to five copies. Checkout validation also preserves the five-active-copy compatibility limit. CheckoutQueue.java:56-60; RentalManager.java:122-144.
20 total stock	Available copies plus copies currently rented cannot exceed 20 for a game. Queued copies are not reserved stock. RentalManager.java:44-55; RentalDataStore.java:161-209.
$1-$10 game price	This is the base price for the default three-day rental; duration and quantity determine the charged amount. GameCatalog.java:45-51; RentalManager.java:58-65.
10% VIP discount	VIP base price receives a 10% reduction. Membership upgrade is free and remains saved. RentalManager.java:58-65; Queue2Play.java:1106-1137.
Stock changes	Submitting a queue request does not deduct stock. Approved checkout deducts it; return restores it; queue cancellation does not change stock. RentalManager.java:147-173; RentalManager.java:196-209.

2. Java files and classes

FILE / LOCATION	RESPONSIBILITY	CONNECTIONS
Main.java Class declaration: line 1	Entry point; handles input-end and fatal startup/storage errors.	Queue2Play, InputEndedException, StorageException.
Queue2Play.java Class declaration: line 9	Application controller: menus, carts, checkout session, returns, inventory and navigation. Owns the shared in-memory state.	All application services, TerminalUI and domain classes. Nested CheckoutEntry and CheckoutSession hold temporary session records.
TerminalUI.java Class declaration: line 13	Input, masking, colors, headings, tables, wrapping, pagination, messages, confirmations and result navigation.	Receives User, Game, Rental, RentalRequest, CheckoutQueue and service/session data; does not own the business inventory.
AccountManager.java Class declaration: line 3	Load account records; restore built-in accounts; validate registration; find users; register and authenticate.	User, PasswordSecurity, RentalDataStore and TerminalUI.
GameCatalog.java Class declaration: line 4	Game validation and ID counters; starter games; platform list; linear search; stable insertion sort.	Game and the shared game-ID map.
RentalManager.java Class declaration: line 7	Rental price, active-copy checks, checkout validation, due-order listing, late fees, returns and cart validation.	CheckoutQueue, Game, User, Rental and the shared heap/rental list/catalog.
CheckoutQueue.java Class declaration: line 5	Manual doubly linked list with VIP insertion, fairness, cancellation and shared-stock preview.	RentalRequest and Game. Nested Node has request, previous and next references.
RentalDataStore.java Class declaration: line 7	Load and validate records; serialize all five files; checkpoint and restore memory; logs.	Queue2Play and AppStorage. Nested Record preserves file line numbers; Snapshot holds rollback state.
AppStorage.java Class declaration: line 9	Find the data directory; acquire file lock; remember bytes; recover interrupted saves; commit all files.	Java NIO filesystem APIs and StorageException.
PasswordSecurity.java Class declaration: line 7	Create and verify salted PBKDF2 password hashes; validate stored hash encoding.	User and AccountManager; standard Java security/crypto APIs.
User.java Class declaration: line 3	Username, full name, email, hash, role, membership and a rental-history stack. Role and membership queries.	PasswordSecurity and Rental.
Game.java Class declaration: line 1	Game ID, title, platform, three-day base price and available stock.	Used by catalog, request, cart and rental records.
RentalRequest.java Class declaration: line 1	Queue number, client, game, VIP flag, rental days and quantity. Default request is three days and one copy.	User, Game and CheckoutQueue.
Rental.java Class declaration: line 3	Rental/history number, client, game, dates, amount, quantity, status and return/late-fee fields. Implements Comparable.	PriorityQueue uses compareTo() to order due date, then rental number.
InputEndedException.java Class declaration: line 1	Signals that terminal input ended without submitting the pending input.	TerminalUI throws it; Main handles it. No explicit constructor or method is declared.
StorageException.java Class declaration: line 2	Runtime exception carrying a storage message and optional cause.	AppStorage/RentalDataStore throw it; transaction handlers or Main report it.

2.1 Domain records and shared identity

A Game is shared by the catalog, carts, queue and active rentals. The catalog map must hold that same canonical Game object for a live request or rental. A User is shared by requests, rentals and account records. sameUser() and sameGame() compare stable username/game ID values where appropriate; canonical-object checks protect saved state. See RentalManager.java:23-25, RentalManager.java:28-30 and RentalDataStore.java:161-209.

RentalRequest is pending work, not an active rental. Rental represents ACTIVE, COMPLETED and administrator-created CANCELLED history. CheckoutSession is temporary and not saved as a separate session file. Its entries, counters, copies and total build the current receipt. Actual approved rentals and cancellation history are saved. See Queue2Play.java:1216-1218, Queue2Play.java:1221-1226 and Queue2Play.java:1229-1232.

2.2 Test Java file and support fixtures

Backfiles testing/tests/BehaviorRegressionTest.java is the seventeenth current Java file, used for verification rather than application startup. BehaviorRegressionTest is declared at line 9. Its main at lines 2890-2996 captures terminal output, creates a disposable root, selects targeted or complete groups and prints the assertion count. It connects directly to the application classes; the production launcher compiles only top-level application sources.

TEST SUPPORT CLASS / METHOD	RESPONSIBILITY / EXACT LOCATION
ScriptedUI extends TerminalUI	Begins at BehaviorRegressionTest.java:14. Feeds predetermined choices, captures screens/errors/slip counters/positions and delegates many renderers to the real UI.
AccountUI extends ScriptedUI	Begins at line 174. Supplies scripted password input, counts mocked one-second waits and records attempt/countdown/registration summaries. It does not exercise real Windows password capture.
FailingApprovalSystem extends Queue2Play	Begins at line 226. Makes the second approval fail while the first follows the real saved checkout; checks partial batch success behavior.
check(condition,message)	Lines 239-247: increments checks, reports recent scripted UI errors and throws AssertionError if the condition fails.
freshSystem(name)	Lines 249-252: points q2p.dataDir at a named disposable subfolder and constructs an application controller.
script(system,choices)	Lines 254-259: installs ScriptedUI into the controller and returns it.
client / enqueue / accountScript	Lines 261-276: create synthetic User/request objects and an AccountManager using the scripted UI.

The test code uses arrays and ArrayLists for input/output recording, temporary filesystem paths for isolation, and test overrides for controlled flows. Some checks inspect internal state through reflection. These are testing tools, not customer features or new production DSA. Chapter 12 maps the important test groups to their exact starting lines and states what actually ran.

3. DSA analysis

The main DSA are linear search, stable insertion sort, a custom doubly linked checkout queue, a min-heap for active rentals, stacks for history/audit, and hash-based maps and sets. ArrayLists and arrays support forms, results, snapshots and pagination. There is no custom binary search tree, binary search, jump search, bubble sort or selection sort in the current application.

DSA	HOW AND WHY IT IS USED	ACTUAL LOCATIONS
Linear search	Accounts and duplicates are scanned without requiring sorted input. Catalog search checks every selected-platform game because it can match part of a title.	AccountManager.java:168-175; AccountManager.java:178-190; GameCatalog.java:118-129.
Stable insertion sort	Small display lists are copied and then shifted into order. Equal title/price values retain their incoming order. Also sorts saved arrival/history numbers for rebuilding.	GameCatalog.java:141-152; GameCatalog.java:155-165; CheckoutQueue.java:43-53; RentalDataStore.java:336-346.
Doubly linked queue	Each node links to previous and next. VIP insertion can occur before the first regular; unlinking an already located node handles head, tail and middle.	CheckoutQueue.java:11-13; CheckoutQueue.java:79-103; CheckoutQueue.java:106-115; CheckoutQueue.java:128-141.
PriorityQueue min-heap	Active rentals are ordered by earliest due date, then rental number. Copying the heap and polling produces a sorted return/display list without removing real rentals.	Queue2Play.java:49-50; Rental.java:30-42; RentalManager.java:112-119.
Stacks	New rentals/history and administrator actions are pushed onto stacks. Reverse indexed viewing shows the newest records without destructive pop().	User.java:15-23; Queue2Play.java:195-199; Queue2Play.java:1058-1103; Queue2Play.java:1635-1690.
LinkedHashMap	Inventory uses game ID keys with insertion order. Cart uses canonical Game keys with merged quantity. Preview stock maps give all requests for one game the same simulated stock.	Queue2Play.java:43-58; Queue2Play.java:599-635; CheckoutQueue.java:256-286.
HashSet	Detect duplicate usernames/emails, record numbers and active records during file validation. Efficient average membership checks.	RentalDataStore.java:161-209; RentalDataStore.java:212-250; RentalDataStore.java:253-333; RentalDataStore.java:349-399.
ArrayList and arrays	Resizable user/rental/result lists, selected returns, row heights and page starts. Parallel snapshot arrays preserve stocks, membership and return fields for rollback.	Queue2Play.java:1448-1477; TerminalUI.java:65-82; RentalDataStore.java:474-512; RentalDataStore.java:514-559.

3.1 Complexity and assumptions

Let u = users, g = games, q = queued requests, a = active rentals, r = all history/rental records, and b = bytes serialized. String operations also depend on text length. Bounds below treat validated short strings as bounded unless stated. Hash lookups are average O(1), not a guaranteed worst-case promise. Saving is separate work and can cost more than the in-memory operation.

OPERATION	TIME	EXTRA SPACE / IMPORTANT LIMIT
findUser / findUserByLogin	O(u)	O(1). Username and email search are not hash-map lookup. Password verification adds the deliberately expensive PBKDF2 work.
getGameList / linearSearch	O(g) / O(n) selected-list scan	O(n) results. Partial-title matching requires substring work too.
getDisplayList + insertionSort	Best O(n); worst $O(n^2)$	O(n) copied list; insertionSort itself uses O(1) working variables.
Regular enqueue / VIP enqueue	O(1) / O(q)	O(1) new node; VIP walks the existing VIP group.

OPERATION	TIME	EXTRA SPACE / IMPORTANT LIMIT
find/remove request / unlink node	O(q) / O(1) after locating node	Search is not constant time merely because the list is linked.
findNextEligibleRequest	O(q)	O(1); may scan regular requests, then the complete priority list.
previewReadyRequests / predicted position	$O(q^2)$ under average map access	O(q+g). Copying/insertion, repeated linear selection/removal and original-ID mapping cause quadratic worst-case work.
getDisplayRequests	$O(q^2)$ with ArrayList contains	O(q). Ready entries first; then waiting entries.
Heap add / peek / sorted copy	O(log a) / O(1) / O(a log a)	Sorted copy uses O(a). Heap iteration alone is not guaranteed due-date order.
Heap remove(object) / active-copy scan	O(a) search plus heap repair / O(a)	Do not claim every heap removal is O(log a): this code removes a supplied object.
Stack push / reverse display	Amortized O(1) / O(k)	Display does not pop records. File loading also performs number sorting.
Return search	O(a log a) + O(a) text scan	O(a); it first builds the due-ordered heap copy.
pageStarts	O(n)	O(number of pages); text/row measurement is additional work.
Serialization and save	O(b) physical byte writing, plus validation	State validation includes repeated list/heap scans; can include $O(a^2+q^2+qa+ga+ru)$. It is not simply O(1) or only O(n).
History/queue rebuild	Number insertion sort can be $O(r^2)$ / $O(q^2)$	Collections and canonical references consume O(u+g+r+q).

3.2 Catalog search and stable sorting

linearSearch() ignores case using a trimmed lower-case query. A game matches if its complete ID equals the query, its title contains the query, or its complete platform equals the query. Searching runs against the whole selected platform list before pagination. For example, "P001" is an ID match and "spider" is a title match; "PS" is not a complete platform match. An empty search returns all input games.

Actual source: GameCatalog.java:118-129

```
118  ArrayList<Game> linearSearch(ArrayList<Game> list, String keyword) {
119      ArrayList<Game> results = new ArrayList<>();
120      String key = keyword.trim().toLowerCase(java.util.Locale.ROOT);
121      for (Game game : list) {
122          if (game.gameId.toLowerCase(java.util.Locale.ROOT).equals(key)
123              || game.title.toLowerCase(java.util.Locale.ROOT).contains(key)
124              || game.platform.toLowerCase(java.util.Locale.ROOT).equals(key)) {
125              results.add(game);
126          }
127      }
128      return results;
129  }
```

Insertion sort takes the next game, shifts earlier games only when they should be after it, then inserts it. Equal values are not shifted, which makes the sort stable. Default mode bypasses sorting. getDisplayList() creates the search result/copy, so inventory order is unchanged.

Actual source: GameCatalog.java:141-152

```
141  void insertionSort(ArrayList<Game> list, int option) {
142      RentalDataStore.require(option >= 1 && option <= 4, "Invalid sort option");
143      for (int i = 1; i < list.size(); i++) {
144          Game currentGame = list.get(i);
145          int j = i - 1;
146          while (j >= 0 && shouldMoveRight(list.get(j), currentGame, option)) {
147              list.set(j + 1, list.get(j));
148              j--;
149          }
150          list.set(j + 1, currentGame);
151      }
152  }
```

3.3 Queue node operations and fairness are different concerns

The doubly linked list stores VIP requests before regular requests, with FIFO within each membership group. That physical list is not itself the final checkout order. findNextEligibleRequest() adds the VIP streak and stock eligibility rules. A blocked head can be skipped; no request is partially fulfilled. Membership refresh sorts by original queue number and reinserts each request, retaining arrival order after an upgrade.

Actual source: CheckoutQueue.java:229-245

```
229  RentalRequest findNextEligibleRequest() {
230      if (vipStreak >= VIP_BURST) {
231          for (Node current = front; current != null; current = current.next) {
232              RentalRequest request = current.request;
233              if (validRequest(request) && !request.vip && request.game.availableCopies >=
     request.quantity) {
234                  return request;
235              }
236          }
237      }
238      for (Node current = front; current != null; current = current.next) {
239          RentalRequest request = current.request;
240          if (validRequest(request) && request.game.availableCopies >= request.quantity) {
241              return request;
242          }
243      }
244      return null;
245  }
```

previewReadyRequests() copies requests and uses one copied game for each real game. Several requests for the same game therefore consume the same simulated stock. It runs the same fairness selector and updates only the simulated streak and stock. Admin queue positions, customer positions and receipt waiting counts use that predicted readiness. See CheckoutQueue.java:256-286, CheckoutQueue.java:213-215, TerminalUI.java:2628-2633.

3.4 Heap, stack and hash defense points

Actual source: Rental.java:30-42

```
30  public int compareTo(Rental other) {
31      int byDate = dueDate.compareTo(other.dueDate);
32      if (byDate != 0) {
33          return byDate;
34      }
35      if (rentalNumber < other.rentalNumber) {
36          return -1;
37      }
38      if (rentalNumber > other.rentalNumber) {
39          return 1;
40      }
41      return 0;
42  }
```

The heap compares due date first, then rental number to make equal-date ordering deterministic. getRentalsByDueDate() copies and polls the heap. A stack is useful for newest-first history/audit, but this project reads it by reverse index rather than popping. A LinkedHashMap supports lookup plus insertion order; it does not automatically sort by title or price. Password hashing is cryptographic protection, not the hash-table DSA used by the game map.

3.5 DSA in every module

MODULE	DSA ACTUALLY USED
Login	ArrayList<User> and linear identifier search; character replacement for safe logging. PBKDF2 verifies the password, including a dummy hash for unknown users.
Registration	Same ArrayList and linear duplicate search; sequential field state machine; character loops and regex validation. Field editing preserves saved draft variables.
Browsing / rental selection	Linear search, copied-list stable insertion sort, ArrayList results and arrays of page heights.
Cart / order	LinkedHashMap<Game,Integer> combines quantity and preserves order; list views and paged summaries.
Queue / VIP upgrade	Manual doubly linked queue, FIFO within groups, streak counter, insertion sort of arrival numbers, stock-map simulation.
Checkout / receipt	Queue selector; ArrayList session entries; counters; heap insertion; history stack push; paged row lists.
Returns / active rentals	Min-heap order, linear text search, boolean[] duplicate selection checks, selected ArrayLists and shared-game stock map.
History / audit	Stacks displayed newest first; ArrayList snapshots for pages; wrapped record heights.
Inventory	LinkedHashMap storage; linear duplicate title/platform check; search/sort; prefix counters for IDs.
Persistence / recovery	Maps of original file bytes, HashSets for uniqueness, record lists, insertion-sorted rebuilds and snapshot arrays.

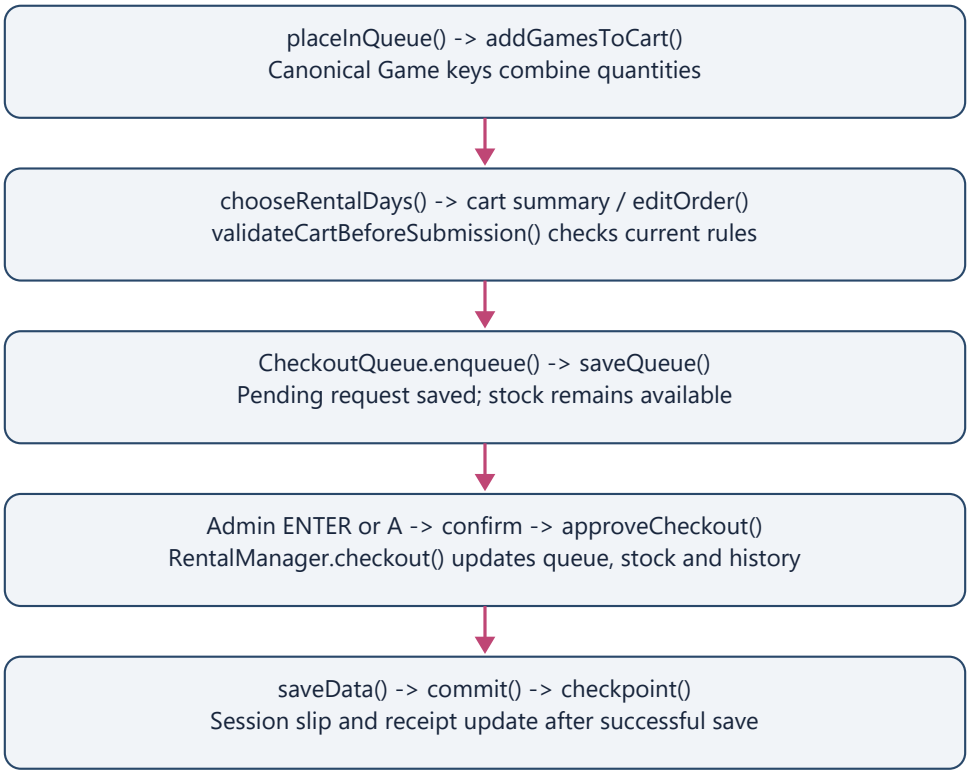
4. Important methods and transaction connections

Appendix A lists every explicitly declared method and constructor in all current application Java files. This chapter explains the most important call paths. Parameters and return types below follow the actual declarations; a null result commonly means no selection, cancellation or validation success, depending on the method.

METHOD / EXACT LOCATION	INPUTS AND RETURN	PURPOSE AND CONNECTIONS
Main.java main() Main.java:3-16	args: command-line arguments; void.	Creates Queue2Play and calls start(); handles input end and fatal exceptions.
Queue2Play.java start() Queue2Play.java:144-187	No parameters; void.	Loads accounts, games, rentals, queue and logs, checkpoints state, then dispatches the landing page and role dashboards.
AccountManager.java register() AccountManager.java:207-337	No parameters; void.	Maintains draft fields and step number; validates each field; creates a Regular client, logs, saves and shows the result.
AccountManager.java login() AccountManager.java:353-400	No parameters; User or null.	Finds username/email, masks password, verifies hash, records failures/cooldown or returns the authenticated account.
GameCatalog.java getDisplayList() GameCatalog.java:132-138	platformGames, keyword, sortOption; ArrayList<Game>.	linearSearch() builds the display copy; insertionSort() orders it without changing inventory.
Queue2Play.java placeInQueue() Queue2Play.java:833-938	client: User; void.	Creates a local cart, selects games and duration, edits/reviews, validates, enqueues one request per cart key and saves.
Queue2Play.java askQuantity() Queue2Play.java:542-544	game and inCart; int.	Convenience overload. Calls the three-parameter form with no preexisting queued copies. The three-parameter form returns a positive quantity or navigation sentinel.
RentalManager.java validateCart() RentalManager.java:212-239	client and cart; String.	Returns null when valid or the first exact error message. Checks account, entries, canonical game, quantities, queued totals, active rental and available stock.
CheckoutQueue.java findNextEligibleRequest() CheckoutQueue.java:229-245	No parameters; RentalRequest or null.	Uses VIP streak and current stock to choose the next whole request. Does not deduct stock.
CheckoutQueue.java previewReadyRequests() CheckoutQueue.java:256-286	No parameters; ArrayList<RentalRequest>.	Simulates the complete current checkout order using copied stock and streak; returns original request objects in predicted order.
Queue2Play.java runCheckoutSession() Queue2Play.java:1251-1313	session: CheckoutSession; void.	Routes ENTER/A/C/V/R/pages/exit. Successful entries print a slip and status; blocked/invalid actions only print message/input.
Queue2Play.java approveCheckout() Queue2Play.java:1417-1430	request: RentalRequest; Rental.	Calls RentalManager.checkout(), advances the record number, logs and saves. The saved result is added to session counters only after success.
RentalManager.java checkout() RentalManager.java:147-173	request, rentalNumber; Rental.	Validates, calculates cost/due date, deducts stock, adds active/all/history records, removes queue request and records the successful VIP/regular turn.

METHOD / EXACT LOCATION	INPUTS AND RETURN	PURPOSE AND CONNECTIONS
Queue2Play.java recordAdminCancellation() Queue2Play.java:1402-1414	request: RentalRequest; void.	Creates a zero-charge CANCELLED history record and advances the shared history/rental counter. Caller removes queue request and saves.
Queue2Play.java parseReturnSelection() Queue2Play.java:1448-1477	rentals, start, end, input; ArrayList<Rental> or null.	Maps one or comma-separated page-relative numbers to active records; rejects duplicates, blanks and out-of-range values.
Queue2Play.java completeSelectedReturns() Queue2Play.java:1538-1563	selected rentals, returnedAt; void.	Validates the whole selection and combined restored stock, calls completeReturn(), logs each rental, then commits the batch once.
RentalManager.java completeReturn() RentalManager.java:196-209	rental, returnedAt; void.	Checks active status and dates, computes late days/fee, removes the heap entry, marks COMPLETED and restores full stock.
Queue2Play.java addGame() Queue2Play.java:1740-2014	No parameters; void.	Draft form and independent field editing; creates a unique title/platform game, generates ID, logs and saves.
RentalDataStore.java saveData() RentalDataStore.java:89-99	No parameters; void.	Calls serialized(), AppStorage.commit() and checkpoint(); restores the prior in-memory snapshot if a RuntimeException occurs.
AppStorage.java commit() AppStorage.java:145-207	contents: Map<String,String>; void.	Locks files, rejects stale bytes, writes the journal, replaces five files, marks committed, remembers bytes and cleans up.
TerminalUI.java pageStarts() TerminalUI.java:65-82	heights, maximum, reservedRows; ArrayList<Integer>.	Builds page start/end indices from record heights and the configured viewport. Last entry is the exclusive end sentinel.
TerminalUI.java showConfirmQuestion() TerminalUI.java:2397-2406	text: String; void.	Centers a checkout question between pink full-width separator lines and emits the pending message before it.

Customer order to approved rental



5. Complete user manual: start, login and registration

5.1 Starting the project

1. Install a Java JDK and open a terminal in the project folder. JDK 24 was used for the verified build in this report.

2. In PowerShell, run .\run.ps1. It compiles top-level .java files into .build, uses UTF-8, reads the terminal width/height and passes the project data folder to Main.

3. Keep the terminal open while operating the system. Type the displayed number or letter, then press Enter. For approval in the admin checkout tape, an empty Enter is itself an action.

4. Use the displayed Back destination, Dashboard or Main Menu option. Most letter keys ignore capitalization. Follow each screen: the same letter can serve a different purpose in a different context.

5. Use Q on the landing page to exit and answer the confirmation. L on either dashboard logs out after confirmation. A normal empty-result screen remains open until its navigation action is entered.

If the launcher is not used, compile with javac -encoding UTF-8 -d .build *.java and run Main with the correct data-directory and size properties. In PowerShell, use the source-list command shown in Chapter 11 rather than relying on shell wildcard expansion. The Java class containing the entry point is Main, not Queue2Play. See Main.java:1-17 and run.ps1:1-20.

TUI ELEMENT	WHAT IT MEANS
Numbered vertical menu	Select a module, such as Login, Rent Games or Inventory. Menu options have top/bottom padding.
Grouped choices	Actions inside a feature; centered with spacing and separate categories. Enter the bracketed key.
B / M	B returns to the named previous screen; M goes to Dashboard/Main Menu where implemented. A one-level screen can show only its direct return option.
N / P	Next/previous page on ordinary lists, receipts, cart summaries and result pages. Only usable page keys are shown.
F / P on confirmations	Next/previous preview page. N/NO remains the decline action. Y/YES confirms the entire preview selection.
Pink separator and >	The input label and place to type. Borders and prompts remain pink. Helpers are bold soft white; examples are bold gray.
Colors	Regular action keys mustard; navigation white; positive actions green; destructive actions red. Browse All Games violet; PS5 blue; Xbox green; Nintendo red. Delete Game in the inventory menu intentionally uses the regular-option style.

AVAILABLE uses the separate dark-green color #154734, while OUT OF STOCK uses mustard. This avoids confusing Xbox green with stock status. These are presentation choices and do not determine whether a transaction is allowed. See TerminalUI.java:373-421, TerminalUI.java:487-521, TerminalUI.java:1162-1165.

5.2 Built-in demonstration accounts

ACCOUNT	PASSWORD	ROLE / MEMBERSHIP
admin	admin123	Administrator / Regular membership field
alex	alex12345	Client / VIP
juan	juan12345	Client / Regular

These are code-defined demonstration accounts, not customer data extracted from storage. loadAccounts() creates missing built-ins and ensureEmbeddedAccount() restores their code-defined name, email, role and password at startup. Existing client VIP upgrades are retained. This restoration is an intentional current project behavior, as previously requested; it is not described as a new fix. Registered customer accounts are still saved and loaded. See AccountManager.java:107-127 and

AccountManager.java:130-160.

5.3 Login from start to finish

1. On the landing page, choose 1 Login.

2. Enter the account username or email. Surrounding spaces are trimmed; matching ignores capitalization. B returns to Main Menu. An empty identifier shows "Username or email cannot be empty." and asks again.

3. At PASSWORD, enter the exact password. Passwords are case-sensitive and are not trimmed. On Windows the PowerShell key reader masks characters with *. B returns to the identifier; C returns to Main Menu. An empty password shows "Password cannot be empty.".

4. If authentication fails, the system shows "Incorrect username/email or password." and the current attempt number. It does not reveal whether the username/email exists.

5. Attempt 1/3 leaves two attempts; attempt 2/3 leaves one; attempt 3/3 starts a 30-second countdown. Do not close the app to expect a saved cooldown: failures exist only in the running application. Empty fields and navigation do not count.

6. After the countdown, retry. Failures carry across identifier changes and menu returns. A successful login clears the failure counter and opens the matching administrator or client dashboard.

The countdown goes from 30 to 0 using carriage-return updates. q2p.fast disables cosmetic pauses, but waitLoginSecond() keeps the login wait. Failure logs record a sanitized identifier, never the password. Unknown users are checked against a dummy hash. See AccountManager.java:340-350, AccountManager.java:353-400, TerminalUI.java:1647-1663, TerminalUI.java:1632-1644 and TerminalUI.java:783-804.

5.4 Registration from start to finish

1. Choose 2 Register on the landing page.

2. Enter FULL NAME: 5-60 characters, starts with a letter, at least two letters overall, and only letters, ordinary spaces and dots. Example: Juan Cruz. This validator does not require two separate words.

3. Enter USERNAME: 3-20 ASCII letters/digits/dots/underscores/hyphens, with at least one letter. It must not collide with any saved username or email, ignoring case.

4. Enter EMAIL: at most 100 characters and the implemented email format. Example: juan@gmail.com. Consecutive dots and invalid domain labels are rejected. The system does not send a verification email.

5. Enter PASSWORD: 8-20 characters, no whitespace, control characters or |. Digits only, letters only or a mixture are accepted. Other allowed punctuation is not universally prohibited.

6. Re-enter CONFIRM PASSWORD exactly. A mismatch stays at confirmation; it does not replace a previously confirmed password during editing.

7. Review the summary. Choose E Edit Information to change one field: 1 Full Name, 2 Username, 3 Email or 4 Password. An accepted edit returns to the summary and preserves the other fields. B returns from the edit menu to the summary.

8. Choose 1 Create Account to save a CLIENT/REGULAR account. The account-created result remains visible until M Main Menu. Log in normally; registration does not automatically log in.

9. To stop before creation, use C Cancel Registration where displayed. Yes discards the unsaved draft and returns to Main Menu. No keeps the entered fields and current step. At the first empty name step, B can return directly because no draft has been entered.

B from a normal field returns to the previous field. In field-edit mode, B returns to Edit Information; from password confirmation it returns to PASSWORD. Input navigation is recognized before field validation. See AccountManager.java:207-337, TerminalUI.java:1578-1603, TerminalUI.java:1615-1629.

6. Complete user manual: Regular and VIP customers

DASHBOARD KEY	FEATURE
1	Browse Games
2	Rent Games
3	My Queue
4	Rental History
5	VIP Membership
L	Logout with Yes/No confirmation

6.1 Browse, search, sort and pages

1. Choose 1 Browse Games. Select 1 Browse All Games, 2 PS5 Games, 3 Xbox Games or 4 Nintendo Games. B returns to Dashboard.

2. The open table shows displayed number, game ID, title, platform, base price, available stock and stock status. At most nine games appear on a catalog page; long records or shorter terminals can reduce this.

3. Choose S to search. Enter a complete game ID, any part of the title or a complete platform name. Search ignores case and uses the whole selected platform list. An unrelated platform search within a selected platform gives no matches.

4. Choose T to cycle Default -> Title A-Z -> Title Z-A -> Price Low-High -> Price High-Low -> Default. Each change returns to page one.

5. Choose X Clear Search when a search is active. This restores the selected platform list, keeps the sort and returns to page one.

6. Choose N or P only when the corresponding page exists. B Select Platform returns to platform selection; M Dashboard returns to the dashboard.

7. Ordinary browsing is read-only. Displayed game numbers select games only in selection screens such as Rent Games or inventory operations.

No-match results remain visible with their choices; they are not skipped to Dashboard. Search/sort do not change saved stock or inventory order. See Queue2Play.java:214-334, GameCatalog.java:132-138, TerminalUI.java:1953-1992.

6.2 Place an order from start to finish

1. Choose 2 Rent Games on the client dashboard, then choose a platform or All Games.

2. Search, sort or page if needed. Enter a number currently displayed beside a game. It is page-relative, not the game ID.

3. Enter the whole-copy quantity. It must be at least one, fit current available stock and keep queued copies plus cart copies at or below three for that same game. A game already actively rented by this client cannot be added.

4. Answer whether to add another game. Y returns to the currently selected platform list. Re-selecting a game adds to the existing cart line if the remaining quantity permits it. B Select Platform changes the platform while retaining the cart.

5. When the cart is nonempty, F Finish Selection is available from game/platform selection. This lets you proceed without being forced to add another game. Answering N to add-another also finishes selection.

6. Choose rental duration: 1 means one day, 2 means three days, 3 means seven days, or 4 opens a custom whole-number input from 1 to 30 days. The chosen duration applies to the whole cart.

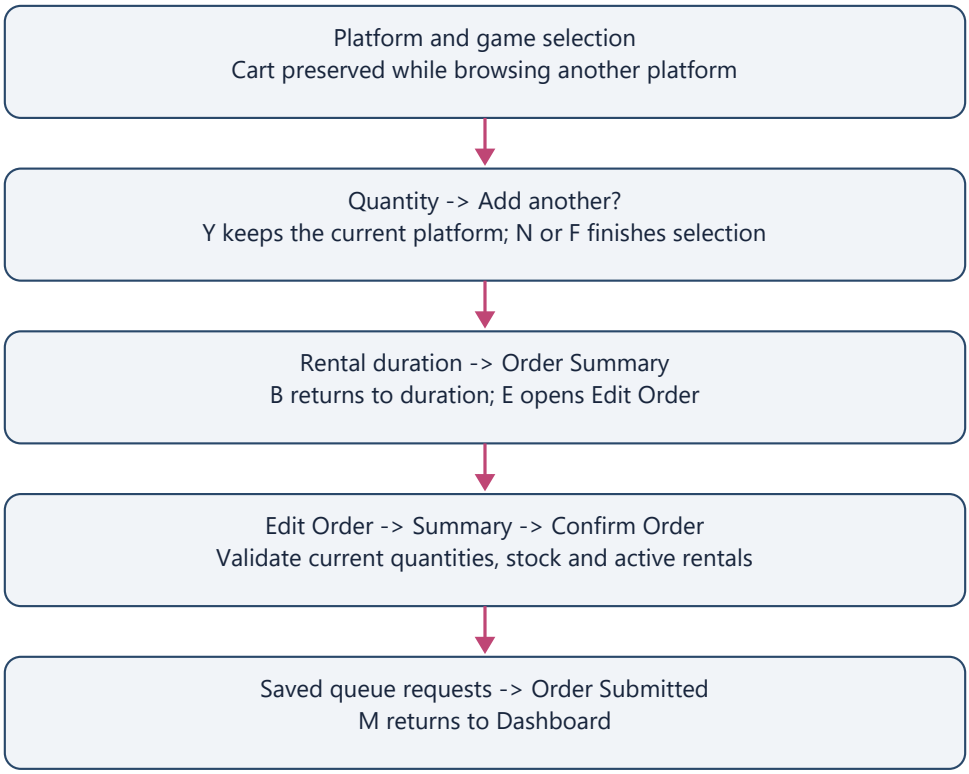
7. Review quantities, unit amounts, subtotals, membership and total. N/P pages keep all cart lines accessible. E opens Edit Order; B returns to Rental Duration.

8. In Edit Order, 1 adds games, 2 removes a whole cart line, 3 changes duration, and B returns to Order Summary. Removal uses the cart line number shown in the summary, which is numbered over the full cart. It does not remove only one copy.

9. Choose 1 Confirm Order. The cart is checked again against queued quantities, active rentals and current stock. If it fails, correct the order; no request is submitted.

10. Successful submission creates one queue request for each distinct game in that cart, saves it and shows Order Submitted. Stock remains unchanged. Use M Dashboard when finished.

Order form navigation



Across separate submissions, the same game can have separate request numbers. Example: one queued copy plus a later two-copy request is allowed if current stock fits and the client has no active rental. A fourth total queued copy is rejected. Requests from different clients do not reserve stock, so the sum of all waiting quantities can exceed available stock. See Queue2Play.java:599-635, Queue2Play.java:547-596, Queue2Play.java:833-938 and RentalManager.java:212-239.

6.3 Cost examples and what is charged

calculateRentalCost(client,game) applies the VIP discount to the three-day base price. The duration overload rounds the single-copy duration cost to two decimals. The quantity overload multiplies that rounded unit cost by quantity and rounds again. There is no payment gateway, cash tender or balance settlement module: the program records calculated amounts.

BASE PRICE / DURATION / QTY	REGULAR	VIP
$5.00 / 3 days / 2 copies	$10.00	$9.00
$5.00 / 1 day / 2 copies	$3.34 (2 x $1.67)	$3.00 (2 x $1.50)
$5.00 / 7 days / 1 copy	$11.67	$10.50

These are source-based worked examples, not screenshots of production transactions. The game price and current membership are read when the administrator actually approves the request. Queue records save duration and quantity, not a locked quoted price. Checkout starts at approval time and due date is approval time plus the chosen number of days. Formula overloads: RentalManager.java:58-65, RentalManager.java:68-72, RentalManager.java:75-78.

6.4 My Queue and cancellation

1. Choose 3 My Queue. Only your requests are displayed, but their position is the predicted global checkout position across all clients.

2. Read READY processing positions or WAITING when the current shared stock/fairness simulation cannot complete that request. Gaps between your positions are normal because other clients are between your requests.

3. Use N/P when offered. B Dashboard returns; C Cancel Request enters a queue-number prompt.

4. Enter your queue number as digits, such as 003, without #. A number belonging to another client is rejected. B My Requests leaves the cancellation prompt; M Dashboard exits.

5. Confirm Y/YES to remove the request or N/NO to leave it queued. Cancelled quantity does not deduct or restore stock.

6. If other requests remain, answer whether to cancel another; if the last request was removed, the result waits for M Dashboard.

Current difference: customer cancellation is logged as an EVENT and removed from the queue, but it does not create a CANCELLED rental-history record. Administrator cancellation does create that record. See Queue2Play.java:940-1055 and Queue2Play.java:1402-1414.

6.5 Rental history

1. Choose 4 Rental History. Newest record numbers appear first.

2. Read record number, game, quantity, recorded date, due date, amount and status. ACTIVE is an approved rental still checked out; COMPLETED is a processed return; CANCELLED is an administrator cancellation with no charge.

3. A completed rental also shows return time, days late and recorded late fee. CANCELLED shows no due date in the UI even though an internal date interval is used by the record model.

4. History pages have a maximum of three records and may show fewer when wrapping/height needs it. N/P navigate; B Dashboard returns. An empty history stays visible.

6.6 Upgrade to VIP

1. Choose 5 VIP Membership. A Regular client sees Free lifetime membership, 10% discount and the queue-priority description.

2. Choose 1 Confirm Upgrade. The membership changes to VIP; all existing requests of that client are reinserted in the VIP group using their original arrival numbers; the change is saved.

3. Use M Dashboard on the completed result. Choosing C before upgrade leaves membership unchanged. A current VIP sees an already-VIP message and the dashboard return option.

4. There is no downgrade option. Both Regular and VIP clients use the same rental interface; only discount and queue membership behavior differ.

6.7 Back, discard and error recovery

Back to another order step retains the cart. M Discard Cart / Dashboard confirms discarding a nonempty cart: Yes ends the unsaved transaction; No stays at the current step and keeps quantities and duration. F Finish Selection proceeds with the current cart. Cancelled unsaved drafts never become queue records. No does not undo an already saved transaction; it only declines the proposed action. See TerminalUI.java:1929-1945, Queue2Play.java:68-74 and Queue2Play.java:728-820.

For a quantity, duration or line-number error, read the message and retry the same field or return using its displayed destination. If a save error occurs, the checkpoint restores committed in-memory records; stop and resolve the reported storage issue before assuming anything was saved. A failed storage transaction can return you to a surrounding menu rather than preserve every unsaved UI draft.

7. Complete user manual: administrator

DASHBOARD KEY	FEATURE
1	Checkout
2	Process Returns
3	Inventory
4	Active Rentals
5	Audit Log
L	Logout with confirmation

7.1 Checkout queue and one request

1. Log in as an administrator and choose 1 Checkout. The queue shows predicted processing positions, VIP/Regular membership, quantities, duration, simulated stock and READY/WAITING status.

2. Review NEXT. This is the next request with enough stock under the current fairness rule. A waiting head request can be skipped for a later eligible request.

3. Press Enter with no typed key to approve NEXT. The separated question shows queue number, customer, game, quantity and cost. Y/YES approves; N/NO declines without changing queue or stock.

4. An invalid confirmation answer shows "ERROR: Use Y or N." then repeats the question and prompt. It does not repeat a result slip or session status.

5. After successful approval, stock is deducted, an ACTIVE rental is created, history is pushed, the request is removed, the fairness streak is recorded and the files are saved. A numbered APPROVED slip, then session status/NEXT, then choices appear.

6. If nothing is ready, Enter shows "WARNING: No request is ready." plus choices only. It does not increment the session counters or print a new status/slip.

7. V Queue redraws the queue list. N/P move between queue pages. R Receipt opens the session receipt. B Dashboard leaves checkout; M also works as a dashboard shortcut.

CHECKOUT KEY	EXACT CURRENT ACTION
ENTER	Approve next eligible request after confirmation
A	Preview and approve all currently predicted-ready requests
C	Cancel a queue number after confirmation
V	View/redraw queue
R	View current session receipt
N / P	Queue pages when available
B	Dashboard; receipt shown first if this session has entries

Slip numbers are per checkout session and include approvals and cancellations: #01, #02, #03. They are different from persistent rental and queue numbers. A successful action prints the slip(s) followed by one session box and the input choices. A blocked, declined or invalid action prints only messages/input. Action tape rendering does not clear the screen; queue and receipt viewing redraw their respective screens. See Queue2Play.java:1251-1313, TerminalUI.java:2337-2346, TerminalUI.java:2349-2356, TerminalUI.java:2359-2367.

7.2 Batch checkout

1. Choose A. If no request is predicted ready, the warning appears and no preview is approved.

2. Review the planned requests and their order. Use F Next Page and P Previous Page where offered. N/NO declines the whole batch; it is not the preview next-page key.

3. Choose Y/YES to approve the whole preview. Before each item, the code verifies that it is still the next eligible request.

4. Each approved rental produces one slip and is saved separately. The final tape status/input is printed once after the action. If an item fails or the queue changes, processing stops; earlier successfully saved approvals remain approved.

5. Use V to inspect the remaining queue or R for the session receipt.

Batch checkout is intentionally sequential, not one all-or-nothing commit. In contrast, multiple returns are saved as one batch. See Queue2Play.java:1321-1340, Queue2Play.java:1343-1361 and Queue2Play.java:1417-1430.

7.3 Administrator queue cancellation

1. Choose C Cancel # on the checkout tape.

2. Enter the queue number as digits, for example 011. Do not enter a game ID or the slip sequence. B Checkout returns; M Dashboard exits.

3. Read the Cancel question with customer, game and quantity. Y/YES confirms; N/NO leaves the request untouched.

4. Successful cancellation removes the pending request, creates a zero-cost CANCELLED history record for that customer, saves and prints the next numbered CANCELLED slip.

5. Stock, active rentals and VIP streak do not change. The session cancellation count increases only after save succeeds.

A cancellation history record uses the shared nextRentalNumber counter. Approved rental numbers can therefore skip numbers consumed by cancellation records. The receipt uses the cancelled queue number in the cancellation row and a separate STATUS column; it does not mix administrator/session metadata into rental rows.

7.4 Receipt navigation

1. Choose R. Administrator and session timestamp are separate details above the transaction records.

2. Read APPROVED/CANCELLED rows, approved count, copies, cancellation count, total and left-in-queue/waiting counts. Waiting uses the same shared-stock prediction as queue positions.

3. Use N/P for larger receipts. Totals describe the whole session, not just the current page. An empty session has zero transaction records.

4. B Checkout returns to checkout from an explicitly opened receipt. M Dashboard leaves. After checkout exit, the receipt can show only M because there is no continuing checkout screen to return to.

5. The current session receipt is not archived as a separate persistent receipt. Closing the session loses its aggregation, while saved rental/history records remain.

7.5 Process one or several returns

1. Choose 2 Process Returns. The list is earliest due date first, with rental number as the tie-breaker. It includes only ACTIVE rentals.

2. Use N/P to move pages. S searches a partial customer username/full name/game title or a complete game ID, ignoring case. X clears search. B Dashboard returns.

3. Enter one displayed page number or comma-separated numbers, such as 1,3,5. Numbers are current-page row numbers, not game IDs or persistent rental numbers. Duplicate numbers, blanks and out-of-range numbers are rejected.

4. The selected records are processed in their due-list order. Full recorded quantities are returned; there is no partial-return input.

5. Review individual due dates, late days, late fees and the selected total. F/P page a long confirmation. Y/YES confirms all selected returns; N/NO returns to the list without changes.

6. Confirmation captures the return timestamp when the preview opens. Validation checks every selected rental and the combined stock for shared games before modifying anything.

7. After confirmation, each rental becomes COMPLETED, its return date/fee is recorded, it leaves the active heap and all its copies return to available stock. One recoverable save commits the whole selected return batch.

8. Read processed results with N/P. B Return List is available only when active rentals remain; M Dashboard finishes. If none are active initially, the empty screen remains visible until its dashboard return action.

Late days are rounded up by started 24-hour periods after the exact due timestamp. Exactly on time has zero late days; a fraction of a second late counts as one day. Fee = $1.00 x late days x rental quantity. Example: two copies returned 25 hours after due produce two late days and a $4.00 recorded fee. Overdue rentals were removed as a separate browsing feature; late-fee calculation still works. See RentalManager.java:182-193, RentalManager.java:176-179, Queue2Play.java:1566-1586.

7.6 Inventory: browse and add a game

1. Choose 3 Inventory, then 1 Browse Inventory to inspect/search/sort the existing catalog. B returns to Manage Inventory where named; M goes to Dashboard in catalog screens.

2. Choose 2 Add Game. Enter a nonblank title of at most 100 characters, with no | or control characters.

3. Choose platform: 1 PS5, 2 Xbox or 3 Nintendo. This platform numbering differs from the catalog menu, which includes Browse All Games first.

4. Enter a finite price from $1.00 to $10.00 and a whole initial stock from 1 to 20. Price is the three-day base price.

5. Review Game Summary. E Edit Information lets you change 1 Game Title, 2 Platform, 3 Rental Price or 4 Available Copies without resetting other entered values. B Available Copies is a step-back option, not a second edit-all action.

6. Choose 1 to save. A case-insensitive duplicate title on the same platform is rejected; the same title on a different platform is permitted. The generated ID uses P/X/N with at least three digits.

7. On success, answer whether to add another game. No returns to Manage Inventory; Yes starts a new draft.

8. C Cancel Transaction or M Dashboard on draft screens asks whether to discard the entered game. No retains the draft and step. B moves to the named prior step.

7.7 Inventory: update stock

1. In Inventory choose 3 Update Stock. Select platform and then a displayed game number.

2. Choose 1 Add Stock or 2 Decrease Stock. Enter a positive whole quantity to add/subtract, not the desired final stock value.

3. Available stock cannot become negative. Available copies plus all currently rented copies of that game cannot exceed 20. Queued requests do not consume this limit.

4. Review previous and new available stock and confirm Y/YES. N/NO declines without saving.

5. After saving, choose whether to update more. C cancels toward Inventory and M exits to Dashboard with discard confirmation where draft input exists. B returns to the named earlier selection.

7.8 Inventory: delete a game

1. Choose 4 Delete Game in Inventory, then select platform and displayed game number.

2. If the game currently has an active rental, deletion is blocked with "Cannot delete this game because it currently has an active rental.".

3. Review game details and the count of pending requests. A warning states that deleting also cancels pending queue requests for that game.

4. Y/YES to Permanently delete this game removes the catalog game and its pending requests, logs the action and saves. N/NO retains everything.

5. Completed/cancelled historical rentals remain in history. Requests removed by deletion do not receive individual CANCELLED history records. Choose whether to delete another game.

7.9 Active rentals, audit and logout

1. Choose 4 Active Rentals, then the available active-rental list option. Rentals appear earliest due first, up to nine records per page or fewer to fit the configured viewport. The separate Overdue Rentals filter is absent.

2. Use N/P when usable; B returns to Active Rentals from the list and M Dashboard can exit the deeper view. Empty results still show their navigation.

3. Choose 5 Audit Log for administrator actions, most recent first. Wrapped long actions stay together on a page. Login, registration and customer EVENT entries remain in logs.txt but are not included in the ADMIN-only audit stack.

4. Use N/P and B Dashboard on Audit Log. There is no audit search/export UI.

5. Choose L Logout on the dashboard and confirm. Logout returns to the landing page; it does not delete saved accounts or transactions.

8. Regular and VIP queue behavior

8.1 Eligibility, position and the three-VIP rule

A valid request has a non-admin client, game, positive queue number, quantity 1-5 under the saved-record rule and duration 1-30 days. findNextEligibleRequest() requires enough currently available copies for its entire quantity. After three successful VIP checkouts, it first looks for an eligible regular request. If none exists, it still allows an eligible VIP request and keeps the streak at three. A regular checkout resets the streak to zero. Merely viewing, previewing, declining or cancelling does not count as checkout.

The fairness turn is for a request, not a customer or copy. One five-copy legacy VIP request counts as one VIP checkout. Multiple requests belonging to one VIP can therefore occupy consecutive turns. FIFO is preserved within the membership group among requests that are currently eligible; a stock-blocked earlier request can be skipped.

CURRENT SITUATION	PREDICTED RESULT
V1, V2, V3, V4 and R1; streak 0; enough stock for all	V1 -> V2 -> V3 -> R1 -> V4.
Same requests; streak already 2	V1 -> R1 -> V2 -> V3 -> V4.
Streak 3; regular request needs 3 but has stock 2; VIP fits	The regular stays WAITING. VIP proceeds; streak remains capped at 3.
VIP needs 3, stock 2; later regular needs 1 and fits	Regular can proceed even before the three-VIP boundary because the VIP cannot be fulfilled.
Two same-game requests each need 2; stock 3	First eligible request is READY and consumes 2 in the preview. The second is WAITING because simulated stock is 1.
Request needs 2 and stock is exactly 2	Entire request is READY; preview stock becomes 0.
All requests need more than available stock	No NEXT is ready; positions are WAITING; approval does not change counters.
Two customers view their own requests	Each sees the same global predicted position used by the administrator. Positions can have gaps in each filtered customer list.

Examples use named requests rather than actual private records. Ready positions are numbered 1, 2, 3 in predicted processing order. Waiting requests appear after ready requests and retain their stored priority/arrival ordering. Internally getPosition() returns zero when not found in the ready preview; the customer display renders that as WAITING rather than a valid position zero. See CheckoutQueue.java:218-226, CheckoutQueue.java:213-215, TerminalUI.java:2087-2096.

8.2 Shared stock and recalculation

1. Build copied Game objects, one per original game, with current available copies.

2. Copy requests and VIP streak into a simulation queue.

3. Select the next eligible whole request using the same fairness selector.

4. Reduce the copied game stock, remove that copied request and record its simulated VIP/regular turn.

5. Map the result to the original request by queue number and repeat until none fits.

6. Use this ready list for positions, READY/WAITING labels and receipt waiting count. Never deduct real stock while calculating a position.

Restocking, returns, cancellations, approvals and upgrades affect the next display calculation. Position is a current prediction, not a permanent reservation or guaranteed checkout time. The customer has no way to approve a request directly. Queue records persist the VIP streak so a restart does not reset the fairness turn.

8.3 VIP upgrade and different cancellation paths

ACTION	WHAT CHANGES	WHAT DOES NOT CHANGE
Regular -> VIP	User membership; all client request VIP flags; physical group insertion; price calculations after upgrade.	Arrival queue numbers, quantities, days, available stock and VIP streak.
Customer queue cancellation	Queue request removed; EVENT log saved.	Stock and VIP streak; no new rental-history cancellation record.
Administrator cancellation	Queue request removed; ADMIN log; CANCELLED history record; session cancellation count/slip.	Stock, active heap and VIP streak; no rental cost.
Delete catalog game	Catalog entry and all pending requests for that game removed; ADMIN deletion log.	Active rental deletion is blocked; completed/cancelled history remains; no per-request history cancellation.
Successful checkout	Stock reduced; request removed; ACTIVE/all/history entry; streak/counter/log updated.	Other request quantities/days and unrelated game stock.
Successful return	Full copies restored; heap entry removed; rental COMPLETED with return/fee fields; log updated.	Queue requests are not automatically approved, and VIP streak is unchanged.

Actual source: CheckoutQueue.java:63-71

```
63  void recordCheckout(RentalRequest request) {
64      if (request.vip) {
65          if (vipStreak < VIP_BURST) {
66              vipStreak++;
67          }
68      } else {
69          vipStreak = 0;
70      }
71  }
```

9. Validations and error handling

Validation exists at three layers: input forms, business transactions, and saved-file/state checks. The first failed condition is normally reported. A Java exception message can be wrapped with a file/line context or save prefix. TerminalUI.error() adds "ERROR: "; warning() adds "WARNING: ". Appendix B preserves exact literal messages or original dynamic Java expressions, their conditions/context, methods and lines.

9.1 Login and registration validation

FIELD / CONDITION	CURRENT RULE	RESPONSIBLE SOURCE
Login identifier	Trim surrounding spaces; nonempty; case-insensitive username OR email. B/0 navigate before authentication.	AccountManager.java:353-400; AccountManager.java:178-190.
Login password	Nonempty, case-sensitive, untrimmed. Incorrect account/password uses one generic message. B/C/0 are navigation inputs.	AccountManager.java:353-400; PasswordSecurity.java:53-65.
Failure count	Three consecutive actual authentication failures; wait 30 seconds every third failure. Successful login clears counter. Empty fields/navigation do not count.	AccountManager.java:340-350; TerminalUI.java:1632-1644.
New full name	5-60 Java characters; starts with Character.isLetter; at least two letters; only letters, ordinary spaces and dots. No word-count or real-person check.	AccountManager.java:35-49.
New username	3-20 ASCII characters from letters/digits/_.-; at least one letter; duplicate identifiers rejected case-insensitively.	AccountManager.java:52-62; AccountManager.java:207-337.
New email	Max 100; regex local/domain/TLD; local cannot start/end with dot; no double dots; domain labels <=63, not empty or leading/trailing hyphen.	AccountManager.java:65-82.
New password	8-20; reject whitespace, Unicode space characters, ISO control characters and |. Digits-only or letters-only accepted.	AccountManager.java:85-97.
Confirmation	Exact equality to pendingPassword; mismatch retries. Existing confirmed password is kept until replacement confirmation.	AccountManager.java:207-337.
Saved accounts	Older limits intentionally differ: username 3-32, name 2-100 with some extra punctuation/marks, email up to 254. Role/membership and identifier uniqueness also checked.	AccountManager.java:19-21; AccountManager.java:24-26; AccountManager.java:29-31; AccountManager.java:107-127.

9.2 Rental, quantity and stock validation

CONDITION	RULE AND RESPONSE SOURCE
Adding a game	Reject active rental, total queued+cart >=3, zero stock or cart already consuming all available copies. Queue2Play.java:522-539.
Quantity input	Whole number >=1; cap is min(3 - queued - cart, stock - cart). Reject decimal, text, negative, zero and excessive quantity. Queue2Play.java:547-596.
Duration	Default 3; options 1,3,7 or custom 1-30 whole days. Back/cancel/dashboard sentinels are separate from valid days. Queue2Play.java:643-720.
Before queue submission	Client required, nonempty cart, canonical game, each quantity 1-3, queued+cart <=3, no active rental and quantity <= current stock. RentalManager.java:212-239.
Checkout	Request still exists by identity; valid fields; canonical catalog game; active+requested <=5 compatibility cap; finite game price 1-10; whole quantity fits stock. RentalManager.java:122-144.
Checkout numbers	Positive unused rental number; queue/history counters cannot be exhausted; amount/due calculation must succeed. RentalManager.java:147-173; Queue2Play.java:1417-1430; Queue2Play.java:1402-1414.
Return selection	Nonempty current-page list; valid whole row numbers; no repeated row; current active record; no repeated rental object. Queue2Play.java:1448-1477; Queue2Play.java:1538-1563.
Return dates/stock	Return time not before checkout; restored copies <=20, including cumulative restoration of selected same-game rentals. Double return rejected. RentalManager.java:196-209.
Inventory title/platform	Title nonblank <=100, no | or control character. Platform exactly PS5/Xbox/Nintendo. Same title/platform duplicate rejected ignoring title case. GameCatalog.java:45-51; Queue2Play.java:1730-1737.
Price/stock	Finite $1-$10 price. Game model accepts available stock 0-20; Add Game requires initial copies 1-20. Stock change positive; result >=0 and result+rented <=20. GameCatalog.java:45-51; RentalManager.java:44-55.
Delete game	Any active rental blocks deletion. Pending requests are warned and removed only after Yes. Queue2Play.java:2017-2071.

9.3 Exact common diagnostics

login() messages - AccountManager.java:353-400

EXACT JAVA MESSAGE EXPRESSION	LINE
"Username or email cannot be empty."	AccountManager.java:362
"Password cannot be empty."	AccountManager.java:378

register() messages - AccountManager.java:207-337

EXACT JAVA MESSAGE EXPRESSION	LINE
"Use 1 to create, E to edit, or C to cancel."	AccountManager.java:231
"Choose 1, 2, 3, 4, or B to return to the summary."	AccountManager.java:245
"Use 3-20 letters, digits, dots, underscores or hyphens, with at least one letter."	AccountManager.java:288
"Username is already taken. Please try another one."	AccountManager.java:292
"Enter a valid email of at most 100 characters. Example: juan@gmail.com"	AccountManager.java:298
"Email is already registered. Please use another email."	AccountManager.java:302
"Use 8-20 characters without whitespace, control characters or |."	AccountManager.java:308
"Passwords do not match. Please try again."	AccountManager.java:317

validateRentalSelection() messages - Queue2Play.java:522-539

EXACT JAVA MESSAGE EXPRESSION	LINE
"You are already renting this game."	Queue2Play.java:526
"You already have the maximum of " + MAX_COPIES_PER_GAME + " copies of this game queued or in your cart."	Queue2Play.java:529
"This game is out of stock."	Queue2Play.java:533
"Your cart already contains all available copies of this game."	Queue2Play.java:536

validateCheckout() messages - RentalManager.java:122-144

EXACT JAVA MESSAGE EXPRESSION	LINE
"This queue request no longer exists."	RentalManager.java:124
"Invalid request client, duration or quantity."	RentalManager.java:127
"This game is no longer in the catalog."	RentalManager.java:130
"This checkout would exceed " + Queue2Play.MAX_SAVED_COPIES_PER_GAME + " active copies of this game for the client."	RentalManager.java:133
"Invalid game price."	RentalManager.java:138
"Not enough stock to process this checkout."	RentalManager.java:141

validateCart() messages - RentalManager.java:212-239

EXACT JAVA MESSAGE EXPRESSION	LINE
"A client account is required."	RentalManager.java:214
"Your cart is empty. Add a game before submitting."	RentalManager.java:217
"This game is no longer in the catalog."	RentalManager.java:221
"Invalid quantity for " + game.title + ". Maximum is " + Queue2Play.MAX_COPIES_PER_GAME + "."	RentalManager.java:225
"Maximum queued quantity is " + Queue2Play.MAX_COPIES_PER_GAME + " copies for " + game.title + ", including this cart."	RentalManager.java:228
"You are already renting " + game.title + "."	RentalManager.java:232
"Not enough stock for " + game.title + ". Available: " + game.availableCopies + "."	RentalManager.java:235

The quoted Java expressions above show the exact literal content; concatenated values are substituted at runtime. For all other menus, confirmations, storage constraints and exception wrappers, see Appendix B. It also includes diagnostic sites that forward a previously returned validation message.

9.4 Error handling and safe retry

Invalid numbers are caught as NumberFormatException and reported instead of treating them as valid choices. A malformed saved record becomes StorageException with filename and source-file data line context. An ended input becomes InputEndedException. Main reports fatal startup/storage failures and exits with code 1. Dashboard transaction handlers catch certain StorageException/IllegalArgumentException failures, report them and return to the controlling loop. There is no broad promise that every unexpected programming exception is converted into a friendly message.

State validation protects duplicate identifiers, canonical references, counters, active heap membership, status/date/fee fields and stock totals. Saved queue loading can accept a quantity above current stock because a waiting request is allowed; new order placement still requires its own quantity to fit available stock. The distinction prevents incorrectly treating every waiting record as corrupt.

10. File storage, updates and recovery

10.1 File formats

FILE	CURRENT FIELD ORDER / COUNT
accounts.txt	6: username | full name | email | password hash | role | membership.
games.txt	5: game ID | title | platform | price | available copies. #COUNTERS stores PS5, Xbox and Nintendo ID counters.
rentals.txt	13: rental/history number | username | game ID | title | platform | checkout date | due date | amount | status | quantity | return date | days late | late fee. #COUNTERS stores next record number.
queue.txt	6: queue number | username | game ID | VIP boolean | rental days | quantity. #COUNTERS stores next queue number and VIP streak.
logs.txt	3: timestamp | type | action. Current timestamp format yyyy-MM-dd HH:mm:ss. Message parsing uses a split limit of 3, so an action may contain |.

Files use UTF-8. Blank lines and normal # comments are skipped; #COUNTERS is processed as data. Current rental timestamps use LocalDateTime.toString(). Empty return-date fields represent no recorded return. User/game links are resolved to the loaded canonical objects. Field limits and forbidden delimiters prevent accidental record splitting. See RentalDataStore.java:40-59 and RentalDataStore.java:112-158.

10.2 Locate and load data

1. AppStorage selects -Dq2p.dataDir if supplied, otherwise Q2P_DATA_DIR, otherwise a data folder beside the compiled class location. run.ps1 explicitly passes the project data folder.

2. Create/open the folder and briefly acquire .lock. Recover an interrupted journal. With no configured directory and no saved files, supported old flat-file records can be copied from the parent directory.

3. Queue2Play.start() loads accounts first, then games, rentals, pending queue and logs. The service constructors wire state; they do not themselves perform this complete startup load.

4. Missing accounts are supplemented with built-ins. Missing games load starter games. Missing rental/queue/log files contribute no records. A present empty games file remains an empty inventory; starter games are not forcibly re-added.

5. Parse and validate every record. Rebuild the active heap and user history stacks; sort request arrival numbers and re-enqueue by current membership.

6. Checkpoint the loaded state. If built-ins changed or required account/game/rental/queue files were missing, save the normalized state.

Running java -cp .build Main without q2p.dataDir can use .build/data, rather than the project data directory used by run.ps1. Always pass an explicit data path when switching launch methods. A missing logs file alone does not trigger the startup missing-file save condition; a later write will create it. See AppStorage.java:16-51 and Queue2Play.java:144-187.

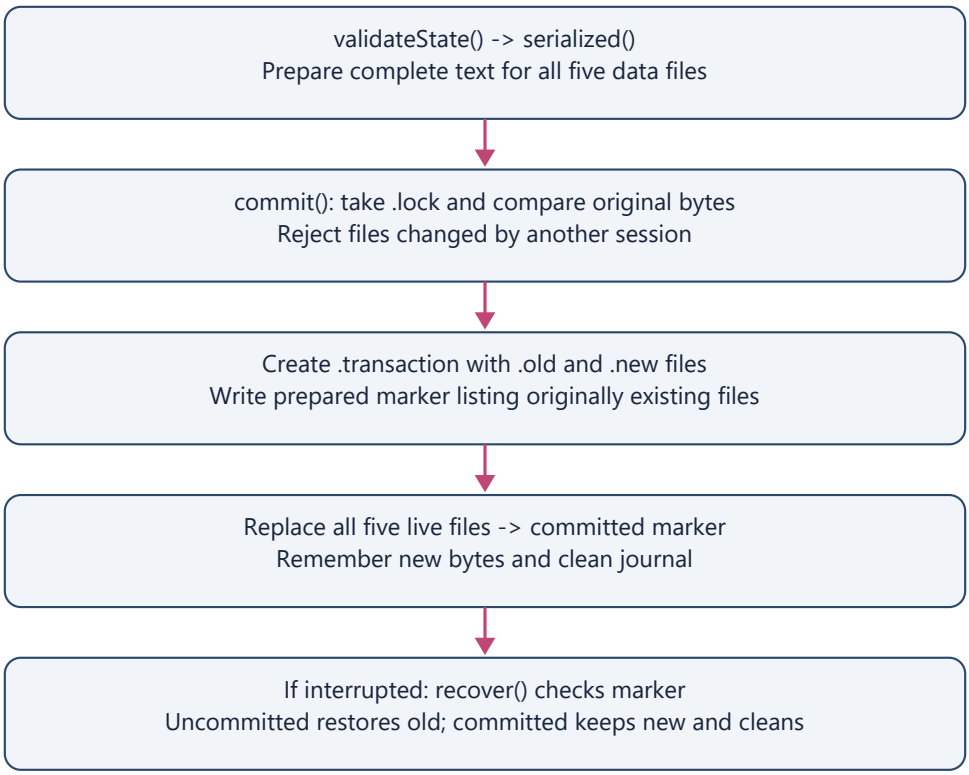
10.3 Backward compatibility

OLDER INPUT	LOAD BEHAVIOR
Accounts	Saved-account validators allow earlier field limits. Plaintext legacy passwords are converted by the User constructor to a salted hash before later save. Current serialization requires a recognized hash.
Rental 9/10-field record	9 fields default quantity to one; 10 adds quantity; 13 includes returns/late fields. Older date-only values are converted to a start-of-day checkout or 23:59 due time.
Queue 4/5-field record	Default rental days is three and quantity one. Later fields can supply days and quantity. Stored true/false is validated, but request priority is built from the loaded user membership.
Old quantities	Saved quantities up to five remain accepted; new forms use a three-copy limit. Combined saved queued+active copies per customer/game are checked against five.
Deleted historical game	Non-ACTIVE history may reconstruct a Game from its recorded ID/title/platform with placeholder price 1 and stock 0. ACTIVE references to missing games are rejected.
Log records	Current timestamp/type/action records and supported earlier date/time layouts are loaded. Administrator entries rebuild the newest-first audit stack.

Backward compatibility does not silently forgive malformed data. Duplicate numbers, unknown clients, missing active games, invalid dates/amounts/fees and inventory inconsistencies stop loading with a contextual error. Previously cancelled requests are not reconstructed into history. See RentalDataStore.java:253-333, RentalDataStore.java:402-412, RentalDataStore.java:349-399 and RentalDataStore.java:426-448.

10.4 Recoverable five-file save

Normal save and crash recovery



durableWrite() forces temporary file bytes to disk. replace() requests atomic replacement and falls back when the filesystem does not support it. Before committed exists, recovery copies original .old contents back for previously existing files and deletes newly created live files that were absent before the save. It copies rather than moves old files so an interrupted recovery can be retried. After committed exists, recovery retains the new files and cleans the journal.

AppStorage remembers the bytes originally loaded. At each commit it compares current file bytes against that snapshot. If another app or editor changed a file, it refuses to overwrite stale data with "<file> changed outside this session. Restart to reload it." The .lock is held for recovery/save operations, not for the full application lifetime. Two copies can therefore open at different times, but concurrent saving is locked and later stale saving is rejected.

There is no permanent .txt.bak creation in current source. Temporary .old files inside .transaction are recovery work files, not a retained backup archive; successful saves remove them. Existing old .txt.bak files are not part of the current five-file load/save set. Do not delete .transaction while a save or unresolved recovery is in progress. There is no built-in permanent backup browser or restore command.

10.5 In-memory rollback and save boundaries

checkpoint() captures the current collections and mutable transaction fields. Snapshot.restore() restores accounts, games, rentals, queue, audit/log lists, stocks, membership/passwords, rental return/status/fee fields, counters and VIP streak after a failed save. It uses copied collection contents and parallel field arrays; it is not a general deep-clone utility for every future field. Any new mutable field added later must be included explicitly if rollback should restore it.

TRANSACTION	SAVE BOUNDARY
Register / upgrade / inventory changes	One saveData() commit through their named save helper; all five files are serialized together.
Submit multi-game order	All new requests are added before one saveQueue()/saveData() commit. Stock is unchanged.
Single checkout / admin cancellation	Business mutation is saved before session counters/slip entry are added. Failed save restores the prior checkpoint.
Approve batch	One save per rental; previously committed approvals survive a later failure.
Multiple returns	Validate whole selection, mutate each selected rental, then one commit. Failure restores the whole selection.
Login attempt	Successful/failed authentication logs are saved. A failed password is never included in log text.

The journal covers process-interruption recovery with file-force calls; the code does not claim database-level guarantees against every disk, operating-system or power-loss failure. A conditional post-commit error risk is documented in Chapter 13. See AppStorage.java:90-98, AppStorage.java:101-107, AppStorage.java:125-142, RentalDataStore.java:89-99 and RentalDataStore.java:514-559.

11. Developer and operational guide

11.1 Build and test commands

Run from the project root in PowerShell. The following test commands follow Backfiles testing/tests/README.md, using a fresh disposable output directory. The full suite never loads production data. These commands are operational instructions, not edits to application source.

```
$sources = Get-ChildItem -LiteralPath . -Filter '*.java' | Select-Object -ExpandProperty
FullName

$testBuild = Join-Path (Get-Location) ('test-run-' + [guid]::NewGuid())

New-Item -ItemType Directory -Path $testBuild | Out-Null

javac -encoding UTF-8 -d $testBuild $sources 'Backfiles
testing/tests/BehaviorRegressionTest.java'

java "-Djava.io.tmpdir=$testBuild" -cp $testBuild BehaviorRegressionTest
```

The build used for this document invoked the same source/test compilation through the installed JDK 24 executable and used tmp/pdfs/queue2play-complete-20261012/test-build as the disposable build directory. The first sandboxed run failed at a Windows file replacement; the permission-enabled rerun passed all 63,287 checks. No production data was used.

11.2 Supported runtime settings

SETTING	PURPOSE
q2p.dataDir / Q2P_DATA_DIR	Explicit storage directory; useful for isolated demonstrations. The JVM property takes precedence.
q2p.width / q2p.height	Configured viewport; minimum width 40 and height 20. Defaults use COLUMNS/LINES if valid, otherwise 120/50.
q2p.ansi	Cursor clearing/positioning support; default follows System.console() availability.
q2p.ascii=true	Simpler terminal rendering when box-drawing characters are unavailable.
q2p.color=false / NO_COLOR	Disable ANSI color formatting.
q2p.fast=true	Skip cosmetic animations/pauses; does not remove the login cooldown wait.
q2p.test.failAfterReplace	Testing only: inject a save failure after a selected replaced-file count.
q2p.test.crashAfterReplace	Testing only: halt a process with exit code 91 after a selected replaced-file count. Do not enable in normal operation.

Terminal dimensions are read once when TerminalUI is constructed; the Java UI does not continuously query a resized window. Restart with the desired dimensions when testing a new viewport. Password masking requires an interactive terminal: the Windows path uses PowerShell ReadKey; a non-Windows console uses Console.readPassword(); without a suitable console it reports an error rather than exposing a password through Scanner. See TerminalUI.java:47-54, TerminalUI.java:56-62, TerminalUI.java:807-848.

11.3 How to trace a feature

1. Find its menu renderer in TerminalUI, then its controller method in Queue2Play or AccountManager.

2. Follow the validation call before the mutation. Example: order submission -> validateCartBeforeSubmission -> RentalManager.validateCart.

3. Identify the authoritative structure: games map, checkout linked list, active heap, allRentals list or user history stack.

4. Locate the business update and save call. Understand whether one item or the whole selection is the commit boundary.

5. Find the corresponding test group. Use an explicit isolated data path and disposable build; never point test execution at the live data folder.

6. If changing source later, preserve invariants and update exact-line documentation after the change. The line references in this report describe this snapshot only.

11.4 Core invariants for future maintenance

INVARIANT	WHY IT MATTERS
Game IDs and account identifiers are stable/unique	Requests/rentals resolve back to the canonical game and user. Display position is never identity.
Only successful checkout consumes stock and changes streak	Browsing, queue submission, preview and cancellation cannot silently allocate stock or count as VIP service.
Active rentals match ACTIVE records in allRentals	The heap, stock totals and customer history must describe the same transactions.
Queued VIP flags match current client membership	Membership upgrades must refresh queue placement and saved priority.
Saved stock respects available + rented <=20	Restocks and shared-game returns cannot create excess inventory.
Counter exceeds all current record numbers	Deleted games/cancelled requests do not reuse persistent identity numbers.
History references are retained when completed game is deleted	Historical data remains readable without allowing missing active games.
UI helpers do not mutate business records	Paging/search/rendering must not affect stock, queue order, rental amounts or VIP streak.

12. Testing and DSA defense reviewer

12.1 Tests actually executed for this report

CHECK	ACTUAL RESULT	LIMITS
Compile all 16 application files plus current test class	PASS: javac -encoding UTF-8; JDK 24; exit code 0.	Built only into a disposable test directory.
Full BehaviorRegressionTest, initial sandbox run	FAILED: Windows denied a journal .new -> live-file replacement during batch tape testing.	Environment-specific file access failure; this run did not establish a business-rule defect.
Same compiled full suite, permission-enabled rerun	PASS: 63,287 checks; exit code 0.	Uses isolated synthetic data and scripted TerminalUI subclasses, not production data.
Real cooldown test	PASS: Real cooldown lasted 30 seconds; exit code 0.	Exercises actual elapsed wait with q2p.fast=true; it does not visually inspect a live terminal.
Interactive password mask and live terminal appearance	NOT MANUALLY VERIFIED in this documentation task.	Scripted tests cannot validate PowerShell ReadKey capture or every terminal font/ANSI implementation.
Actual abrupt process crash / simultaneous external writer	NOT EXECUTED in this documentation task.	Recovery/stale-write explanations are source-based. Injected save-failure rollback is covered by the suite.

The check count includes many assertions about widths, pages and state; it is not 63,287 independent user journeys. The current test file is Backfiles testing/tests/BehaviorRegressionTest.java, with main at lines 2890-2996. README commands for targeted groups are available, but this task ran the complete suite and the real-cooldown mode only; it did not run each targeted mode separately.

12.2 Verified regression coverage

TEST GROUP / LOCATION	INPUT OR SCENARIO	EXPECTED AND VERIFIED IN FULL SUITE
accountValidationTests BehaviorRegressionTest.java:278	Name/username/email/password boundaries; allowed/rejected characters.	Validation accepts supported values and rejects invalid boundaries.
registrationTests BehaviorRegressionTest.java:321	Valid draft, duplicates, mismatch and save/reload.	Create a saved Regular client; do not save invalid fields.
legacyAccountTests BehaviorRegressionTest.java:375	Earlier saved account formats.	Load compatible fields and retain account integrity.
loginTests BehaviorRegressionTest.java:458	Known/unknown accounts, exact password, failures.	Generic failed authentication; three-attempt wait; success reset.
accountNavigationTests BehaviorRegressionTest.java:401	Back/cancel/edit form flows.	Navigation does not count failures or reset unrelated draft fields.
catalogTests BehaviorRegressionTest.java:879	Partial title/full ID/platform; all sort modes and ties.	Correct results; stable ordering; underlying inventory unchanged.
orderStockTests BehaviorRegressionTest.java:965	Zero stock, equal stock and excessive quantities.	Reject unavailable/excessive requests; do not reserve on queue submit.
repeatQueueTests BehaviorRegressionTest.java:1038	Repeated same-game submissions and three-copy total.	Allow remaining quota; reject fourth queued copy; preserve active block.
legacyQuantityTests BehaviorRegressionTest.java:1142	Previously saved quantities up to five.	Compatible load and checkout while new-order limit remains three.
queueTests BehaviorRegressionTest.java:1504	VIP/regular order, stock blocking and streak.	Correct eligible request; eligible regular turn after three VIPs.
predictedQueueTests BehaviorRegressionTest.java:1213	Shared-game stock, positions and recalculation.	Admin/customer predictions agree; simulation leaves real state unchanged.
returnTests BehaviorRegressionTest.java:1371	Single/comma selection, duplicate/errors/search/fees.	Full stock restoration, due order, no double return and correct late fees.
checkoutSessionTests BehaviorRegressionTest.java:1776	Approve/cancel/session totals and state.	Saved business records match session entries after successful operations.
tapeFlowTests BehaviorRegressionTest.java:1973	Blocked Enter, action order and five-item batch.	No repeated status/slips on retry; one slip per success and final status.
confirmQuestionTests BehaviorRegressionTest.java:2026	Y/N/YES/NO and invalid answers.	81-character separators/question line; invalid message then question.
transactionPaginationTests BehaviorRegressionTest.java:2533	Large receipts/previews/results at several viewports.	All records accessible; usable navigation and prompt fit tested cases.
separatedTransactionTests BehaviorRegressionTest.java:2653	Receipt metadata, cancellation STATUS, cart/return pages.	Details separate from record rows; empty receipt has zero data records.
responsiveListTests BehaviorRegressionTest.java:2251	Catalog/audit wrapping and height budgets.	Record boundaries retained; maximum catalog nine; smaller pages when needed.
discardConfirmationTests BehaviorRegressionTest.java:2392	Decline discard on game, stock, cart and registration.	No preserves drafts/current step; Yes discards only unsaved work.
gameEditingTests BehaviorRegressionTest.java:1597	Edit one new-game field and save/reload.	Other fields retained; duplicate title/platform rejected.

TEST GROUP / LOCATION	INPUT OR SCENARIO	EXPECTED AND VERIFIED IN FULL SUITE
completedResultTests BehaviorRegressionTest.java:1673	Submitted order, upgrade, cancellations, return/account result.	Results stay visible; correct Back/Dashboard destinations.
rentalDisplayAndCancellationTests BehaviorRegressionTest.java:2783	Nine-row active list, admin cancellation history and save failure.	Cancel No/Yes/restart; zero charge; no stock/streak effect; rollback.
choiceAndCooldownLayoutTests BehaviorRegressionTest.java:620	Colors, attempt counts and mocked countdown.	Correct current colors; values 30 through 0; thirty mocked waits.

12.3 Practical demonstration scenarios

Use a copied or fresh isolated data directory for practice. These are manual demonstration instructions with source-derived expected results; they are not individually reported as newly executed manual tests. Pick existing IDs/prices from your isolated catalog instead of assuming a private live record number.

TEST	INPUT / ACTION	EXPECTED RESULT	EVIDENCE
1	Login with username in different capitalization and correct exact password.	Correct role dashboard opens.	Source + full suite login checks.
2	Submit three incorrect passwords, switching identifier if desired.	Attempt counts 1/3,2/3,3/3; 30-second wait; generic error.	Source + scripted suite; real wait check separately reported.
3	Register name A, username ab or a password containing a space.	Each invalid field is rejected with its specific guidance/error; no account saved.	Source + account boundaries.
4	Search spider, full game ID, then PS5.	Partial title/full ID/full platform matching over the selected platform list.	Source + catalog tests.
5	Sort equal-price games low-high, then clear search.	Equal-price ties retain incoming order; clearing retains sort.	Source + stable-sort tests.
6	Queue one copy, then two copies of the same game, with enough available stock.	Separate requests allowed up to three total queued copies. A fourth is rejected.	Source + repeat queue checks.
7	Create four eligible VIP requests and one eligible regular, streak zero.	Predicted sequence V1,V2,V3,R1,V4 on both role views.	Source + queue fairness checks.
8	Two same-game requests each qty2 with available stock3.	Only the first fits in the shared-stock preview; second waits.	Source + predicted queue checks.
9	Press Enter in checkout with no ready request.	Message/input only; counters, queue and stock unchanged.	Source + tape flow checks.
10	Cancel a request as administrator, restart and open customer history.	Saved zero-charge CANCELLED record; no active rental or stock change.	Source + cancellation/reload checks.
11	Choose returns 1,3, then decline N.	No rental changes. Y later completes the selected whole quantities.	Source + return checks.
12	Open long receipt/result; repeatedly type invalid navigation.	Error/choices retry does not duplicate the full data view; N/P reach all records.	Source + pagination checks.
13	Try deleting a game with an active rental.	Deletion is blocked; inventory and rentals remain.	Source; manual exercise not run.

TEST	INPUT / ACTION	EXPECTED RESULT	EVIDENCE
14	Force a crash after a file replacement in an isolated subprocess.	On restart, uncommitted journal restores earlier files.	Source only for actual halt recovery; not executed this task.

12.4 Technical defense questions and answers

1. What is the main purpose of QUEUE2PLAY?

It organizes game rental requests, administrator approval, returns, inventory and customer history in a local Java terminal application.

2. Where does the JVM start?

Main.main(), which creates Queue2Play and calls start(). Queue2Play is the application controller, not the class invoked by java Main.

3. Why use several classes?

Models store records; services enforce focused rules; TerminalUI renders and reads input; RentalDataStore/AppStorage save and recover files. This reduces mixing display with business rules.

4. Is the project based mainly on inheritance?

No. It mainly uses composition and shared references. Rental implements Comparable for heap ordering; the two custom exceptions inherit RuntimeException.

5. Does Login use DSA?

Yes. Accounts are in an ArrayList and findUserByLogin linearly compares username/email. Password derivation is a cryptographic algorithm, distinct from a data-structure lookup.

6. Does registration use binary search?

No. It checks duplicate identifiers by a linear scan and validates text through loops/regex. The account list is not kept sorted for binary search.

7. Why choose linear search for catalog search?

It supports complete ID/platform equality and a substring anywhere in a title over an unsorted platform list. Binary search would not solve arbitrary substring matching directly.

8. Why choose insertion sort?

It is a simple, readable algorithm for small lists, stable with strict comparisons, and efficient on nearly sorted data. Its worst case is quadratic, so it is not ideal for a huge catalog.

9. How is sorting stable?

shouldMoveRight only shifts on a strict greater/less comparison. Equal title/price keys retain their incoming order.

10. Can sorting corrupt inventory order?

No. getDisplayList builds a separate result list. The underlying LinkedHashMap inventory is not rearranged.

11. Why a doubly linked checkout list?

Nodes allow insertion before the first regular request and unlinking front, rear or middle through previous/next references. Searching for the request still requires traversal.

12. Is checkout strict FIFO?

Within each membership group, arrival is retained. Across groups, VIP priority, the three-VIP regular turn and stock eligibility determine actual processing order.

13. Why can the first physical node be skipped?

The whole requested quantity may not fit available stock. The selector scans for another eligible request; no partial fulfillment is performed.

14. What happens after three VIP approvals when no regular fits?

An eligible VIP may proceed and the streak stays at three. The next eligible regular gets priority as soon as one fits.

15. Does cancellation reset VIP streak?

No. recordCheckout changes the streak only after successful checkout: VIP increments up to three, regular resets to zero.

16. How do customer/admin positions agree?

Both use previewReadyRequests, which simulates selection and shared-game stock using copies, then maps the planned order back to original requests.

17. Why is readiness preview not O(n)?

It repeatedly performs linear selection/removal/mapping and VIP insertion can also scan the queue. The overall worst-case bound is quadratic under average hash-map lookup.

18. Why does a client see positions 2 and 5 instead of 1 and 2?

The positions are global predicted checkout positions. Requests from other clients occupy the missing positions.

19. Why use a PriorityQueue for active rentals?

Earliest due rental is the minimum. add/poll are O(log n), peek is O(1), and a sorted copy serves due-first displays without changing the real heap.

20. Is PriorityQueue iteration sorted?

No. getRentalsByDueDate copies the heap and polls it. Natural ordering is due date, then rental number.

21. Is removal of a chosen rental O(log n)?

Not for remove(object) used here: it first finds the object in O(n), then repairs the heap. Do not confuse it with poll().

22. Where are stacks used?

Each User has rentalHistory and Queue2Play has auditLog. New records are pushed; newest-first viewing uses reverse indexing without pop.

23. Where is hash-table DSA used?

Game-ID LinkedHashMap, cart/preview stock maps and validation HashSets. These are Java library implementations, not a manual modulo/probing table written in this project.

24. Is a tree implemented?

No custom search tree or traversal is implemented. The active PriorityQueue is a heap, conceptually a complete binary tree stored by Java, not a BST module.

25. When is stock reserved?

It is not reserved at cart or queue submission. Admin approval deducts stock. Return restores it. Multiple clients can queue quantities whose sum exceeds current stock.

26. Why is the new limit three but some validators allow five?

The current forms limit new requests to three copies; saved-record and checkout compatibility retains older records that were created under a five-copy limit.

27. How does a same-game cart line work?

LinkedHashMap uses the same canonical Game key. Repeated selection adds to its integer quantity rather than creating another line.

28. What is the price formula?

Apply current membership discount to the three-day base price, scale by days/3, round single-copy cost to cents, multiply by quantity and round again.

29. How are late days determined?

Any time after due is rounded up to started 24-hour periods. The fee is $1 per late day per copy, recorded on return.

30. Are checkout and return batches equally atomic?

No. Checkout saves each approval separately. Multiple returns validate and save the complete selected batch once.

31. Why hash passwords with a salt?

Random salt makes identical passwords produce different stored values. PBKDF2 deliberately performs many iterations. Masking is only a terminal display feature.

32. What exactly is the hash format?

pbkdf2$iterations$base64-salt$base64-key. New hashes use 210000 iterations, a 16-byte salt and a 256-bit derived key.

33. Are .txt.bak files required?

No permanent .txt.bak files are created/loaded now. Temporary .old journal files support interrupted-save recovery and are removed after a normal commit.

34. Does .lock block two app instances for their whole lifetime?

No. File locks are acquired around storage operations. Original-byte comparison rejects a stale session after another instance saves.

35. How does rollback work?

AppStorage restores uncommitted files from the journal; RentalDataStore.restore restores the last committed in-memory snapshot after a failed save.

36. What testing remains manual?

Live password capture/masking, terminal appearance on actual fonts/ANSI implementations, resizing and source-only crash/concurrent-writer scenarios not run for this report.

13. Program analysis: issues and recommendations

The current complete regression suite passed. The findings below are code-inspection findings or conditional risks, not claims that the suite reproduced every issue. Priority reflects impact and practical likelihood. No fix is implemented by this documentation task. Preserve the established queue, stock, pricing, VIP and rental behavior if addressing a presentation issue.

High, conditional: Post-commit read failure can disagree with the memory rollback

AppStorage.commit writes committed, then rememberFiles reads all files. If that read fails, the catch reports failure; recover sees committed and keeps new files, while RentalDataStore.saveData restores old memory. This requires a filesystem/read failure after the commit marker.

Evidence: AppStorage.java:145-207; AppStorage.java:74-87; RentalDataStore.java:89-99.

Recommendation: Treat durable-commit completion separately from post-commit bookkeeping; reload or reconcile memory if files are already committed. Not reproduced.

Medium, source-based: Order Submitted remains unpaged

placeInQueue prints all newly submitted request rows in one loop. A large cart can push the heading/earlier rows above the viewport, although cart summaries and receipts are paged.

Evidence: Queue2Play.java:833-938.

Recommendation: Use the existing rendered-row/height paging pattern for the submitted-order result.

Medium, source-based: Remove-line prompt adds output after a full cart page

editOrder draws a full cart/actions page before appending the removal navigation and line-number input. Extra panels can exceed an otherwise correctly budgeted viewport.

Evidence: Queue2Play.java:728-820; TerminalUI.java:1848-1858.

Recommendation: Give removal its own screen or reserve its extra prompt/navigation rows.

Medium, source-based: Checkout tape is fixed at 81 columns

Checkout rendering deliberately keeps CHECKOUT_QUEUE_TABLE_WIDTH =81 even when the configured terminal is narrower. Wrapped tables elsewhere can adapt, but a narrow physical terminal can wrap fixed slips/status/input.

Evidence: TerminalUI.java:861-863; TerminalUI.java:2216-2297; TerminalUI.java:2370-2383.

Recommendation: State the minimum tape width or introduce a separately approved narrow-tape rendering mode. Keep the required 81-column normal design.

Medium, source-based: Cancellation history is not consistent across paths

Admin cancellation creates CANCELLED history; customer cancellation and game-deletion request removal do not. Users looking only at history cannot see every cancelled request.

Evidence: Queue2Play.java:940-1055; Queue2Play.java:1402-1414; Queue2Play.java:2017-2071.

Recommendation: Decide whether all cancellations should be history records or document the existing distinction. Any change is a behavior decision, not only rendering.

Medium, boundary-dependent: Return preview freezes return time before confirmation

processReturn captures LocalDateTime.now before preview navigation and Yes/No. A long review across a due/late-day boundary records the earlier time and can reduce late fees compared with final confirmation time.

Evidence: Queue2Play.java:1566-1586; RentalManager.java:182-193.

Recommendation: Define whether return time means start of review or final approval. Retain current semantics until explicitly changed.

Low, source-based: Oversized record can exceed height on its own page

pageStarts keeps a record together and splits only when there is already a record on the page. One record taller than available rows is still placed whole; a fixed reserved-row budget can also leave too little space on very short displays.

Evidence: TerminalUI.java:65-82; TerminalUI.java:2189-2204.

Recommendation: Handle unusually tall individual records with a detail view or continuation strategy while preserving complete text.

Low, source-based: Viewport is fixed for the session

terminalWidth and terminalHeight are final instance values. Resizing after start does not recalculate dimensions.

Evidence: TerminalUI.java:40-43; TerminalUI.java:47-54.

Recommendation: Document restart after resizing or refresh dimensions at screen boundaries in a later UI change.

Low, source-based: Character count is not terminal cell width

center, fit and row height use String.length/substrings. Some wide Unicode characters, combining marks or emoji may occupy a different number of terminal cells.

Evidence: TerminalUI.java:1323-1333; TerminalUI.java:1306-1320; TerminalUI.java:137-147.

Recommendation: Test real-world non-ASCII names/titles; optionally restrict/measure display width with a simple, approved policy.

Low, extreme boundary: All platform counters can block one platform

generateGameId checks all three counters for exhaustion before selecting the requested platform. If one reaches the upper limit, IDs for another platform are blocked too.

Evidence: GameCatalog.java:64-78.

Recommendation: Check only the selected platform if supporting extremely large counters; not a normal-size school-project issue.

13.1 Missing features versus bugs

CURRENT ABSENCE	INTERPRETATION / OPTIONAL NEXT STEP
Password change / profile update / account deletion	No customer/admin UI for these operations. Registration editing affects only an unsaved registration draft. Add only if required.
Editing an existing catalog title/platform/price	Add Game can edit a new draft, but existing game metadata has no edit module. Stock updates and deletion exist.
Archived session receipts / export	Current receipt is session-only; saved rentals/history remain. No receipt PDF/export/search or reopening past session totals.
Email verification / payment processing	Format validation and calculated amount recording exist; email ownership and actual payment collection are outside current scope.
Partial return / selective cross-page return cart	Returns select full rentals from one current page. Search/page again to process other records; no partial-copy return.
Universal runtime resize / perfect Unicode width	Responsive configured pagination exists, but continuous resizing and terminal-cell-aware text measurement do not.

Intentional design notes: built-in account defaults are restored at startup; newer forms can be stricter than older saved records; customer submission does not reserve stock; queue prices use current membership/game price at checkout. These are described explicitly rather than silently classified as defects. There is no implementation change in this document.

13.2 TUI recommendations

Retain the established double-line headings/status boxes, open tables, padded centered choices and pink input separators. Keep details such as session administrator/time separate from rental rows. Use one set of responsive record measurements for every long result, including submitted orders and removal prompts. Continue showing only usable pagination keys and the named Back destination. Keep F/P on Yes/No previews so N remains No.

For discoverability, keep platform menus clearly distinguished: catalog includes All Games, while Add Game does not. Show whether an input expects a current-page row, whole-cart line or queue number. Keep useful centered constraints/examples, rather than long generic tutorial text. Add a minimum-width notice or narrow view for fixed checkout tape only after agreeing on how it should coexist with the 81-column card requirement.

Appendix A. Complete application method reference

This appendix covers all 412 explicitly declared methods and constructors found in the 16 current application Java files, including nested support classes and overloads. Methods inherited from Java library classes and implicit constructors are not invented here. The declaration gives exact parameter types/names. Purpose is based on the source comment and reviewed role; the main chapters explain the important behavior in detail.

Connections list direct call names from the source. Qualified calls are included when their receiver is a project class or a recognized project-object field/parameter/local variable; ordinary collection methods are omitted. This is a navigation aid, not a compiler-generated call graph: overloads and dynamic dispatch still need the actual declaration/body. A method can mutate a shared record directly without calling another project method. A constructor returns a new object to the new expression; it has no declared return type. void means no return value.

Main.java - 1 declarations

Entry point; handles input-end and fatal startup/storage errors.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
main Main.java:3-16	public static void main(String[] args) Purpose: Start Queue2Play and report input or storage failures. Return: void. Project call names: Queue2Play, system.start

Queue2Play.java - 61 declarations

Application controller: menus, carts, checkout session, returns, inventory and navigation. Owns the shared in-memory state.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
confirmRentalDashboard Queue2Play.java:68-74	private boolean confirmRentalDashboard(LinkedHashMap<Game, Integer> cart) Purpose: Confirm discarding a nonempty cart before leaving for Dashboard. Return: boolean. Project call names: ui.confirmDiscardCart, requestDashboardReturn
parseMenuChoice Queue2Play.java:77-82	private int parseMenuChoice(String text) Purpose: Convert a numeric choice, treating Back as zero. Return: int. Project call names: No other project method call name identified.
readMenuChoice Queue2Play.java:85-95	private int readMenuChoice(String exitKey) Purpose: Read a menu number or exit key without crashing on invalid text. Return: int. Project call names: ui.readText, ui.prompt
waitForScreenNavigation Queue2Play.java:98-116	private void waitForScreenNavigation(String destination, boolean allowDashboard) Purpose: Keep a result visible until Back or Dashboard is selected. Return: void. Project call names: ui.showNavigationOptions, ui.inputBar, ui.readText, ui.prompt, requestDashboardReturn, ui.error
Queue2Play Queue2Play.java:119-124	Queue2Play() Purpose: Connect the interface, catalog, accounts, rentals and storage. Return: constructor. Project call names: RentalDataStore, AccountManager, GameCatalog, RentalManager
requestDashboardReturn Queue2Play.java:127-129	void requestDashboardReturn() Purpose: Mark that the current function should return to Dashboard. Return: void. Project call names: No other project method call name identified.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
isDashboardReturnRequested Queue2Play.java:132-134	boolean isDashboardReturnRequested() Purpose: Check whether a nested screen requested Dashboard. Return: boolean. Project call names: No other project method call name identified.
clearDashboardReturnRequest Queue2Play.java:137-139	void clearDashboardReturnRequest() Purpose: Reset the Dashboard request before starting another function. Return: void. Project call names: No other project method call name identified.
start Queue2Play.java:144-187	public void start() Purpose: Load saved data and run the main menu until the user exits. Return: void. Project call names: accountService.loadAccounts, dataStore.loadGames, dataStore.loadRentals, dataStore.loadQueue, dataStore.loadLogs, dataStore.checkpoint, accountService.embeddedAccountsChanged, dataStore.file, dataStore.saveData, ui.showMainMenu, readMenuChoice, accountService.login, user.isAdmin, adminMenu, clientMenu, accountService.register, ui.askYesNo, ui.showGoodbye, ui.error
writeLog Queue2Play.java:190-192	void writeLog(String type, String message) Purpose: Add a timestamped event to the log records for saving. Return: void. Project call names: dataStore.writeLog
logAdmin Queue2Play.java:195-199	void logAdmin(String message) Purpose: Push an admin action onto the audit stack and add it to saved logs. Return: void. Project call names: dataStore.writeLogRecord
logEvent Queue2Play.java:202-204	void logEvent(String message) Purpose: Record a general event without adding it to the admin audit stack. Return: void. Project call names: writeLog
paginateGames Queue2Play.java:209-211	Game paginateGames(ArrayList<Game> list, String title, boolean selectable) Purpose: Browse or select games using shared search, sort and page controls. Return: Game. Project call names: paginateGames
paginateGames Queue2Play.java:214-334	private Game paginateGames(ArrayList<Game> platformGames, String title, boolean selectable, User client, LinkedHashMap<Game, Integer> cart, int[] selectedQuantity) Purpose: Browse or select games using shared search, sort and page controls. Return: Game. Project call names: gameCatalog.getDisplayList, ui.gamePages, ui.printHeader, ui.showCatalogState, ui.showEmptyState, ui.showGameTableHeader, ui.showGameRow, ui.info, ui.tableLine, ui.showGameNavigation, ui.inputBar, ui.readText, ui.prompt, confirmRentalDashboard, requestDashboardReturn, ui.warning, ui.showNavigationOptions, ui.error, validateRentalSelection, askQuantity, checkoutQueue.queuedCopies
choosePlatform Queue2Play.java:336-338	String choosePlatform(String heading) Purpose: Read a platform choice while supporting cart and navigation options. Return: String. Project call names: choosePlatform
choosePlatform Queue2Play.java:341-392	private String choosePlatform(String heading, LinkedHashMap<Game, Integer> cart) Purpose: Read a platform choice while supporting cart and navigation options. Return: String. Project call names: ui.showPlatformMenu, ui.readText, ui.prompt, confirmRentalDashboard, requestDashboardReturn, parseMenuChoice, ui.error
viewGames Queue2Play.java:395-402	void viewGames(String platform) Purpose: Display the selected platform's catalog without changing stock. Return: void. Project call names: gameCatalog.getGameList, ui.platformColor, paginateGames

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
clientMenu Queue2Play.java:407-439	void clientMenu(User client) Purpose: Run the client's dashboard and route its choices. Return: void. Project call names: clearDashboardReturnRequest, ui.showClientMenu, readMenuChoice, browseGames, placeInQueue, cancelMyRequest, viewHistory, upgradeToVip, ui.askYesNo, ui.loading, ui.success, ui.pause, ui.error
browseGames Queue2Play.java:442-444	void browseGames() Purpose: Open platform selection and browse the chosen catalog. Return: void. Project call names: browseGames
browseGames Queue2Play.java:447-483	void browseGames(boolean fromInventory) Purpose: Open platform selection and browse the chosen catalog. Return: void. Project call names: ui.showBrowseMenu, ui.readText, ui.prompt, requestDashboardReturn, parseMenuChoice, ui.error, viewGames, isDashboardReturnRequested
chooseOneGame Queue2Play.java:486-519	Game chooseOneGame(User client, LinkedHashMap<Game, Integer> cart, int[] selectedQuantity) Purpose: Select one displayed game while preserving the current platform. Return: Game. Project call names: choosePlatform, isDashboardReturnRequested, gameCatalog.getGameList, paginateGames
validateRentalSelection Queue2Play.java:522-539	private String validateRentalSelection(User client, LinkedHashMap<Game, Integer> cart, Game game) Purpose: Check stock, active rentals and remaining queued-copy allowance. Return: String. Project call names: checkoutQueue.queuedCopies, rentalService.hasActiveRental
askQuantity Queue2Play.java:542-544	int askQuantity(Game game, int inCart) Purpose: Read a valid quantity within stock and per-game limits. Return: int. Project call names: askQuantity
askQuantity Queue2Play.java:547-596	int askQuantity(Game game, int inCart, int queued) Purpose: Read a valid quantity within stock and per-game limits. Return: int. Project call names: ui.error, ui.showSelectQuantityScreen, ui.readText, ui.prompt, ui.askYesNo, ui.confirmDiscardCart, requestDashboardReturn
addGamesToCart Queue2Play.java:599-635	int addGamesToCart(User client, LinkedHashMap<Game, Integer> cart) Purpose: Use a map to combine repeated game selections into one cart line. Return: int. Project call names: chooseOneGame, isDashboardReturnRequested, ui.success, ui.askYesNo
chooseRentalDays Queue2Play.java:638-640	int chooseRentalDays() Purpose: Read a preset or custom rental duration within the allowed range. Return: int. Project call names: chooseRentalDays
chooseRentalDays Queue2Play.java:643-720	int chooseRentalDays(String previousScreen) Purpose: Read a preset or custom rental duration within the allowed range. Return: int. Project call names: ui.printHeader, ui.showRentalDurationOptions, ui.inputBar, ui.readText, ui.prompt, ui.askYesNo, ui.confirmDiscardCart, requestDashboardReturn, parseMenuChoice, ui.error, ui.showNavigationOptions
showOrderSummary Queue2Play.java:723-725	void showOrderSummary(User client, LinkedHashMap<Game, Integer> cart, int days) Purpose: Display cart quantities and duration using the shared order summary. Return: void. Project call names: ui.showOrderSummary

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
editOrder Queue2Play.java:728-820	int editOrder(User client, LinkedHashMap<Game, Integer> cart, int days) Purpose: Edit the existing cart or duration without restarting the order. Return: int. Project call names: ui.cartPages, ui.showCartPage, ui.showCartActions, ui.readText, ui.prompt, ui.askYesNo, ui.confirmDiscardCart, requestDashboardReturn, parseMenuChoice, ui.error, addGamesToCart, ui.showNavigationOptions, ui.inputBar, ui.success, chooseRentalDays
validateCartBeforeSubmission Queue2Play.java:823-830	boolean validateCartBeforeSubmission(User client, LinkedHashMap<Game, Integer> cart) Purpose: Recheck the cart before creating any queue requests. Return: boolean. Project call names: rentalService.validateCart, ui.error
placeInQueue Queue2Play.java:833-938	void placeInQueue(User client) Purpose: Create and save one request per distinct cart game without deducting stock. Return: void. Project call names: addGamesToCart, ui.info, chooseRentalDays, ui.cartPages, ui.showCartPage, ui.showCartActions, ui.readText, ui.prompt, validateCartBeforeSubmission, ui.warning, editOrder, ui.askYesNo, ui.confirmDiscardCart, requestDashboardReturn, ui.error, ui.loading, RentalDataStore.require, RentalRequest, checkoutQueue.enqueue, logEvent, ui.queueLabel, dataStore.saveQueue, ui.printHeader, ui.success, ui.showSubmittedQueueHeader, checkoutQueue.previewReadyRequests, checkoutQueue.getDisplayRequests, ui.showQueueRequestRow, ui.tableLine, ui.finishScreen, ui.waitForResultNavigation
cancelMyRequest Queue2Play.java:940-1055	void cancelMyRequest(User client) Purpose: Confirm and remove only the client's own queued request. Return: void. Project call names: checkoutQueue.getRequestsOf, ui.printHeader, ui.showEmptyState, waitForScreenNavigation, checkoutQueue.previewReadyRequests, checkoutQueue.getDisplayRequests, ui.queuePages, ui.showListStatus, ui.showQueueRequestHeader, ui.showQueueRequestRow, ui.info, ui.tableLine, ui.showPagination, ui.showQueueCancellationOptions, ui.readText, ui.prompt, requestDashboardReturn, ui.error, ui.showNavigationOptions, ui.inputBar, checkoutQueue.findByNumber, ui.askYesNo, ui.queueLabel, ui.loading, checkoutQueue.remove, logEvent, dataStore.saveQueue, ui.success, isEmpty, ui.finishScreen, ui.waitForResultNavigation
viewHistory Queue2Play.java:1058-1103	void viewHistory(User client) Purpose: Read the client's history stack from newest to oldest without popping it. Return: void. Project call names: ui.printHeader, ui.showEmptyState, waitForScreenNavigation, ui.rentalPages, ui.showListStatus, ui.showHistoryTableHeader, ui.showHistoryRow, ui.info, ui.tableLine, ui.showPagination, ui.showBackToDashboard, ui.inputBar, ui.readText, ui.prompt, ui.error
upgradeToVip Queue2Play.java:1106-1137	void upgradeToVip(User client) Purpose: Confirm membership upgrade and refresh the client's queued priority. Return: void. Project call names: client.isVip, ui.printHeader, ui.showEmptyState, waitForScreenNavigation, ui.showVipBenefits, ui.inputBar, ui.readText, ui.prompt, ui.loading, checkoutQueue.refreshMembership, logEvent, accountService.saveAccounts, ui.success, ui.finishScreen, ui.waitForResultNavigation, ui.info, ui.error
adminMenu Queue2Play.java:1142-1174	void adminMenu(User admin) Purpose: Run the admin dashboard and route its choices. Return: void. Project call names: clearDashboardReturnRequest, ui.showAdminMenu, readMenuChoice, checkoutManagement, processReturn, manageInventory, activeRentalsMenu, viewAuditLog, ui.askYesNo, ui.loading, ui.success, ui.pause, ui.error
activeRentalsMenu Queue2Play.java:1177-1192	void activeRentalsMenu() Purpose: Open the active-rental list or return to Dashboard. Return: void. Project call names: ui.showActiveRentalsMenu, ui.readText, ui.prompt, viewActiveRentals, ui.error, isDashboardReturnRequested

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
CheckoutEntry Queue2Play.java:1200-1203	CheckoutEntry(Rental rental, RentalRequest cancelled) Purpose: Store one approval or cancellation for the session tape. Return: constructor. Project call names: No other project method call name identified.
CheckoutSession Queue2Play.java:1216-1218	CheckoutSession(String admin) Purpose: Start a display session for one administrator. Return: constructor. Project call names: No other project method call name identified.
addRental Queue2Play.java:1221-1226	void addRental(Rental rental) Purpose: Append an approval and update the session counts and total. Return: void. Project call names: CheckoutEntry
addCancellation Queue2Play.java:1229-1232	void addCancellation(RentalRequest request) Purpose: Append a cancellation and update the session cancellation count. Return: void. Project call names: CheckoutEntry
checkoutManagement Queue2Play.java:1236-1238	void checkoutManagement() Purpose: Run the admin checkout tape and show its final receipt. Return: void. Project call names: checkoutManagement
checkoutManagement Queue2Play.java:1241-1248	void checkoutManagement(String admin) Purpose: Run the admin checkout tape and show its final receipt. Return: void. Project call names: CheckoutSession, runCheckoutSession, ui.endCheckoutDisplay
runCheckoutSession Queue2Play.java:1251-1313	private void runCheckoutSession(CheckoutSession session) Purpose: Handle approvals, cancellations, queue views and receipt navigation. Return: void. Project call names: ui.showCheckoutQueue, ui.readText, ui.prompt, ui.showCheckoutReceipt, ui.finishScreen, ui.waitForReceiptNavigation, requestDashboardReturn, ui.moveCheckoutPage, checkoutQueue.findNextEligibleRequest, ui.warning, ui.confirmCheckout, approveSessionRequest, approveReadyBatch, cancelSessionRequest, isDashboardReturnRequested, ui.error, ui.showCheckoutTapeFooter, ui.showTapeInput
previewReadyCheckouts Queue2Play.java:1316-1318	ArrayList<RentalRequest> previewReadyCheckouts() Purpose: Get predicted ready requests without changing stock or queue state. Return: ArrayList<RentalRequest>. Project call names: checkoutQueue.previewReadyRequests
approveReadyBatch Queue2Play.java:1321-1340	void approveReadyBatch(CheckoutSession session) Purpose: Confirm the preview and approve its ready requests in order. Return: void. Project call names: previewReadyCheckouts, ui.warning, ui.showCheckoutBatch, ui.confirmCheckoutBatch, checkoutQueue.findNextEligibleRequest, ui.error, ui.queueLabel, approveSessionRequest
approveSessionRequest Queue2Play.java:1343-1361	boolean approveSessionRequest(RentalRequest request, CheckoutSession session) Purpose: Validate and approve one request, then append its result slip. Return: boolean. Project call names: rentalService.validateCheckout, ui.error, ui.queueLabel, approveCheckout, session.addRental, ui.showApprovedSlip
cancelSessionRequest Queue2Play.java:1364-1399	void cancelSessionRequest(CheckoutSession session) Purpose: Confirm an admin cancellation and save its customer history record. Return: void. Project call names: ui.inputBar, ui.readText, ui.prompt, requestDashboardReturn, checkoutQueue.findByNumber, ui.error, ui.confirmCheckoutCancellation, recordAdminCancellation, checkoutQueue.remove, logAdmin, ui.queueLabel, dataStore.saveQueue, session.addCancellation, ui.showCancelledSlip

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
recordAdminCancellation Queue2Play.java:1402-1414	void recordAdminCancellation(RentalRequest request) Purpose: Add zero-charge cancelled history without changing stock or active rentals. Return: void. Project call names: RentalDataStore.require, Rental
approveCheckout Queue2Play.java:1417-1430	Rental approveCheckout(RentalRequest request) Purpose: Process and save an eligible queued request as an active rental. Return: Rental. Project call names: rentalService.validateCheckout, ui.error, rentalService.checkout, logAdmin, dataStore.saveData
searchReturnRentals Queue2Play.java:1433-1445	ArrayList<Rental> searchReturnRentals(String keyword) Purpose: Use linear search on due-date-ordered rentals to filter return choices. Return: ArrayList<Rental>. Project call names: rentalService.getRentalsByDueDate
parseReturnSelection Queue2Play.java:1448-1477	ArrayList<Rental> parseReturnSelection(ArrayList<Rental> rentals, int start, int end, String input) Purpose: Use a boolean array to reject duplicate rows and keep due-date order. Return: ArrayList<Rental>. Project call names: ui.error
selectActiveRentalsForReturn Queue2Play.java:1480-1535	ArrayList<Rental> selectActiveRentalsForReturn() Purpose: Page and search active rentals to select one or several returns. Return: ArrayList<Rental>. Project call names: ui.printHeader, ui.showEmptyState, waitForScreenNavigation, searchReturnRentals, ui.rentalPages, ui.showReturnSelectionList, ui.readText, ui.prompt, ui.showReturnSearchPrompt, requestDashboardReturn, ui.error, parseReturnSelection
completeSelectedReturns Queue2Play.java:1538-1563	void completeSelectedReturns(ArrayList<Rental> selected, LocalDateTime returnedAt) Purpose: Check the full batch, restore stock and save all selected returns together. Return: void. Project call names: RentalDataStore.require, rentalService.completeReturn, logAdmin, ui.money, dataStore.saveData
processReturn Queue2Play.java:1566-1586	void processReturn() Purpose: Confirm selected returns and display the saved results. Return: void. Project call names: selectActiveRentalsForReturn, ui.confirmSelectedReturns, completeSelectedReturns, ui.showSelectedReturnResults, ui.waitForReturnResultNavigation
viewActiveRentals Queue2Play.java:1588-1632	void viewActiveRentals() Purpose: Display a copy of the rental heap in earliest-due order. Return: void. Project call names: rentalService.getRentalsByDueDate, ui.printHeader, ui.showEmptyState, waitForScreenNavigation, ui.rentalPages, ui.showListStatus, ui.showActiveRentalHeader, ui.showActiveRentalRow, ui.info, ui.tableLine, ui.showPagination, ui.showNavigationOptions, ui.inputBar, ui.readText, ui.prompt, requestDashboardReturn, ui.error
viewAuditLog Queue2Play.java:1635-1690	void viewAuditLog() Purpose: Read the audit stack newest first without removing its entries. Return: void. Project call names: ui.printHeader, ui.showEmptyState, waitForScreenNavigation, ui.auditPages, ui.showListStatus, ui.showAuditHeader, ui.showAuditRow, ui.showLegacyAuditRow, ui.info, ui.tableLine, ui.showPagination, ui.showBackToDashboard, ui.inputBar, ui.readText, ui.prompt, ui.error
manageInventory Queue2Play.java:1695-1727	void manageInventory() Purpose: Route admin choices for browsing, adding, deleting and updating games. Return: void. Project call names: ui.showInventoryMenu, ui.readText, ui.prompt, parseMenuChoice, ui.error, browseGames, addGame, updateGameStock, deleteGame, isDashboardReturnRequested

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
gameAlreadyExists Queue2Play.java:1730-1737	private boolean gameAlreadyExists(String title, String platform) Purpose: Use linear search to detect a duplicate title on the same platform. Return: boolean. Project call names: No other project method call name identified.
addGame Queue2Play.java:1740-2014	void addGame() Purpose: Reject duplicates, then add a game to the inventory map by ID. Return: void. Project call names: ui.showGameEditMenu, ui.readText, ui.prompt, ui.error, ui.showGameEditField, ui.printHeader, ui.showAddGameInstructions, ui.inputBar, ui.confirmDiscardTransaction, requestDashboardReturn, ui.info, GameCatalog.validTitle, gameAlreadyExists, ui.showPlatformOptions, ui.showPriceInstructions, ui.money, ui.showStockInstructions, ui.showNewGameConfirmation, ui.loading, gameCatalog.addGame, logAdmin, dataStore.saveGames, ui.success, ui.showGameAdded, ui.askYesNo
deleteGame Queue2Play.java:2017-2071	void deleteGame() Purpose: Confirm game deletion and remove its pending requests when allowed. Return: void. Project call names: choosePlatform, isDashboardReturnRequested, gameCatalog.getGameList, paginateGames, rentalService.gameHasActiveRental, ui.error, checkoutQueue.countRequestsForGame, ui.printHeader, ui.showDeleteGameDetails, ui.warning, ui.askYesNo, ui.loading, checkoutQueue.removeRequestsForGame, logAdmin, dataStore.saveData, ui.success
updateGameStock Queue2Play.java:2074-2199	void updateGameStock() Purpose: Validate and confirm a stock adjustment before saving it. Return: void. Project call names: choosePlatform, isDashboardReturnRequested, paginateGames, gameCatalog.getGameList, ui.printHeader, ui.showStockUpdateOptions, ui.inputBar, ui.readText, ui.prompt, ui.confirmDiscardTransaction, requestDashboardReturn, ui.info, ui.error, ui.showStockQuantityPrompt, rentalService.updatedStock, ui.showStockConfirmation, ui.askYesNo, logAdmin, dataStore.saveGames, ui.loading, ui.success, ui.showStockUpdated

AccountManager.java - 19 declarations

Load account records; restore built-in accounts; validate registration; find users; register and authenticate.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
AccountManager AccountManager.java:12-16	AccountManager(ArrayList<User> users, TerminalUI ui, RentalDataStore dataStore) Purpose: Connect account validation, login and registration to storage. Return: constructor. Project call names: No other project method call name identified.
validUsername AccountManager.java:19-21	static boolean validUsername(String value) Purpose: Check usernames using the saved-account rules. Return: boolean. Project call names: No other project method call name identified.
validName AccountManager.java:24-26	static boolean validName(String value) Purpose: Check names using the saved-account rules. Return: boolean. Project call names: No other project method call name identified.
validEmail AccountManager.java:29-31	static boolean validEmail(String value) Purpose: Check emails using the saved-account rules. Return: boolean. Project call names: No other project method call name identified.
validRegistrationName AccountManager.java:35-49	static boolean validRegistrationName(String value) Purpose: Check the length, letters, spaces and dots of a new name. Return: boolean. Project call names: No other project method call name identified.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
validRegistrationUsername AccountManager.java:52-62	static boolean validRegistrationUsername(String value) Purpose: Check a new username and require at least one letter. Return: boolean. Project call names: No other project method call name identified.
validRegistrationEmail AccountManager.java:65-82	static boolean validRegistrationEmail(String value) Purpose: Reject long emails and invalid local or domain parts. Return: boolean. Project call names: validEmail
validRegistrationPassword AccountManager.java:85-97	static boolean validRegistrationPassword(String value) Purpose: Check password length and reject spaces or file separators. Return: boolean. Project call names: No other project method call name identified.
embeddedAccountsChanged AccountManager.java:102-104	boolean embeddedAccountsChanged() Purpose: Tell startup whether built-in accounts need saving. Return: boolean. Project call names: No other project method call name identified.
loadAccounts AccountManager.java:107-127	void loadAccounts() Purpose: Load valid accounts into the list and restore built-in logins. Return: void. Project call names: dataStore.records, RentalDataStore.require, validUsername, validName, validEmail, GameCatalog.containsControlCharacter, findUserByLogin, User, dataStore.invalidRecord, ensureEmbeddedAccount
ensureEmbeddedAccount AccountManager.java:130-160	private void ensureEmbeddedAccount(String username, String name, String email, String password, String role, String membership) Purpose: Create or restore a built-in account while keeping client upgrades. Return: void. Project call names: findUser, findUserByLogin, RentalDataStore.require, User, PasswordSecurity.matches, PasswordSecurity.hashPlaintext
saveAccounts AccountManager.java:163-165	void saveAccounts() Purpose: Save accounts with the other related data files. Return: void. Project call names: dataStore.saveData
findUser AccountManager.java:168-175	User findUser(String username) Purpose: Use linear search to find a username, ignoring case. Return: User. Project call names: No other project method call name identified.
findUserByLogin AccountManager.java:178-190	User findUserByLogin(String login) Purpose: Use linear search to find a username or email, ignoring case. Return: User. Project call names: findUser
emailExists AccountManager.java:193-195	boolean emailExists(String email) Purpose: Check whether an email already matches a login identifier. Return: boolean. Project call names: findUserByLogin
cancelRegistration AccountManager.java:198-204	private boolean cancelRegistration() Purpose: Confirm discarding the unsaved account details. Return: boolean. Project call names: ui.confirmRegistrationCancellation, ui.info

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
register AccountManager.java:207-337	void register() Purpose: Collect, validate and save a new account; keep drafts when editing. Return: void. Project call names: ui.clearScreen, ui.showRegistrationConfirmation, ui.inputBar, ui.readText, ui.prompt, cancelRegistration, ui.error, ui.showRegistrationEditMenu, ui.showRegistrationStep, ui.readPassword, validRegistrationName, ui.showFullNameInstructions, validRegistrationUsername, findUserByLogin, validRegistrationEmail, emailExists, validRegistrationPassword, ui.loading, User, PasswordSecurity.hashPlaintext, dataStore.writeLog, saveAccounts, ui.showAccountCreated, ui.finishScreen, ui.waitForResultNavigation
failedLogin AccountManager.java:340-350	private void failedLogin(String login) Purpose: Record a failed attempt and apply the three-attempt cooldown. Return: void. Project call names: ui.error, ui.showLoginAttempt, ui.showLoginCooldown, dataStore.writeLog, dataStore.saveData
login AccountManager.java:353-400	User login() Purpose: Check username or email and password, then return the signed-in user. Return: User. Project call names: ui.clearScreen, ui.showLoginUsernameScreen, ui.readText, ui.prompt, login.isEmpty, ui.error, findUserByLogin, ui.showLoginPasswordScreen, ui.readPassword, ui.loading, PasswordSecurity.matches, failedLogin, dataStore.writeLog, dataStore.saveData, ui.success, ui.pause

GameCatalog.java - 14 declarations

Game validation and ID counters; starter games; platform list; linear search; stable insertion sort.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
validTitle GameCatalog.java:6-9	static boolean validTitle(String title) Purpose: Reject empty, long or unsafe game titles. Return: boolean. Project call names: containsControlCharacter
containsControlCharacter GameCatalog.java:12-19	static boolean containsControlCharacter(String value) Purpose: Scan characters to detect unsafe control codes. Return: boolean. Project call names: No other project method call name identified.
validId GameCatalog.java:22-42	static boolean validId(String id, String platform) Purpose: Check the platform prefix and numeric part of a game identifier. Return: boolean. Project call names: No other project method call name identified.
validate GameCatalog.java:45-51	static void validate(String title, String platform, double price, int stock) Purpose: Reject invalid game titles, platforms, prices or stock. Return: void. Project call names: RentalDataStore.require, validTitle
GameCatalog GameCatalog.java:59-61	GameCatalog(Map<String, Game> games) Purpose: Use the shared inventory map for catalog operations. Return: constructor. Project call names: No other project method call name identified.
generateGameId GameCatalog.java:64-78	String generateGameId(String platform) Purpose: Create the next numbered identifier for the selected platform. Return: String. Project call names: RentalDataStore.require

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
addGame GameCatalog.java:81-91	Game addGame(String title, String platform, double price, int copies) Purpose: Reject duplicates, then add a game to the inventory map by ID. Return: Game. Project call names: validate, RentalDataStore.require, generateGameId, Game
loadStartingGames GameCatalog.java:94-104	void loadStartingGames() Purpose: Create the initial catalog when no games file exists. Return: void. Project call names: addGame
getGameList GameCatalog.java:107-115	ArrayList<Game> getGameList(String platform) Purpose: Copy all games or one platform's games into a browsing list. Return: ArrayList<Game>. Project call names: No other project method call name identified.
linearSearch GameCatalog.java:118-129	ArrayList<Game> linearSearch(ArrayList<Game> list, String keyword) Purpose: Scan each game for an ID, title or platform match; O(n) time. Return: ArrayList<Game>. Project call names: No other project method call name identified.
getDisplayList GameCatalog.java:132-138	ArrayList<Game> getDisplayList(ArrayList<Game> platformGames, String keyword, int sortOption) Purpose: Search the full platform list and sort a copy without changing inventory. Return: ArrayList<Game>. Project call names: linearSearch, insertionSort
insertionSort GameCatalog.java:141-152	void insertionSort(ArrayList<Game> list, int option) Purpose: Use stable insertion sort for titles or prices; worst O(n squared). Return: void. Project call names: RentalDataStore.require, shouldMoveRight
shouldMoveRight GameCatalog.java:155-165	private boolean shouldMoveRight(Game left, Game current, int option) Purpose: Compare two games to decide whether insertion sort should shift one. Return: boolean. Project call names: No other project method call name identified.
updateCounterFromId GameCatalog.java:168-185	void updateCounterFromId(String id) Purpose: Update platform counters so loaded game IDs are not reused. Return: void. Project call names: No other project method call name identified.

RentalManager.java - 18 declarations

Rental price, active-copy checks, checkout validation, due-order listing, late fees, returns and cart validation.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
RentalManager RentalManager.java:14-20	RentalManager(CheckoutQueue checkoutQueue, PriorityQueue<Rental> activeRentals, ArrayList<Rental> allRentals, Map<String, Game> catalog) Purpose: Use shared queue, heap, history and catalog data for rental operations. Return: constructor. Project call names: No other project method call name identified.
sameUser RentalManager.java:23-25	static boolean sameUser(User a, User b) Purpose: Compare clients by username, ignoring case. Return: boolean. Project call names: No other project method call name identified.
sameGame RentalManager.java:28-30	static boolean sameGame(Game a, Game b) Purpose: Compare games by their stable identifiers. Return: boolean. Project call names: No other project method call name identified.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
rentedCopies RentalManager.java:33-41	int rentedCopies(Game game) Purpose: Traverse active rentals to count the copies currently out for one game. Return: int. Project call names: sameGame
updatedStock RentalManager.java:44-55	int updatedStock(Game game, boolean adding, int quantity) Purpose: Calculate a stock adjustment while preserving the total-copy limit. Return: int. Project call names: RentalDataStore.require, rentedCopies
calculateRentalCost RentalManager.java:58-65	double calculateRentalCost(User client, Game game) Purpose: Calculate the base rental price with the client's VIP discount. Return: double. Project call names: RentalDataStore.require, GameCatalog.validate, client.isVip
calculateRentalCost RentalManager.java:68-72	double calculateRentalCost(User client, Game game, int days) Purpose: Scale the price for the chosen duration and round to cents. Return: double. Project call names: RentalDataStore.require, calculateRentalCost
calculateRentalCost RentalManager.java:75-78	double calculateRentalCost(User client, Game game, int days, int quantity) Purpose: Multiply the duration price by quantity and round to cents. Return: double. Project call names: RentalDataStore.require, calculateRentalCost
hasActiveRental RentalManager.java:81-88	boolean hasActiveRental(User client, Game game) Purpose: Use linear search to check whether a client is renting a game. Return: boolean. Project call names: sameUser, sameGame
activeCopies RentalManager.java:91-99	int activeCopies(User client, Game game) Purpose: Traverse active rentals to total a client's copies of one game. Return: int. Project call names: sameUser, sameGame
gameHasActiveRental RentalManager.java:102-109	boolean gameHasActiveRental(Game game) Purpose: Use linear search to check whether any client is renting a game. Return: boolean. Project call names: sameGame
getRentalsByDueDate RentalManager.java:112-119	ArrayList<Rental> getRentalsByDueDate() Purpose: Poll a copied min-heap into due-date order without changing active rentals. Return: ArrayList<Rental>. Project call names: No other project method call name identified.
validateCheckout RentalManager.java:122-144	String validateCheckout(RentalRequest request) Purpose: Check that a queued request can safely become an active rental. Return: String. Project call names: checkoutQueue.findByNumber, CheckoutQueue.validRequest, activeCopies
checkout RentalManager.java:147-173	Rental checkout(RentalRequest request, int rentalNumber) Purpose: Deduct stock, add rental and history entries, then remove the queued request. Return: Rental. Project call names: RentalDataStore.require, validateCheckout, Rental, calculateRentalCost, checkoutQueue.recordCheckout, checkoutQueue.remove
calculateLateFee RentalManager.java:176-179	double calculateLateFee(Rental rental, int daysLate) Purpose: Calculate the per-copy fee for the recorded number of late days. Return: double. Project call names: RentalDataStore.require

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
daysOverdue RentalManager.java:182-193	static int daysOverdue(LocalDateTime due, LocalDateTime now) Purpose: Count each started day after the due time as one late day. Return: int. Project call names: No other project method call name identified.
completeReturn RentalManager.java:196-209	void completeReturn(Rental rental, LocalDateTime returnedAt) Purpose: Remove the active heap entry, record late fees and restore stock. Return: void. Project call names: RentalDataStore.require, daysOverdue, calculateLateFee
validateCart RentalManager.java:212-239	String validateCart(User client, LinkedHashMap<Game, Integer> cart) Purpose: Check each cart line against catalog, stock, rental and queue limits. Return: String. Project call names: client.isAdmin, checkoutQueue.queuedCopies, hasActiveRental

CheckoutQueue.java - 22 declarations

Manual doubly linked list with VIP insertion, fairness, cancellation and shared-stock preview.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
Node CheckoutQueue.java:11-13	Node(RentalRequest request) Purpose: Store one request and its links to nearby queue nodes. Return: constructor. Project call names: No other project method call name identified.
clear CheckoutQueue.java:22-25	void clear() Purpose: Empty the linked queue by clearing its front and rear links. Return: void. Project call names: No other project method call name identified.
refreshMembership CheckoutQueue.java:28-40	void refreshMembership(User client) Purpose: Update VIP flags and rebuild queue groups in arrival order. Return: void. Project call names: getRequests, client.isVip, sortByQueueNumber, clear, enqueue
sortByQueueNumber CheckoutQueue.java:43-53	static void sortByQueueNumber(ArrayList<RentalRequest> requests) Purpose: Use stable insertion sort to restore request arrival order; worst O(n squared). Return: void. Project call names: No other project method call name identified.
validRequest CheckoutQueue.java:56-60	static boolean validRequest(RentalRequest request) Purpose: Check the client, game, number, quantity and duration of a request. Return: boolean. Project call names: request.client.isAdmin
recordCheckout CheckoutQueue.java:63-71	void recordCheckout(RentalRequest request) Purpose: Update the VIP streak after an actual checkout. Return: void. Project call names: No other project method call name identified.
isEmpty CheckoutQueue.java:74-76	boolean isEmpty() Purpose: Check whether the linked queue has no front node. Return: boolean. Project call names: No other project method call name identified.
enqueue CheckoutQueue.java:79-103	void enqueue(RentalRequest request) Purpose: Link a request into its VIP or regular group, keeping arrival order. Return: void. Project call names: Node, isEmpty, insertBefore

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
insertBefore CheckoutQueue.java:106-115	private void insertBefore(Node target, Node node) Purpose: Connect a new node before another node in the doubly linked list. Return: void. Project call names: No other project method call name identified.
remove CheckoutQueue.java:118-125	void remove(RentalRequest request) Purpose: Find a request by traversal and unlink its queue node. Return: void. Project call names: unlink
unlink CheckoutQueue.java:128-141	private void unlink(Node node) Purpose: Reconnect nearby nodes when removing a queue node. Return: void. Project call names: No other project method call name identified.
countRequestsForGame CheckoutQueue.java:144-152	int countRequestsForGame(Game game) Purpose: Traverse the linked queue to count requests for one game. Return: int. Project call names: RentalManager.sameGame
removeRequestsForGame CheckoutQueue.java:155-167	int removeRequestsForGame(Game game) Purpose: Traverse and unlink all requests for a deleted game. Return: int. Project call names: RentalManager.sameGame, unlink
hasRequest CheckoutQueue.java:170-177	boolean hasRequest(User client, Game game) Purpose: Use linear search to check a client's queued game. Return: boolean. Project call names: RentalManager.sameUser, RentalManager.sameGame
queuedCopies CheckoutQueue.java:180-189	int queuedCopies(User client, Game game) Purpose: Traverse the queue to total a client's requested copies of one game. Return: int. Project call names: RentalManager.sameUser, RentalManager.sameGame
findByNumber CheckoutQueue.java:192-199	RentalRequest findByNumber(int queueNumber) Purpose: Use linear search through queue nodes to find a request number. Return: RentalRequest. Project call names: No other project method call name identified.
getRequestsOf CheckoutQueue.java:202-210	ArrayList<RentalRequest> getRequestsOf(User client) Purpose: Copy one client's queued requests into a list for display. Return: ArrayList<RentalRequest>. Project call names: RentalManager.sameUser
getPosition CheckoutQueue.java:213-215	int getPosition(RentalRequest request) Purpose: Find the request's predicted ready position; zero means waiting. Return: int. Project call names: previewReadyRequests
getDisplayRequests CheckoutQueue.java:218-226	ArrayList<RentalRequest> getDisplayRequests(ArrayList<RentalRequest> ready) Purpose: List predicted ready requests first, then waiting requests. Return: ArrayList<RentalRequest>. Project call names: getRequests
findNextEligibleRequest CheckoutQueue.java:229-245	RentalRequest findNextEligibleRequest() Purpose: Traverse for enough stock and apply the three-VIP fairness rule. Return: RentalRequest. Project call names: validRequest
getRequests CheckoutQueue.java:248-254	ArrayList<RentalRequest> getRequests() Purpose: Copy linked queue entries into a list without changing their links. Return: ArrayList<RentalRequest>. Project call names: No other project method call name identified.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
previewReadyRequests CheckoutQueue.java:256-286	ArrayList<RentalRequest> previewReadyRequests() Purpose: Simulate queue fairness on copied stock to predict processing order. Return: ArrayList<RentalRequest>. Project call names: getRequests, CheckoutQueue.validRequest, Game, RentalRequest, previewQueue.enqueue, previewQueue.findNextEligibleRequest, findByNumber, previewQueue.recordCheckout, previewQueue.remove

RentalDataStore.java - 24 declarations

Load and validate records; serialize all five files; checkpoint and restore memory; logs.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
RentalDataStore RentalDataStore.java:14-16	RentalDataStore(Queue2Play system) Purpose: Connect file loading and saving to the shared application data. Return: constructor. Project call names: No other project method call name identified.
file RentalDataStore.java:19-21	Path file(String name) Purpose: Return a data-file path through the shared storage object. Return: Path. Project call names: storage.file
checkpoint RentalDataStore.java:24-26	void checkpoint() Purpose: Copy current state so a failed save can restore it. Return: void. Project call names: Snapshot
Record RentalDataStore.java:33-36	Record(String[] fields, int lineNumber) Purpose: Keep a parsed row with its file line number for error reporting. Return: constructor. Project call names: No other project method call name identified.
records RentalDataStore.java:40-59	List<Record> records(String name) Purpose: Read nonempty data rows and split their fields for validation. Return: List<Record>. Project call names: file, Record, StorageException
invalidRecord RentalDataStore.java:62-65	StorageException invalidRecord(String name, Record record, RuntimeException cause) Purpose: Report the data file and line that failed validation. Return: StorageException. Project call names: StorageException
require RentalDataStore.java:68-72	static void require(boolean condition, String message) Purpose: Reject a condition that would break a data rule. Return: void. Project call names: No other project method call name identified.
positive RentalDataStore.java:75-79	static int positive(String text) Purpose: Parse and validate a positive record number. Return: int. Project call names: require
counter RentalDataStore.java:82-86	static int counter(String text) Purpose: Parse and validate a saved next-record counter. Return: int. Project call names: require
saveData RentalDataStore.java:89-99	void saveData() Purpose: Validate and save all data; restore the snapshot if saving fails. Return: void. Project call names: storage.commit, serialized, checkpoint, committed.restore

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
saveGames RentalDataStore.java:102-104	void saveGames() Purpose: Save inventory together with the related account, queue and rental data. Return: void. Project call names: saveData
saveQueue RentalDataStore.java:107-109	void saveQueue() Purpose: Save the queue together with the related application data. Return: void. Project call names: saveData
serialized RentalDataStore.java:112-158	private Map<String, String> serialized() Purpose: Build the text contents for all saved data files. Return: Map<String, String>. Project call names: validateState, require, PasswordSecurity.isHash, system.checkoutQueue.getRequests
validateState RentalDataStore.java:161-209	private void validateState() Purpose: Use sets and shared rules to reject duplicates and inconsistent records. Return: void. Project call names: require, AccountManager.validUsername, AccountManager.validName, AccountManager.validEmail, GameCatalog.validate, GameCatalog.validId, system.rentalService.rentedCopies, rental.client.isAdmin, system.rentalService.activeCopies, system.checkoutQueue.getRequests, CheckoutQueue.validRequest, request.client.isVip, system.checkoutQueue.queuedCopies
loadGames RentalDataStore.java:212-250	void loadGames() Purpose: Validate saved games and rebuild the inventory map and ID counters. Return: void. Project call names: file, system.gameCatalog.loadStartingGames, records, require, GameCatalog.validId, GameCatalog.validate, Game, system.gameCatalog.updateCounterFromId, invalidRecord
loadRentals RentalDataStore.java:253-333	void loadRentals() Purpose: Load rental records, refill the active heap and rebuild history stacks. Return: void. Project call names: records, require, counter, positive, system.accountService.findUser, client.isAdmin, GameCatalog.validId, GameCatalog.validTitle, Game, parseDate, Rental, isEmpty, RentalManager.daysOverdue, system.rentalService.calculateLateFee, system.rentalService.activeCopies, invalidRecord, system.rentalService.rentedCopies, sortRentalsByNumber
sortRentalsByNumber RentalDataStore.java:336-346	private void sortRentalsByNumber() Purpose: Use insertion sort to restore saved history record order. Return: void. Project call names: No other project method call name identified.
loadQueue RentalDataStore.java:349-399	void loadQueue() Purpose: Validate requests, sort arrivals and rebuild the linked checkout queue. Return: void. Project call names: records, require, counter, positive, system.accountService.findUser, client.isAdmin, RentalRequest, CheckoutQueue.validRequest, system.rentalService.activeCopies, RentalManager.sameUser, RentalManager.sameGame, invalidRecord, CheckoutQueue.sortByQueueNumber, system.checkoutQueue.enqueue
parseDate RentalDataStore.java:402-412	private LocalDateTime parseDate(String text, boolean end) Purpose: Read current timestamps or convert older date-only records. Return: LocalDateTime. Project call names: No other project method call name identified.
writeLog RentalDataStore.java:415-417	void writeLog(String type, String message) Purpose: Add the current timestamp before appending a log record. Return: void. Project call names: writeLogRecord

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
writeLogRecord RentalDataStore.java:420-423	void writeLogRecord(String stamp, String type, String message) Purpose: Append a validated timestamp, event type and message to the log list. Return: void. Project call names: require
loadLogs RentalDataStore.java:426-448	void loadLogs() Purpose: Restore saved logs and push admin entries onto the audit stack. Return: void. Project call names: file, require, StorageException
Snapshot RentalDataStore.java:474-512	Snapshot(Queue2Play system) Purpose: Copy lists and field values needed to undo an unsuccessful save. Return: constructor. Project call names: system.checkoutQueue.getRequests
restore RentalDataStore.java:514-559	void restore(Queue2Play system) Purpose: Restore saved state, including links, heap entries and history stacks. Return: void. Project call names: system.checkoutQueue.clear, system.checkoutQueue.enqueue

AppStorage.java - 10 declarations

Find the data directory; acquire file lock; remember bytes; recover interrupted saves; commit all files.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
AppStorage AppStorage.java:16-51	AppStorage() Purpose: Open the data folder and recover any interrupted save. Return: constructor. Project call names: lockChannel, recover, hasSavedFiles, file, rememberFiles, StorageException
hasSavedFiles AppStorage.java:54-61	private boolean hasSavedFiles() Purpose: Check whether the data folder already contains saved records. Return: boolean. Project call names: file
file AppStorage.java:64-66	Path file(String name) Purpose: Return the path of a named data file. Return: Path. Project call names: No other project method call name identified.
lockChannel AppStorage.java:69-71	private FileChannel lockChannel() throws IOException Purpose: Open the lock file used to prevent simultaneous saves. Return: FileChannel. Project call names: No other project method call name identified.
rememberFiles AppStorage.java:74-87	private void rememberFiles() throws IOException Purpose: Keep file bytes in a map to detect outside changes before saving. Return: void. Project call names: file
durableWrite AppStorage.java:90-98	private static void durableWrite(Path path, byte[] bytes) throws IOException Purpose: Write all bytes and flush them before replacing a saved file. Return: void. Project call names: No other project method call name identified.
replace AppStorage.java:101-107	private static void replace(Path from, Path to) throws IOException Purpose: Replace a file, using an atomic move when supported. Return: void. Project call names: No other project method call name identified.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
cleanJournal AppStorage.java:110-122	private void cleanJournal() throws IOException Purpose: Delete only the known temporary transaction files. Return: void. Project call names: No other project method call name identified.
recover AppStorage.java:125-142	private void recover() throws IOException Purpose: Restore files from an interrupted transaction before cleanup. Return: void. Project call names: file, cleanJournal
commit AppStorage.java:145-207	synchronized void commit(Map<String, String> contents) Purpose: Save related files together with locking and interrupted-save recovery. Return: void. Project call names: lockChannel, recover, file, durableWrite, replace, rememberFiles, cleanJournal, StorageException

PasswordSecurity.java - 6 declarations

Create and verify salted PBKDF2 password hashes; validate stored hash encoding.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
isHash PasswordSecurity.java:12-14	static boolean isHash(String value) Purpose: Check whether a saved password uses the PBKDF2 format. Return: boolean. Project call names: No other project method call name identified.
hash PasswordSecurity.java:17-26	static String hash(String password) Purpose: Keep a valid existing hash or hash a plaintext password. Return: String. Project call names: isHash, validateHash, hashPlaintext
hashPlaintext PasswordSecurity.java:29-37	static String hashPlaintext(String password) Purpose: Create a salted PBKDF2 password hash for safe storage. Return: String. Project call names: derive
validateHash PasswordSecurity.java:40-50	static void validateHash(String encoded) Purpose: Check the saved hash format, iteration count, salt and key length. Return: void. Project call names: No other project method call name identified.
matches PasswordSecurity.java:53-65	static boolean matches(String password, String encoded) Purpose: Verify a password against its saved hash without exposing the password. Return: boolean. Project call names: validateHash, derive
derive PasswordSecurity.java:68-77	private static byte[] derive(String password, byte[] salt, int iterations) Purpose: Use Java's PBKDF2 implementation to derive the password key. Return: byte[]. Project call names: No other project method call name identified.

User.java - 3 declarations

Username, full name, email, hash, role, membership and a rental-history stack. Role and membership queries.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
User User.java:15-23	User(String username, String fullName, String email, String password, String role, String membership) Purpose: Store account details and hash the password before keeping it. Return: constructor. Project call names: PasswordSecurity.hash
isVip User.java:26-28	boolean isVip() Purpose: Check whether the account has VIP membership. Return: boolean. Project call names: No other project method call name identified.
isAdmin User.java:31-33	boolean isAdmin() Purpose: Check whether the account can access administrator functions. Return: boolean. Project call names: No other project method call name identified.

Game.java - 2 declarations

Game ID, title, platform, three-day base price and available stock.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
Game Game.java:9-15	Game(String gameId, String title, String platform, double price, int availableCopies) Purpose: Store a game's identifier, platform, price and available stock. Return: constructor. Project call names: No other project method call name identified.
isAvailable Game.java:18-20	boolean isAvailable() Purpose: Check whether a game has any available copies. Return: boolean. Project call names: No other project method call name identified.

RentalRequest.java - 1 declarations

Queue number, client, game, VIP flag, rental days and quantity. Default request is three days and one copy.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
RentalRequest RentalRequest.java:11-16	RentalRequest(int queueNumber, User client, Game game) Purpose: Store the client, game and membership for a queued order line. Return: constructor. Project call names: client.isVip

Rental.java - 2 declarations

Rental/history number, client, game, dates, amount, quantity, status and return/late-fee fields. Implements Comparable.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
Rental Rental.java:17-26	Rental(int rentalNumber, User client, Game game, LocalDateTime checkoutDate, LocalDateTime dueDate, double amount) Purpose: Store rental dates, quantity, amount and current status. Return: constructor. Project call names: No other project method call name identified.
compareTo Rental.java:30-42	public int compareTo(Rental other) Purpose: Order rentals by due date, then record number, for the min-heap. Return: int. Project call names: No other project method call name identified.

TerminalUI.java - 227 declarations

Input, masking, colors, headings, tables, wrapping, pagination, messages, confirmations and result navigation.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
configuredWidth TerminalUI.java:47-54	private static int configuredWidth() Purpose: Read the configured terminal width or use the default. Return: int. Project call names: No other project method call name identified.
configuredHeight TerminalUI.java:56-62	private static int configuredHeight() Purpose: Read the configured terminal height or use the default. Return: int. Project call names: No other project method call name identified.
pageStarts TerminalUI.java:65-82	ArrayList<Integer> pageStarts(int[] heights, int maximum, int reservedRows) Purpose: Build page boundaries from record heights while keeping each record together. Return: ArrayList<Integer>. Project call names: No other project method call name identified.
wrappedLines TerminalUI.java:85-87	private int wrappedLines(String text, int width) Purpose: Count how many terminal rows a piece of wrapped text needs. Return: int. Project call names: wrapText
columnPart TerminalUI.java:90-96	private String columnPart(String text, int row, int width) Purpose: Get one wrapped line from a table cell. Return: String. Project call names: wrapText
wrapText TerminalUI.java:99-111	private ArrayList<String> wrapText(String text, int width) Purpose: Split text into fitting lines, preferring word boundaries. Return: ArrayList<String>. Project call names: No other project method call name identified.
displayWidths TerminalUI.java:114-134	private int[] displayWidths(int[] widths, int tableWidth) Purpose: Shrink table columns to fit the terminal width. Return: int[]. Project call names: No other project method call name identified.
recordHeight TerminalUI.java:137-147	private int recordHeight(String[] cells, int[] widths, int tableWidth) Purpose: Measure the rows needed to display a complete table record. Return: int. Project call names: wrapText, recordText, displayWidths, wrappedLines
recordText TerminalUI.java:150-161	private String recordText(String[] cells) Purpose: Join nonempty fields for compact record display. Return: String. Project call names: No other project method call name identified.
printRecord TerminalUI.java:164-204	private void printRecord(String format, String[] cells, int[] widths) Purpose: Print a full record with wrapped cells or compact narrow-screen lines. Return: void. Project call names: wrapText, recordText, printLine, displayWidths, wrappedLines, columnPart
gamePages TerminalUI.java:207-218	ArrayList<Integer> gamePages(ArrayList<Game> games) Purpose: Use an array of game row heights to build pages of at most nine games. Return: ArrayList<Integer>. Project call names: recordHeight, money, pageStarts
rentalPages TerminalUI.java:221-270	ArrayList<Integer> rentalPages(ArrayList<Rental> rentals, String kind) Purpose: Use rental row heights to keep each page within its record and height limits. Return: ArrayList<Integer>. Project call names: recordHeight, rentalLabel, historyDueDate, money, pageStarts

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
auditPages TerminalUI.java:273-292	ArrayList<Integer> auditPages(ArrayList<String> logs) Purpose: Measure wrapped audit entries before assigning them to pages. Return: ArrayList<Integer>. Project call names: recordHeight, pageStarts
queuePages TerminalUI.java:295-306	ArrayList<Integer> queuePages(ArrayList<RentalRequest> requests) Purpose: Measure queue records before grouping them into fitting pages. Return: ArrayList<Integer>. Project call names: recordHeight, queueLabel, pageStarts
showPagination TerminalUI.java:309-318	void showPagination(int page, int pages) Purpose: Display only the Next and Previous keys that can be used. Return: void. Project call names: showChoiceGroup
showListStatus TerminalUI.java:321-332	void showListStatus(String first, String second, int width) Purpose: Display the list order, page and counts in a divided status box. Return: void. Project call names: printLine, border, printBoxCentered, flushBorder
confirmDiscardTransaction TerminalUI.java:335-350	boolean confirmDiscardTransaction(String transaction, String destination) Purpose: Ask before discarding unsaved work and leaving the current step. Return: boolean. Project call names: printLine, border, printWrappedBoxText, inputBar, readText, prompt, error
colorChoices TerminalUI.java:373-421	private String colorChoices(String text) Purpose: Apply option colors while keeping their visible labels unchanged. Return: String. Project call names: color
moveCheckoutPage TerminalUI.java:465-475	boolean moveCheckoutPage(String choice) Purpose: Move to a valid checkout queue page when N or P is entered. Return: boolean. Project call names: No other project method call name identified.
color TerminalUI.java:479-484	private String color(String text, String tone) Purpose: Wrap text in the selected ANSI color when colors are enabled. Return: String. Project call names: No other project method call name identified.
colorStatuses TerminalUI.java:487-521	private String colorStatuses(String text) Purpose: Apply the existing platform and status colors to matching words. Return: String. Project call names: color
styleContent TerminalUI.java:524-541	private String styleContent(String text) Purpose: Apply shared text colors and symbols to a displayed line. Return: String. Project call names: colorChoices, color, matches, colorStatuses
styleSeparators TerminalUI.java:557-567	private String styleSeparators(String text) Purpose: Color table separators without changing their positions. Return: String. Project call names: styleContent, color
printLine TerminalUI.java:570-572	private void printLine() Purpose: Render a styled terminal line, or a blank line when no text is supplied. Return: void. Project call names: emitLine
writeTerminal TerminalUI.java:575-591	private void writeTerminal(String text, boolean newLine) Purpose: Write text with the screen's horizontal margin and optional newline. Return: void. Project call names: No other project method call name identified.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
emit TerminalUI.java:594-598	private void emit(String text) Purpose: Write text without adding a newline. Return: void. Project call names: flushBorder, writeTerminal
emitLine TerminalUI.java:601-605	private void emitLine(String text) Purpose: Write text followed by a newline. Return: void. Project call names: flushBorder, writeTerminal
wall TerminalUI.java:608-613	private String wall() Purpose: Choose the side-border character for the current box style. Return: String. Project call names: color
queueBorder TerminalUI.java:616-637	private void queueBorder(String line) Purpose: Hold a border until nearby rows reveal its required shape. Return: void. Project call names: flushBorder
flushBorder TerminalUI.java:640-653	private void flushBorder(boolean nextIsRow) Purpose: Print a pending border with the correct connections. Return: void. Project call names: renderBorder, writeTerminal
renderBorder TerminalUI.java:656-695	private String renderBorder(String line, char fill, boolean light, int edge, String tone) Purpose: Build a colored border using the selected style and corners. Return: String. Project call names: color
printStyledRow TerminalUI.java:698-702	private void printStyledRow(String styled) Purpose: Print a styled box row with matching side borders. Return: void. Project call names: flushBorder, writeTerminal
printLine TerminalUI.java:705-729	private void printLine(String line) Purpose: Render a styled terminal line, or a blank line when no text is supplied. Return: void. Project call names: printLine, printAsciiLine, queueBorder, printStyledRow, wall, styleSeparators, emitLine, color, replace
printAsciiLine TerminalUI.java:732-750	private void printAsciiLine(String line) Purpose: Print the plain-character version of a line. Return: void. Project call names: emitLine, color, styleSeparators
printFormatted TerminalUI.java:753-757	private void printFormatted(String output) Purpose: Split formatted output into lines for shared rendering. Return: void. Project call names: printLine
readText TerminalUI.java:760-780	String readText(String prompt) Purpose: Read and trim a text line, stopping safely when input ends. Return: String. Project call names: showPendingMessage, emit, readWindowsInput, printLine
readPassword TerminalUI.java:783-804	String readPassword(String prompt) Purpose: Read a hidden password without trimming or printing its contents. Return: String. Project call names: showPendingMessage, emit, readWindowsInput, readPassword

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
readWindowsInput TerminalUI.java:807-848	private String readWindowsInput(boolean mask) Purpose: Capture Windows key input with optional password masking. Return: String. Project call names: No other project method call name identified.
readNumber TerminalUI.java:851-858	int readNumber(String prompt) Purpose: Read a whole-number choice and handle invalid input. Return: int. Project call names: readText
inputBar TerminalUI.java:861-863	void inputBar(String label) Purpose: Draw the input separator and any field-specific guidance. Return: void. Project call names: inputBar
inputBar TerminalUI.java:866-868	void inputBar(String label, int width) Purpose: Draw the input separator and any field-specific guidance. Return: void. Project call names: inputBar, inputGuide
inputBar TerminalUI.java:871-942	void inputBar(String label, int width, String guide) Purpose: Draw the input separator and any field-specific guidance. Return: void. Project call names: showPendingMessage, showNavigationOptions, flushBorder, printLine, border, printBoxCentered, printMessageRows, wrapText, emitLine, color, center, fit
inputGuide TerminalUI.java:945-963	private String inputGuide(String label) Purpose: Choose the short helper text for the current input field. Return: String. Project call names: money
prompt TerminalUI.java:966-968	String prompt() Purpose: Build the pink input marker with the correct screen margin. Return: String. Project call names: prompt
prompt TerminalUI.java:971-974	String prompt(int width) Purpose: Build the pink input marker with the correct screen margin. Return: String. Project call names: color
clearScreen TerminalUI.java:977-990	void clearScreen() Purpose: Start the next screen at the top when terminal clearing is enabled. Return: void. Project call names: No other project method call name identified.
pause TerminalUI.java:993-1002	void pause(int milliseconds) Purpose: Wait briefly for an animation unless fast mode is enabled. Return: void. Project call names: No other project method call name identified.
loading TerminalUI.java:1005-1021	void loading(String message) Purpose: Display a short progress animation for the current action. Return: void. Project call names: printLine, fit, emit, color, center
finishScreen TerminalUI.java:1024-1026	void finishScreen() Purpose: Finish pending borders before showing result navigation. Return: void. Project call names: finishScreen

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
finishScreen TerminalUI.java:1029-1033	void finishScreen(int width) Purpose: Finish pending borders before showing result navigation. Return: void. Project call names: flushBorder
waitForResultNavigation TerminalUI.java:1036-1053	boolean waitForResultNavigation(String previousDestination, String mainDestination) Purpose: Keep a completed result visible until its named destination is chosen. Return: boolean. Project call names: showNavigationOptions, inputBar, readText, prompt, clearResultRetry
clearResultRetry TerminalUI.java:1056-1061	private void clearResultRetry(String message) Purpose: Clear the retry area before showing another navigation error. Return: void. Project call names: clearScreen, error
success TerminalUI.java:1064-1067	void success(String message) Purpose: Display or defer a success message for the current screen. Return: void. Project call names: showPendingMessage, printMessageBox
error TerminalUI.java:1070-1073	void error(String message) Purpose: Display or defer an error message for the current screen. Return: void. Project call names: No other project method call name identified.
warning TerminalUI.java:1076-1079	void warning(String message) Purpose: Display or defer a warning message for the current screen. Return: void. Project call names: No other project method call name identified.
info TerminalUI.java:1082-1088	void info(String message) Purpose: Display an informational line using the shared renderer. Return: void. Project call names: showNavigationOptions, printLine
showPendingMessage TerminalUI.java:1091-1102	private void showPendingMessage(int width) Purpose: Display a deferred message before the next input prompt. Return: void. Project call names: showMessageCard, printMessageBox
printMessageBox TerminalUI.java:1105-1126	private void printMessageBox(String text, int width) Purpose: Display a message in the screen's matching card style. Return: void. Project call names: printLine, border, printMessageRows
printMessageRows TerminalUI.java:1129-1150	private void printMessageRows(String text, int width) Purpose: Wrap and center message text inside its card. Return: void. Project call names: printBoxCentered
showEmptyState TerminalUI.java:1153-1159	void showEmptyState(String message) Purpose: Keep an empty-list message visible within the current screen. Return: void. Project call names: printLine, border, printBoxCentered, printMessageRows
platformColor TerminalUI.java:1162-1165	String platformColor(String platform) Purpose: Prepare a platform label for the shared color renderer. Return: String. Project call names: No other project method call name identified.
membershipColor TerminalUI.java:1167-1172	String membershipColor(boolean vip) Purpose: Return the VIP or regular membership label. Return: String. Project call names: No other project method call name identified.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
statusColor TerminalUI.java:1174-1176	String statusColor(String status) Purpose: Prepare a status label for the shared color renderer. Return: String. Project call names: No other project method call name identified.
printHeader TerminalUI.java:1179-1181	void printHeader(String title) Purpose: Start a screen with its title and optional path at the chosen width. Return: void. Project call names: printHeader
printHeader TerminalUI.java:1184-1193	void printHeader(String title, int width) Purpose: Start a screen with its title and optional path at the chosen width. Return: void. Project call names: clearScreen, printLine, border, printTitle
printHeader TerminalUI.java:1196-1198	void printHeader(String path, String title) Purpose: Start a screen with its title and optional path at the chosen width. Return: void. Project call names: printHeader
printHeader TerminalUI.java:1201-1212	void printHeader(String path, String title, int width) Purpose: Start a screen with its title and optional path at the chosen width. Return: void. Project call names: clearScreen, printLine, border, printTitle
tableLine TerminalUI.java:1215-1217	String tableLine(int width) Purpose: Build a table separator matching the requested width. Return: String. Project call names: No other project method call name identified.
border TerminalUI.java:1220-1225	String border(int width, char fill) Purpose: Build a box border using the requested width and fill character. Return: String. Project call names: No other project method call name identified.
printBoxCentered TerminalUI.java:1228-1231	void printBoxCentered(String text, int width) Purpose: Center content inside the current box borders. Return: void. Project call names: printLine, center, fit
printTitle TerminalUI.java:1234-1239	private void printTitle(String title, int width) Purpose: Print a centered screen title at the chosen width. Return: void. Project call names: printBoxCentered
printCenteredDetails TerminalUI.java:1242-1278	private void printCenteredDetails(String[] details) Purpose: Display centered detail rows in a shared detail box. Return: void. Project call names: printLine, border, printBoxCentered
showFeatureOptions TerminalUI.java:1281-1283	private void showFeatureOptions(String[] options) Purpose: Display action choices using the shared choices layout. Return: void. Project call names: showNavigationOptions
showMenuOptions TerminalUI.java:1286-1303	private void showMenuOptions(String[] options) Purpose: Display a vertical numbered menu with padded rows. Return: void. Project call names: No other project method call name identified.
fit TerminalUI.java:1306-1320	String fit(String text, int maxLength) Purpose: Shorten text only when it exceeds the available display space. Return: String. Project call names: No other project method call name identified.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
center TerminalUI.java:1323-1333	private String center(String text, int width) Purpose: Pad text so it appears centered within the given width. Return: String. Project call names: No other project method call name identified.
money TerminalUI.java:1336-1338	String money(double amount) Purpose: Format an amount with a dollar sign and two decimal places. Return: String. Project call names: No other project method call name identified.
summaryText TerminalUI.java:1341-1346	private String summaryText(String text, int width) Purpose: Fit summary text into the available box width. Return: String. Project call names: No other project method call name identified.
rentalDurationText TerminalUI.java:1349-1354	private String rentalDurationText(int days) Purpose: Format a duration with the correct day or days label. Return: String. Project call names: No other project method call name identified.
queueLabel TerminalUI.java:1356-1358	String queueLabel(int number) Purpose: Format a queue number with leading zeros. Return: String. Project call names: No other project method call name identified.
rentalLabel TerminalUI.java:1360-1362	String rentalLabel(int number) Purpose: Format a rental or history number with leading zeros. Return: String. Project call names: No other project method call name identified.
showNavigationOptions TerminalUI.java:1365-1367	void showNavigationOptions(String[] options) Purpose: Display grouped destination choices using the current or supplied width. Return: void. Project call names: showNavigationOptions
showNavigationOptions TerminalUI.java:1370-1387	void showNavigationOptions(int width, String[] options) Purpose: Display grouped destination choices using the current or supplied width. Return: void. Project call names: showChoiceGroup
showChoiceGroup TerminalUI.java:1390-1423	void showChoiceGroup(String[] options) Purpose: Center related choices and wrap them with spacing between categories. Return: void. Project call names: No other project method call name identified.
showBoxNavigationOptions TerminalUI.java:1426-1428	private void showBoxNavigationOptions(String[] options) Purpose: Display menu navigation inside the existing padded choices box. Return: void. Project call names: showNavigationOptions
showSelectQuantityScreen TerminalUI.java:1431-1433	void showSelectQuantityScreen(Game game, int inCart, int maxPerGame, int maximumToAdd) Purpose: Display stock and copy allowances before quantity input. Return: void. Project call names: showSelectQuantityScreen
showSelectQuantityScreen TerminalUI.java:1436-1451	void showSelectQuantityScreen(Game game, int inCart, int queued, int maxPerGame, int maximumToAdd) Purpose: Display stock and copy allowances before quantity input. Return: void. Project call names: printHeader, printCenteredDetails, platformColor, printBoxCentered, showNavigationOptions, printLine, border, inputBar

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
showYesNoPrompt TerminalUI.java:1454-1463	void showYesNoPrompt(String question) Purpose: Display a centered confirmation question and its Yes/No choices. Return: void. Project call names: printLine, border, printBoxCentered, printWrappedBoxText, inputBar
printWrappedBoxText TerminalUI.java:1466-1488	private void printWrappedBoxText(String text, int width) Purpose: Wrap long text into complete rows inside a box. Return: void. Project call names: printBoxCentered
showMainMenu TerminalUI.java:1491-1504	void showMainMenu() Purpose: Display the landing page with Login, Register and Exit choices. Return: void. Project call names: clearScreen, showPlainMainMenu, printLine, printArtBlock, printLogo, showMenuOptions, showBoxNavigationOptions, inputBar
showPlainMainMenu TerminalUI.java:1507-1512	private void showPlainMainMenu() Purpose: Display the simpler landing page for plain terminal output. Return: void. Project call names: printHeader, showMenuOptions, showBoxNavigationOptions, inputBar
printArtBlock TerminalUI.java:1515-1528	private void printArtBlock(String[] art) Purpose: Print aligned logo artwork line by line. Return: void. Project call names: printStyledRow, wall, color, center
printLogo TerminalUI.java:1531-1541	void printLogo() Purpose: Display the Queue2Play logo. Return: void. Project call names: printStyledRow, wall, color, center
showGoodbye TerminalUI.java:1544-1559	void showGoodbye() Purpose: Display the application's exit message. Return: void. Project call names: clearScreen, emitLine, color, printLine, center
showLoginUsernameScreen TerminalUI.java:1562-1567	void showLoginUsernameScreen() Purpose: Display username-or-email input and its navigation choices. Return: void. Project call names: printHeader, printCenteredDetails, showBoxNavigationOptions, inputBar
showLoginPasswordScreen TerminalUI.java:1570-1575	void showLoginPasswordScreen(String login) Purpose: Display password input for the selected login identifier. Return: void. Project call names: printHeader, printCenteredDetails, fit, showBoxNavigationOptions, inputBar
showRegistrationStep TerminalUI.java:1578-1603	void showRegistrationStep(int step, boolean editing, String fullName, String username, String email) Purpose: Display the current registration field and preserved draft details. Return: void. Project call names: printHeader, printCenteredDetails, showNavigationOptions, inputBar
showRegistrationEditMenu TerminalUI.java:1606-1612	void showRegistrationEditMenu(String fullName, String username, String email) Purpose: Display the fields that can be edited without resetting the draft. Return: void. Project call names: printHeader, printCenteredDetails, showMenuOptions, showNavigationOptions, inputBar
confirmRegistrationCancellation TerminalUI.java:1615-1629	boolean confirmRegistrationCancellation() Purpose: Ask before discarding unsaved registration details. Return: boolean. Project call names: printHeader, printBoxCentered, inputBar, readText, prompt, error

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
showLoginCooldown TerminalUI.java:1632-1644	void showLoginCooldown() Purpose: Display the countdown while waiting through the login cooldown. Return: void. Project call names: showPendingMessage, printMessageBox, flushBorder, showCooldownRemaining, waitLoginSecond, emitLine, printLine
showLoginAttempt TerminalUI.java:1647-1663	void showLoginAttempt(int attempt) Purpose: Display the failed attempt number and remaining attempts. Return: void. Project call names: No other project method call name identified.
showCooldownRemaining TerminalUI.java:1666-1670	void showCooldownRemaining(int seconds) Purpose: Update the countdown on the same terminal line. Return: void. Project call names: emit, color, center
waitLoginSecond TerminalUI.java:1673-1680	void waitLoginSecond() Purpose: Wait one real second for the login cooldown. Return: void. Project call names: No other project method call name identified.
showPlatformMenu TerminalUI.java:1682-1684	void showPlatformMenu(String heading) Purpose: Display platform choices and the applicable navigation destinations. Return: void. Project call names: showPlatformMenu
showPlatformMenu TerminalUI.java:1687-1689	void showPlatformMenu(String heading, String destination) Purpose: Display platform choices and the applicable navigation destinations. Return: void. Project call names: showPlatformMenu
showPlatformMenu TerminalUI.java:1692-1712	void showPlatformMenu(String heading, String destination, boolean hasCart) Purpose: Display platform choices and the applicable navigation destinations. Return: void. Project call names: printHeader, showPlatformChoices, showChoiceGroup, showBoxNavigationOptions, inputBar
showPlatformChoices TerminalUI.java:1715-1717	private void showPlatformChoices() Purpose: Display the four catalog filters with their existing colors. Return: void. Project call names: showMenuOptions
showBrowseMenu TerminalUI.java:1720-1728	void showBrowseMenu(boolean fromInventory) Purpose: Display platform browsing choices for the current destination. Return: void. Project call names: printHeader, showPlatformChoices, showBoxNavigationOptions, inputBar
showClientMenu TerminalUI.java:1731-1737	void showClientMenu(User client) Purpose: Display the client's account details and dashboard choices. Return: void. Project call names: printHeader, printCenteredDetails, fit, membershipColor, client.isVip, showMenuOptions, showBoxNavigationOptions, inputBar
showAdminMenu TerminalUI.java:1740-1745	void showAdminMenu() Purpose: Display the administrator's dashboard choices. Return: void. Project call names: printHeader, showMenuOptions, showBoxNavigationOptions, inputBar
showActiveRentalsMenu TerminalUI.java:1748-1753	void showActiveRentalsMenu() Purpose: Display the active-rental list choice and Dashboard navigation. Return: void. Project call names: printHeader, showMenuOptions, showBoxNavigationOptions, inputBar

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
showInventoryMenu TerminalUI.java:1756-1761	void showInventoryMenu() Purpose: Display the inventory-management menu. Return: void. Project call names: printHeader, showMenuOptions, showBoxNavigationOptions, inputBar
showQueue TerminalUI.java:1764-1785	void showQueue(CheckoutQueue queue) Purpose: Display queued requests without removing them. Return: void. Project call names: queue.isEmpty, showEmptyState, printLine, tableLine, printFormatted, queue.getRequests, queueLabel, membershipColor, fit
askYesNo TerminalUI.java:1788-1800	boolean askYesNo(String question) Purpose: Read a Yes or No answer and retry invalid input. Return: boolean. Project call names: showYesNoPrompt, readText, prompt, error
showOrderSummary TerminalUI.java:1803-1805	void showOrderSummary(User client, LinkedHashMap<Game, Integer> cart, int days, RentalManager rentalService) Purpose: Display the current cart, duration and calculated costs. Return: void. Project call names: showCartPage, cartPages
cartRows TerminalUI.java:1808-1824	private ArrayList<String[]> cartRows(User client, LinkedHashMap<Game, Integer> cart, int days, RentalManager service, boolean editing) Purpose: Copy cart entries into numbered display rows without changing the cart map. Return: ArrayList<String[]>. Project call names: money, service.calculateRentalCost
cartSummary TerminalUI.java:1827-1845	private String[] cartSummary(User client, LinkedHashMap<Game, Integer> cart, int days, RentalManager service, boolean editing) Purpose: Calculate full-cart totals for the summary display. Return: String[]. Project call names: rentalDurationText, service.calculateRentalCost, client.isVip, money
cartPages TerminalUI.java:1848-1858	ArrayList<Integer> cartPages(User client, LinkedHashMap<Game, Integer> cart, int days, RentalManager service, boolean editing) Purpose: Measure cart rows to build fitting order-summary pages. Return: ArrayList<Integer>. Project call names: transactionPages, cartRows, cartSummary
showCartPage TerminalUI.java:1861-1880	void showCartPage(User client, LinkedHashMap<Game, Integer> cart, int days, RentalManager service, boolean editing, int page, ArrayList<Integer> starts) Purpose: Display one cart page while keeping whole-cart line numbers. Return: void. Project call names: clearResultRetry, showTransactionPage, cartRows, cartSummary
showCartActions TerminalUI.java:1883-1900	void showCartActions(boolean editing, int page, int pages) Purpose: Display order actions and only usable page controls. Return: void. Project call names: showTransactionNavigation, showPagination, showFeatureOptions, showNavigationOptions, showOrderConfirmationOptions, inputBar
showGameTableHeader TerminalUI.java:1903-1909	void showGameTableHeader(int total, int start, int end) Purpose: Display the game catalog's column headings. Return: void. Project call names: printLine, tableLine, printRecord
showGameRow TerminalUI.java:1912-1921	void showGameRow(int shownNumber, Game game) Purpose: Display a numbered game with platform and stock status. Return: void. Project call names: game.isAvailable, printRecord, money

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
showGameNavigation TerminalUI.java:1924-1926	void showGameNavigation() Purpose: Display usable catalog controls and the correct return destinations. Return: void. Project call names: showGameNavigation
confirmDiscardCart TerminalUI.java:1929-1945	boolean confirmDiscardCart() Purpose: Ask before discarding the cart and returning to Dashboard. Return: boolean. Project call names: printLine, border, printWrappedBoxText, inputBar, readText, prompt, error
showGameNavigation TerminalUI.java:1948-1950	void showGameNavigation(int page, int pages, String keyword) Purpose: Display usable catalog controls and the correct return destinations. Return: void. Project call names: showGameNavigation
showGameNavigation TerminalUI.java:1953-1992	void showGameNavigation(int page, int pages, String keyword, boolean selectable, boolean hasCart, int shown) Purpose: Display usable catalog controls and the correct return destinations. Return: void. Project call names: showChoiceGroup
showCatalogState TerminalUI.java:1995-1997	void showCatalogState(String keyword, int sortOption, int page, int totalPages) Purpose: Display search, sort, page and optional cart totals. Return: void. Project call names: showCatalogState
showCatalogState TerminalUI.java:2000-2024	void showCatalogState(String keyword, int sortOption, int page, int totalPages, LinkedHashMap<Game, Integer> cart) Purpose: Display search, sort, page and optional cart totals. Return: void. Project call names: printLine, border, printLeftRightBox, flushBorder
printLeftRightBox TerminalUI.java:2027-2032	private void printLeftRightBox(String left, String right, int width) Purpose: Align two status values at opposite sides of a box. Return: void. Project call names: fit, printLine
showRentalDurationOptions TerminalUI.java:2035-2037	void showRentalDurationOptions() Purpose: Display preset and custom durations with the correct Back destination. Return: void. Project call names: showRentalDurationOptions
showRentalDurationOptions TerminalUI.java:2040-2048	void showRentalDurationOptions(String previousScreen) Purpose: Display preset and custom durations with the correct Back destination. Return: void. Project call names: showFeatureOptions, showNavigationOptions
showEditOrderTableHeader TerminalUI.java:2051-2056	void showEditOrderTableHeader() Purpose: Display the editable cart's column headings. Return: void. Project call names: printLine, tableLine, printFormatted
showCartRow TerminalUI.java:2059-2063	void showCartRow(int quantity, int line, Game game) Purpose: Display one numbered cart line with its quantity and price. Return: void. Project call names: printFormatted, fit, platformColor
showEditOrderOptions TerminalUI.java:2066-2070	void showEditOrderOptions(int days) Purpose: Display add, remove and duration-edit choices. Return: void. Project call names: printCenteredDetails, rentalDurationText, showFeatureOptions, showNavigationOptions

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
showOrderConfirmationOptions TerminalUI.java:2073-2076	void showOrderConfirmationOptions() Purpose: Display submit, edit and discard order choices. Return: void. Project call names: showFeatureOptions, showNavigationOptions
showSubmittedQueueHeader TerminalUI.java:2079-2084	void showSubmittedQueueHeader() Purpose: Display headings for newly submitted requests. Return: void. Project call names: printLine, tableLine, printRecord
showQueueRequestRow TerminalUI.java:2087-2096	void showQueueRequestRow(int position, RentalRequest request) Purpose: Display a request's predicted ready position or waiting status. Return: void. Project call names: printRecord, queueLabel
showQueueRequestHeader TerminalUI.java:2099-2103	void showQueueRequestHeader() Purpose: Display the customer's queued-request column headings. Return: void. Project call names: printLine, tableLine, printRecord
showHistoryTableHeader TerminalUI.java:2106-2110	void showHistoryTableHeader(int total, int start, int end) Purpose: Display history columns for rental and cancellation records. Return: void. Project call names: printLine, tableLine, printRecord
showHistoryRow TerminalUI.java:2113-2121	void showHistoryRow(Rental rental) Purpose: Display one history record with any recorded return details. Return: void. Project call names: printRecord, rentalLabel, historyDueDate, money, printLine
showBackToDashboard TerminalUI.java:2124-2126	void showBackToDashboard() Purpose: Display the Dashboard return choice. Return: void. Project call names: showNavigationOptions
checkoutRow TerminalUI.java:2129-2131	private void checkoutRow(String text) Purpose: Pad a checkout card row to the fixed 81-column width. Return: void. Project call names: printBoxCentered, fit
checkoutReady TerminalUI.java:2134-2136	boolean checkoutReady(RentalRequest request) Purpose: Check whether a request is valid and has enough stock for display. Return: boolean. Project call names: CheckoutQueue.validRequest
checkoutStatus TerminalUI.java:2139-2150	String checkoutStatus(RentalRequest request) Purpose: Return the request's ready or blocked display status. Return: String. Project call names: CheckoutQueue.validRequest, checkoutReady
checkoutStock TerminalUI.java:2153-2162	String checkoutStock(RentalRequest request) Purpose: Format the available and requested stock for a queue row. Return: String. Project call names: checkoutReady
checkoutStreak TerminalUI.java:2165-2175	String checkoutStreak(int streak) Purpose: Build the three-marker VIP streak display. Return: String. Project call names: No other project method call name identified.
checkoutNext TerminalUI.java:2178-2186	String checkoutNext(CheckoutQueue queue, RentalManager service) Purpose: Describe the next eligible request or report that none is ready. Return: String. Project call names: queue.findNextEligibleRequest, queueLabel, fit, money, service.calculateRentalCost

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
checkoutPageStarts TerminalUI.java:2189-2204	private ArrayList<Integer> checkoutPageStarts(ArrayList<RentalRequest> requests) Purpose: Measure checkout queue rows to build its page boundaries. Return: ArrayList<Integer>. Project call names: wrappedLines, pageStarts
historyDueDate TerminalUI.java:2207-2212	String historyDueDate(Rental rental) Purpose: Show no due date for cancelled requests; show it for rentals. Return: String. Project call names: No other project method call name identified.
showCheckoutQueue TerminalUI.java:2216-2297	void showCheckoutQueue(CheckoutQueue queue, RentalManager service) Purpose: Display predicted queue order with separate status and choices. Return: void. Project call names: clearScreen, queue.previewReadyRequests, queue.getDisplayRequests, checkoutPageStarts, printLine, border, checkoutRow, flushBorder, checkoutStreak, showEmptyState, tableLine, membershipColor, printRecord, queueLabel, fit, checkoutNext, showTapeInput
showCheckoutTapeFooter TerminalUI.java:2300-2305	void showCheckoutTapeFooter(Queue2Play.CheckoutSession session, CheckoutQueue queue, RentalManager service) Purpose: Display session status and input after successful tape actions. Return: void. Project call names: checkoutPageStarts, queue.getDisplayRequests, queue.previewReadyRequests, showSessionStatus, showTapeInput
beginCheckoutCard TerminalUI.java:2308-2316	private void beginCheckoutCard(char fill) Purpose: Start a fixed-width result or message card. Return: void. Project call names: flushBorder, printLine, border
endCheckoutCard TerminalUI.java:2319-2324	private void endCheckoutCard(char fill) Purpose: Close the current checkout card. Return: void. Project call names: printLine, border, flushBorder
showSlipTitle TerminalUI.java:2327-2334	private void showSlipTitle(String title, int number, LocalDateTime timestamp, String tone) Purpose: Align a slip's action, session number and timestamp. Return: void. Project call names: checkoutRow, fit, printLine, border
showApprovedSlip TerminalUI.java:2337-2346	void showApprovedSlip(Rental rental, int number) Purpose: Display one numbered approval slip with rental cost and dates. Return: void. Project call names: beginCheckoutCard, showSlipTitle, checkoutRow, rentalLabel, fit, money, endCheckoutCard
showCancelledSlip TerminalUI.java:2349-2356	void showCancelledSlip(RentalRequest request, int number) Purpose: Display one numbered cancellation slip without a rental charge. Return: void. Project call names: beginCheckoutCard, showSlipTitle, checkoutRow, queueLabel, fit, endCheckoutCard
showSessionStatus TerminalUI.java:2359-2367	void showSessionStatus(Queue2Play.CheckoutSession session, CheckoutQueue queue, RentalManager service) Purpose: Display session totals, VIP streak and the next eligible request. Return: void. Project call names: beginCheckoutCard, money, checkoutRow, fit, checkoutStreak, printLine, border, checkoutNext, endCheckoutCard
showTapeInput TerminalUI.java:2370-2383	void showTapeInput() Purpose: Display the checkout actions and input prompt area. Return: void. Project call names: showPendingMessage, flushBorder, showChoiceGroup, showPagination, inputBar

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
showMessageCard TerminalUI.java:2386-2394	void showMessageCard(String message) Purpose: Display a separate centered warning or error card. Return: void. Project call names: beginCheckoutCard, printMessageRows, endCheckoutCard
showConfirmQuestion TerminalUI.java:2397-2406	void showConfirmQuestion(String text) Purpose: Center a question between full-width pink separators. Return: void. Project call names: showPendingMessage, flushBorder, emitLine, color, colorChoices, center
confirmCheckoutAction TerminalUI.java:2409-2421	private boolean confirmCheckoutAction(String question) Purpose: Read Yes/No and redraw only the question after an invalid answer. Return: boolean. Project call names: showConfirmQuestion, readText, prompt, error
confirmCheckout TerminalUI.java:2424-2430	boolean confirmCheckout(RentalRequest request, RentalManager service) Purpose: Ask before approving the selected queue request. Return: boolean. Project call names: service.calculateRentalCost, queueLabel, fit, money, confirmCheckoutAction
confirmCheckoutCancellation TerminalUI.java:2433-2438	boolean confirmCheckoutCancellation(RentalRequest request) Purpose: Ask before cancelling the selected queue request. Return: boolean. Project call names: queueLabel, fit, confirmCheckoutAction
confirmCheckoutBatch TerminalUI.java:2441-2443	boolean confirmCheckoutBatch(int count) Purpose: Ask before approving the displayed ready batch, with paging when needed. Return: boolean. Project call names: confirmCheckoutAction
compactTransactionRows TerminalUI.java:2447-2449	private boolean compactTransactionRows() Purpose: Choose compact record display for short or narrow terminals. Return: boolean. Project call names: No other project method call name identified.
labelledRecord TerminalUI.java:2452-2463	private String labelledRecord(String[] headings, String[] cells) Purpose: Join field names and values for readable compact records. Return: String. Project call names: isEmpty
printTransactionRecord TerminalUI.java:2466-2476	private void printTransactionRecord(String format, String[] headings, String[] cells, int[] widths, int width) Purpose: Print a complete transaction in table or compact form. Return: void. Project call names: compactTransactionRows, wrapText, labelledRecord, printLine, printRecord
transactionPages TerminalUI.java:2479-2482	private ArrayList<Integer> transactionPages(ArrayList<String[]> rows, String[] headings, int[] widths, int width, String[] summary) Purpose: Measure transaction rows and reserve room for details and navigation. Return: ArrayList<Integer>. Project call names: transactionPages
transactionPages TerminalUI.java:2485-2514	private ArrayList<Integer> transactionPages(ArrayList<String[]> rows, String[] headings, int[] widths, int width, String[] summary, String[] details) Purpose: Measure transaction rows and reserve room for details and navigation. Return: ArrayList<Integer>. Project call names: compactTransactionRows, wrappedLines, labelledRecord, recordHeight, pageStarts
showTransactionPage TerminalUI.java:2517-2520	private void showTransactionPage(String title, ArrayList<String[]> rows, String[] headings, String format, int[] widths, int width, String[] summary, int page, ArrayList<Integer> starts) Purpose: Display one transaction page with separate metadata and record rows. Return: void. Project call names: showTransactionPage

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
showTransactionPage TerminalUI.java:2523-2590	private void showTransactionPage(String title, ArrayList<String[]> rows, String[] headings, String format, int[] widths, int width, String[] summary, int page, ArrayList<Integer> starts, String[] details) Purpose: Display one transaction page with separate metadata and record rows. Return: void. Project call names: clearScreen, printLine, fit, printHeader, border, printWrappedBoxText, flushBorder, wrapText, compactTransactionRows, tableLine, printRecord, printTransactionRecord, center
showTransactionNavigation TerminalUI.java:2593-2608	private void showTransactionNavigation(int page, int pages, String nextKey, String[] destinations) Purpose: Display usable page keys and named return destinations. Return: void. Project call names: showChoiceGroup, inputBar
receiptRows TerminalUI.java:2611-2625	private ArrayList<String[]> receiptRows(Queue2Play.CheckoutSession session) Purpose: Copy session approvals and cancellations into receipt display rows. Return: ArrayList<String[]>. Project call names: rentalLabel, money, queueLabel
receiptSummary TerminalUI.java:2628-2633	private String[] receiptSummary(Queue2Play.CheckoutSession session, CheckoutQueue queue) Purpose: Build session totals and predicted queue waiting counts. Return: String[]. Project call names: queue.getRequests, queue.previewReadyRequests, money
receiptPages TerminalUI.java:2636-2639	ArrayList<Integer> receiptPages(Queue2Play.CheckoutSession session, CheckoutQueue queue) Purpose: Measure receipt rows to keep the prompt and navigation visible. Return: ArrayList<Integer>. Project call names: transactionPages, receiptRows, receiptSummary, receiptDetails
receiptDetails TerminalUI.java:2642-2645	private String[] receiptDetails(Queue2Play.CheckoutSession session) Purpose: Build separate administrator and timestamp details for the receipt. Return: String[]. Project call names: No other project method call name identified.
showCheckoutReceiptPage TerminalUI.java:2648-2654	void showCheckoutReceiptPage(Queue2Play.CheckoutSession session, CheckoutQueue queue, int page, ArrayList<Integer> starts) Purpose: Display one receipt page without changing checkout state. Return: void. Project call names: showTransactionPage, receiptRows, receiptSummary, receiptDetails
waitForReceiptNavigation TerminalUI.java:2657-2687	boolean waitForReceiptNavigation(Queue2Play.CheckoutSession session, CheckoutQueue queue, String previousDestination) Purpose: Keep the receipt visible while handling page and destination choices. Return: boolean. Project call names: receiptPages, waitForResultNavigation, showTransactionNavigation, readText, prompt, clearResultRetry, showCheckoutReceiptPage
batchRows TerminalUI.java:2690-2707	private ArrayList<String[]> batchRows(ArrayList<RentalRequest> planned, CheckoutQueue queue, RentalManager service) Purpose: Build predicted batch rows from copied display data. Return: ArrayList<String[]>. Project call names: queueLabel, money, service.calculateRentalCost
batchSummary TerminalUI.java:2710-2717	private String[] batchSummary(ArrayList<RentalRequest> planned, CheckoutQueue queue, RentalManager service) Purpose: Calculate the displayed batch counts and total cost. Return: String[]. Project call names: service.calculateRentalCost, money, queue.getRequests
batchPages TerminalUI.java:2720-2724	ArrayList<Integer> batchPages(ArrayList<RentalRequest> planned, CheckoutQueue queue, RentalManager service) Purpose: Measure predicted batch records before building preview pages. Return: ArrayList<Integer>. Project call names: transactionPages, batchRows, batchSummary

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
showCheckoutBatchPage TerminalUI.java:2727-2733	void showCheckoutBatchPage(ArrayList<RentalRequest> planned, CheckoutQueue queue, RentalManager service, int page, ArrayList<Integer> starts) Purpose: Display one batch-preview page without processing requests. Return: void. Project call names: showTransactionPage, batchRows, batchSummary
confirmCheckoutBatch TerminalUI.java:2736-2762	boolean confirmCheckoutBatch(ArrayList<RentalRequest> planned, CheckoutQueue queue, RentalManager service) Purpose: Ask before approving the displayed ready batch, with paging when needed. Return: boolean. Project call names: batchPages, confirmCheckoutBatch, showTransactionNavigation, readText, prompt, clearResultRetry, showCheckoutBatchPage
showCheckoutBatch TerminalUI.java:2765-2795	void showCheckoutBatch(ArrayList<RentalRequest> planned, CheckoutQueue queue, RentalManager service) Purpose: Display the ready batch preview without approving it. Return: void. Project call names: batchPages, showCheckoutBatchPage, beginCheckoutCard, printTitle, printLine, border, service.calculateRentalCost, checkoutRow, queueLabel, fit, money, queue.getRequests, endCheckoutCard
showCheckoutReceipt TerminalUI.java:2798-2800	void showCheckoutReceipt(Queue2Play.CheckoutSession session, CheckoutQueue queue) Purpose: Display the first page of the checkout session receipt. Return: void. Project call names: showCheckoutReceiptPage, receiptPages
endCheckoutDisplay TerminalUI.java:2803-2807	void endCheckoutDisplay() Purpose: Reset checkout-only layout and page settings. Return: void. Project call names: No other project method call name identified.
showQueueCancellationOptions TerminalUI.java:2810-2815	void showQueueCancellationOptions() Purpose: Display customer cancellation and navigation choices. Return: void. Project call names: printLine, border, showFeatureOptions, showNavigationOptions, inputBar
showVipBenefits TerminalUI.java:2818-2827	void showVipBenefits() Purpose: Display the free membership, discount and queue-priority description. Return: void. Project call names: printCenteredDetails, membershipColor, showFeatureOptions, showNavigationOptions
showNextCheckout TerminalUI.java:2830-2841	void showNextCheckout(double rentalCost, RentalRequest next) Purpose: Display the next request's customer, game and rental cost. Return: void. Project call names: printCenteredDetails, membershipColor, platformColor, queueLabel, money
showReturnSelectionList TerminalUI.java:2844-2870	void showReturnSelectionList(ArrayList<Rental> rentals, int start, int end, int page, int pages, String keyword) Purpose: Display due-date-ordered return choices with search and page controls. Return: void. Project call names: printHeader, showListStatus, showEmptyState, showReturnTableHeader, showReturnRow, printLine, tableLine, showNavigationOptions, showPagination, inputBar
showReturnSearchPrompt TerminalUI.java:2873-2877	void showReturnSearchPrompt() Purpose: Display the input field for finding an active rental. Return: void. Project call names: showNavigationOptions, inputBar
showSelectedReturnSummary TerminalUI.java:2880-2882	void showSelectedReturnSummary(ArrayList<Rental> selected, LocalDateTime returnedAt, RentalManager service) Purpose: Display the first confirmation page for selected returns. Return: void. Project call names: showReturnConfirmationPage, returnConfirmationPages

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
returnConfirmationRows TerminalUI.java:2885-2894	private ArrayList<String[]> returnConfirmationRows(ArrayList<Rental> selected, LocalDateTime returnedAt, RentalManager service) Purpose: Build selected-return rows with due dates and calculated late fees. Return: ArrayList<String[]>. Project call names: RentalManager.daysOverdue, rentalLabel, money, service.calculateLateFee
returnConfirmationSummary TerminalUI.java:2897-2904	private String[] returnConfirmationSummary(ArrayList<Rental> selected, LocalDateTime returnedAt, RentalManager service) Purpose: Calculate counts and late fees for the full return selection. Return: String[]. Project call names: RentalManager.daysOverdue, service.calculateLateFee, money
returnConfirmationPages TerminalUI.java:2907-2912	ArrayList<Integer> returnConfirmationPages(ArrayList<Rental> selected, LocalDateTime returnedAt, RentalManager service) Purpose: Measure return-confirmation rows before building pages. Return: ArrayList<Integer>. Project call names: transactionPages, returnConfirmationRows, returnConfirmationSummary
showReturnConfirmationPage TerminalUI.java:2915-2921	void showReturnConfirmationPage(ArrayList<Rental> selected, LocalDateTime returnedAt, RentalManager service, int page, ArrayList<Integer> starts) Purpose: Display one confirmation page for the selected return batch. Return: void. Project call names: showTransactionPage, returnConfirmationRows, returnConfirmationSummary
confirmSelectedReturns TerminalUI.java:2924-2948	boolean confirmSelectedReturns(ArrayList<Rental> selected, LocalDateTime returnedAt, RentalManager service) Purpose: Page the preview and read Yes/No for the entire selection. Return: boolean. Project call names: showSelectedReturnSummary, returnConfirmationPages, showTransactionNavigation, readText, prompt, clearResultRetry, showReturnConfirmationPage
showSelectedReturnResults TerminalUI.java:2951-2953	void showSelectedReturnResults(ArrayList<Rental> selected) Purpose: Display the first page of saved return results. Return: void. Project call names: showReturnResultPage, returnResultPages
returnResultRows TerminalUI.java:2956-2964	private ArrayList<String[]> returnResultRows(ArrayList<Rental> selected) Purpose: Build result rows with recorded late fees and restored stock. Return: ArrayList<String[]>. Project call names: rentalLabel, money
returnResultSummary TerminalUI.java:2967-2973	private String[] returnResultSummary(ArrayList<Rental> selected) Purpose: Calculate result totals for all processed returns. Return: String[]. Project call names: money
returnResultPages TerminalUI.java:2976-2980	ArrayList<Integer> returnResultPages(ArrayList<Rental> selected) Purpose: Measure processed-return rows before building result pages. Return: ArrayList<Integer>. Project call names: transactionPages, returnResultRows, returnResultSummary
showReturnResultPage TerminalUI.java:2983-2988	void showReturnResultPage(ArrayList<Rental> selected, int page, ArrayList<Integer> starts) Purpose: Display one page of processed returns without processing them again. Return: void. Project call names: showTransactionPage, returnResultRows, returnResultSummary
waitForReturnResultNavigation TerminalUI.java:2991-3021	boolean waitForReturnResultNavigation(ArrayList<Rental> selected, String previousDestination) Purpose: Keep return results visible while paging or choosing a destination. Return: boolean. Project call names: returnResultPages, waitForResultNavigation, showTransactionNavigation, readText, prompt, clearResultRetry, showReturnResultPage

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
showReturnTableHeader TerminalUI.java:3024-3028	void showReturnTableHeader(int total, int start, int end) Purpose: Display the return selection's numbered column headings. Return: void. Project call names: printLine, tableLine, printRecord
showReturnRow TerminalUI.java:3031-3034	void showReturnRow(int shown, Rental rental) Purpose: Display one active rental with its current-page selection number. Return: void. Project call names: printRecord, rentalLabel
showActiveRentalHeader TerminalUI.java:3037-3041	void showActiveRentalHeader(int total, int start, int end) Purpose: Display the active-rental column headings. Return: void. Project call names: printLine, tableLine, printRecord
showActiveRentalRow TerminalUI.java:3044-3047	void showActiveRentalRow(Rental rental) Purpose: Display one active rental without changing its heap entry. Return: void. Project call names: printRecord, rentalLabel
showAuditHeader TerminalUI.java:3050-3054	void showAuditHeader(int total, int start, int end) Purpose: Display the audit log's date, time and action headings. Return: void. Project call names: printLine, tableLine, printRecord
showAuditRow TerminalUI.java:3057-3062	void showAuditRow(LocalDateTime stamp, String[] parts) Purpose: Display the parsed date, time and action of an audit record. Return: void. Project call names: printRecord, printLine
showLegacyAuditRow TerminalUI.java:3065-3068	void showLegacyAuditRow(String entry) Purpose: Display an older audit record without losing its text. Return: void. Project call names: printRecord, printLine
showAddGameInstructions TerminalUI.java:3071-3073	void showAddGameInstructions() Purpose: Display useful guidance for entering a game title. Return: void. Project call names: showNavigationOptions
showPlatformOptions TerminalUI.java:3076-3083	void showPlatformOptions() Purpose: Display supported platforms for adding or editing a game. Return: void. Project call names: showFeatureOptions, platformColor, showNavigationOptions
showPriceInstructions TerminalUI.java:3086-3089	void showPriceInstructions() Purpose: Display the allowed rental price range. Return: void. Project call names: showNavigationOptions, inputBar
showStockInstructions TerminalUI.java:3092-3095	void showStockInstructions() Purpose: Display the allowed starting-stock range. Return: void. Project call names: showNavigationOptions, inputBar
showNewGameConfirmation TerminalUI.java:3098-3102	void showNewGameConfirmation(String title, String platform, double price, int copies) Purpose: Display draft game details and confirmation choices. Return: void. Project call names: showNewGameDetails, showFeatureOptions, showNavigationOptions
showNewGameDetails TerminalUI.java:3105-3112	private void showNewGameDetails(String title, String platform, double price, int copies) Purpose: Display the current draft's title, platform, price and stock. Return: void. Project call names: printCenteredDetails, platformColor, money

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
showGameEditMenu TerminalUI.java:3115-3121	void showGameEditMenu(String title, String platform, double price, int copies) Purpose: Display independent draft fields that can be edited. Return: void. Project call names: printHeader, showNewGameDetails, showMenuOptions, showNavigationOptions, inputBar
showGameEditField TerminalUI.java:3124-3133	void showGameEditField(int step, String title, String platform, double price, int copies) Purpose: Display one editable game field while preserving other details. Return: void. Project call names: printHeader, showNewGameDetails, showFeatureOptions, showNavigationOptions, inputBar
showGameAdded TerminalUI.java:3136-3144	void showGameAdded(Game game) Purpose: Display the saved game's identifier and details. Return: void. Project call names: printCenteredDetails, platformColor, money
showDeleteGameDetails TerminalUI.java:3147-3155	void showDeleteGameDetails(Game game, int pending) Purpose: Display the game and affected request count before deletion. Return: void. Project call names: printCenteredDetails, platformColor
showStockUpdateOptions TerminalUI.java:3158-3167	void showStockUpdateOptions(Game game) Purpose: Display stock details and increase or decrease choices. Return: void. Project call names: printCenteredDetails, platformColor, showFeatureOptions, showNavigationOptions
showStockQuantityPrompt TerminalUI.java:3170-3177	void showStockQuantityPrompt(boolean adding) Purpose: Display quantity input for the chosen stock adjustment. Return: void. Project call names: showNavigationOptions, inputBar
showStockConfirmation TerminalUI.java:3180-3186	void showStockConfirmation(Game game, int previous, int newStock) Purpose: Display old and proposed stock before confirmation. Return: void. Project call names: printCenteredDetails
showStockUpdated TerminalUI.java:3189-3195	void showStockUpdated(Game game, int previous) Purpose: Display the saved stock adjustment result. Return: void. Project call names: printCenteredDetails
showFullNameInstructions TerminalUI.java:3198-3200	void showFullNameInstructions() Purpose: Display the allowed full-name length and characters. Return: void. Project call names: error
showRegistrationConfirmation TerminalUI.java:3203-3217	void showRegistrationConfirmation(String fullName, String username, String email) Purpose: Display the account draft before creating it. Return: void. Project call names: printLine, printCenteredDetails, showFeatureOptions, showNavigationOptions
showAccountCreated TerminalUI.java:3220-3230	void showAccountCreated() Purpose: Display the successful registration result and next destination. Return: void. Project call names: printLine

StorageException.java - 2 declarations

Runtime exception carrying a storage message and optional cause.

METHOD / SOURCE LINES	DECLARATION, PURPOSE, RETURN AND CONNECTIONS
StorageException StorageException.java:5-7	StorageException(String message, Throwable cause) Purpose: Carry a file error and, when supplied, its original cause. Return: constructor. Project call names: No other project method call name identified.
StorageException StorageException.java:9-11	StorageException(String message) Purpose: Carry a file error and, when supplied, its original cause. Return: constructor. Project call names: No other project method call name identified.

InputEndedException.java - 0 declarations

Signals that terminal input ended without submitting the pending input.

No explicit methods or constructors are declared in this file. The class inherits RuntimeException behavior.

Appendix B. Complete validation and diagnostic reference

This inventory contains 248 explicit diagnostic/constraint sites found in the application source: require(), UI errors/warnings/empty states, explicit thrown exceptions, returned business-validation errors and direct retry/message text. Repeated messages at different branches remain separate because their source conditions differ. Java runtime exception details such as a NumberFormatException value or filesystem path are generated by Java/the OS; they are not invented or fixed in this report.

Exact message expression means literal Java text between quotes, or its original concatenated expression. At runtime the quotes are not printed, concatenated identifiers are evaluated, and Java escape sequences are interpreted. UI errors/warnings receive the renderer prefix. Storage errors can be wrapped as shown in the relevant wrapper entries. A forwarded variable such as error is resolved by the named validator in Chapter 9, not replaced with an invented message.

For require(), the condition column is the exact condition that must be true. Other entries include the actual nearby source context rather than claiming a guessed complete guard. Read the referenced method for nested branches. Silent boolean validators and input/navigation guards are supplemented by the full predicate source and method catalog below; some false results are translated to a UI error by their caller.

AccountManager.java - 18 diagnostic/constraint sites

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
loadAccounts() AccountManager.java:111 Constraint (fails when false)	"Expected six account fields"	Must be true: fields.length == 6
loadAccounts() AccountManager.java:112 Constraint (fails when false)	"Invalid account name or login identifier"	Must be true: validUsername(fields[0]) && validName(fields[1]) && validEmail(fields[2])
loadAccounts() AccountManager.java:113 Constraint (fails when false)	"Invalid role"	Must be true: fields[4].equals("ADMIN") || fields[4].equals("CLIENT")
loadAccounts() AccountManager.java:114 Constraint (fails when false)	"Invalid membership"	Must be true: fields[5].equals("REGULAR") || fields[5].equals("VIP")
loadAccounts() AccountManager.java:115 Constraint (fails when false)	"Invalid stored password"	Must be true: fields[3].length() >= 8 && ! GameCatalog.containsControlCharacter(fields[3])
loadAccounts() AccountManager.java:116 Constraint (fails when false)	"Duplicate login identifier"	Must be true: findUserByLogin(fields[0]) == null && findUserByLogin(fields[2]) == null && ! fields[0].equalsIgnoreCase(fields[2])
ensureEmbeddedAccount() AccountManager.java:133 Constraint (fails when false)	"Embedded account email is already used: " +email	Must be true: emailOwner == null || emailOwner == account
register() AccountManager.java:231 UI error	"Use 1 to create, E to edit, or C to cancel."	Source context: } else if (choice.equalsIgnoreCase("C")) { if (cancelRegistration()) { return; } } else { ui.error("Use 1 to create, E to edit, or C to cancel.");
register() AccountManager.java:245 UI error	"Choose 1, 2, 3, 4, or B to return to the summary."	Source context: } else if (choice.equals("1") || choice.equals("2") || choice.equals("3") || choice.equals("4")) { step = Integer.parseInt(choice); editing = true; } else { ui.error("Choose 1, 2, 3, 4, or B to return to the summary.");
register() AccountManager.java:288 UI error	"Use 3-20 letters, digits, dots, underscores or hyphens, with at least one letter."	Source context: continue; } fullName = value; } else if (step == 2) { if (!validRegistrationUsername(value)) { ui.error("Use 3-20 letters, digits, dots, underscores or hyphens, with at least one letter.");
register() AccountManager.java:292 UI error	"Username is already taken. Please try another one."	Source context: if (!validRegistrationUsername(value)) { ui.error("Use 3-20 letters, digits, dots, underscores or hyphens, with at least one letter."); continue; } if (findUserByLogin(value) != null) { ui.error("Username is already taken. Please try another one.");
register() AccountManager.java:298 UI error	"Enter a valid email of at most 100 characters. Example: juan@gmail.com"	Source context: continue; } username = value; } else if (step == 3) { if (!validRegistrationEmail(value)) { ui.error("Enter a valid email of at most 100 characters. Example: juan@gmail.com");
register() AccountManager.java:302 UI error	"Email is already registered. Please use another email."	Source context: if (!validRegistrationEmail(value)) { ui.error("Enter a valid email of at most 100 characters. Example: juan@gmail.com"); continue; } if (emailExists(value)) { ui.error("Email is already registered. Please use another email.");

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
register() AccountManager.java:308 UI error	"Use 8-20 characters without whitespace, control characters or |."	Source context: continue; } email = value; } else if (step == 4) { if (!validRegistrationPassword(value)) { ui.error("Use 8-20 characters without whitespace, control characters or |.");
register() AccountManager.java:317 UI error	"Passwords do not match. Please try again."	Source context: pendingPassword = value; step = 5; continue; } else { if (!value.equals(pendingPassword)) { ui.error("Passwords do not match. Please try again.");
failedLogin() AccountManager.java:342 UI error	"Incorrect username/email or password."	Source context: { failedLoginAttempts++; ui.error("Incorrect username/email or password.");
login() AccountManager.java:362 UI error	"Username or email cannot be empty."	Source context: String login = ui.readText(ui.prompt()).trim() ; if (login.equalsIgnoreCase("B") || login.equals("0")) { return null; } if (login.isEmpty()) { ui.error("Username or email cannot be empty.");
login() AccountManager.java:378 UI error	"Password cannot be empty."	Source context: } if (password.equalsIgnoreCase("B")) { break; } if (password.isEmpty()) { ui.error("Password cannot be empty.");

AppStorage.java - 9 diagnostic/constraint sites

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
AppStorage() AppStorage.java:34 Exception	"Another QUEUE2PLAY process is using this data folder."	Source context: directory = dataDirectory.toAbsolutePath().normalize(); journal = directory.resolve(".transaction"); Files.createDirectories(directory); try (FileChannel channel = lockChannel(); FileLock lock = channel.tryLock()) { if (lock == null) { throw new IOException("Another QUEUE2PLAY process is using this data folder.");
AppStorage() AppStorage.java:49 Exception	"Cannot open the data folder: " + e.getMessage()	Source context: } } rememberFiles(); } } catch (Exception e) { throw new StorageException("Cannot open the data folder: " + e.getMessage(), e);
rememberFiles() AppStorage.java:79 Exception	name + " must be a regular file."	Source context: { original.clear(); for (String name : FILES) { Path target = file(name); if (Files.exists(target) && !Files.isRegularFile(target)) { throw new IOException(name + " must be a regular file.");
commit() AppStorage.java:149 Exception	"Another process is saving these files."	Source context: { boolean startedJournal = false; try (FileChannel channel = lockChannel(); FileLock lock = channel.tryLock()) { if (lock == null) { throw new IOException("Another process is saving these files.");
commit() AppStorage.java:155 Exception	name + " must be a regular file."	Source context: } recover(); for (String name : FILES) { Path target = file(name); if (Files.exists(target) && !Files.isRegularFile(target)) { throw new IOException(name + " must be a regular file.");
commit() AppStorage.java:162 Exception	name + " changed outside this session. Restart to reload it."	Source context: byte[] current = null; if (Files.exists(target)) { current = Files.readAllBytes(target); } if (!Arrays.equals(current, original.get(name))) { throw new IOException(name + " changed outside this session. Restart to reload it.");
commit() AppStorage.java:181 Exception	"Injected save failure."	Source context: int replaced = 0; for (String name : FILES) { replace(journal.resolve(name + ".new"), file(name)); replaced++; if (replaced == Integer.getInteger("q2p.test.failAfterReplace", -1)) { throw new IOException("Injected save failure.");
commit() AppStorage.java:198 Exception	"Recovery needs the data lock. Restart before making more changes."	Source context: } } catch (Exception e) { if (startedJournal) { try (FileChannel channel = lockChannel(); FileLock lock = channel.tryLock()) { if (lock == null) { throw new IOException("Recovery needs the data lock. Restart before making more changes.");
commit() AppStorage.java:205 Exception	"Changes were not saved. " + e.getMessage()	Source context: recover(); } catch (IOException recovery) { e.addSuppressed(recovery); } } throw new StorageException("Changes were not saved. " + e.getMessage(), e);

CheckoutQueue.java - 1 diagnostic/constraint sites

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
enqueue() CheckoutQueue.java:81 Exception	"Request cannot be null."	Source context: { if (request == null) { throw new IllegalArgumentException("Request cannot be null.");

GameCatalog.java - 8 diagnostic/constraint sites

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
validate() GameCatalog.java:46 Constraint (fails when false)	"Invalid game title"	Must be true: validTitle(title)
validate() GameCatalog.java:47 Constraint (fails when false)	"Invalid platform"	Must be true: platform.equals("PS5") || platform.equals("Xbox") || platform.equals("Nintendo")
validate() GameCatalog.java:48 Constraint (fails when false)	"Invalid rental price"	Must be true: Double.isFinite(price) && price >= Queue2Play.MIN_RENTAL_PRICE && price <= Queue2Play.MAX_RENTAL_PRICE
validate() GameCatalog.java:50 Constraint (fails when false)	"Invalid stock"	Must be true: stock >= 0 && stock <= Queue2Play.MAX_GAME_STOCK
generateGameId() GameCatalog.java:65 Constraint (fails when false)	"Invalid platform"	Must be true: platform.equals("PS5") || platform.equals("Xbox") || platform.equals("Nintendo")
generateGameId() GameCatalog.java:66 Constraint (fails when false)	"Game ID range exhausted"	Must be true: ps5Count < Integer.MAX_VALUE - 1 && xboxCount < Integer.MAX_VALUE - 1 && nintendoCount < Integer.MAX_VALUE - 1
addGame() GameCatalog.java:84 Constraint (fails when false)	"This game already exists on this platform"	Must be true: !existing.title.equalsIgnoreCase(title) || !existing.platform.equals(platform)
insertionSort() GameCatalog.java:142 Constraint (fails when false)	"Invalid sort option"	Must be true: option >= 1 && option <= 4

PasswordSecurity.java - 4 diagnostic/constraint sites

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
hashPlaintext() PasswordSecurity.java:31 Exception	"Password is required."	Source context: { if (password == null) { throw new IllegalArgumentException("Password is required.");
validateHash() PasswordSecurity.java:43 Exception	"Invalid password hash."	Source context: { String[] p = encoded.split("\\$", -1); if (p.length != 4 || !p[0].equals("pbkdf2")) { throw new IllegalArgumentException("Invalid password hash.");
validateHash() PasswordSecurity.java:48 Exception	"Invalid password hash."	Source context: throw new IllegalArgumentException("Invalid password hash."); } int iterations = Integer.parseInt(p[1]); if (iterations < 100000 || iterations > 1000000 || Base64.getDecoder().decode(p[2]).length != 16 || Base64.getDecoder().decode(p[3]).length != 32) { throw new IllegalArgumentException("Invalid password hash.");
derive() PasswordSecurity.java:73 Exception	"Password hashing unavailable."	Source context: { PBEKeySpec spec = new PBEKeySpec(password.toCharArray(), salt, iterations, 256); try { return SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(spec).getEncoded(); } catch (java.security.GeneralSecurityException e) { throw new IllegalStateException("Password hashing unavailable.", e);

Queue2Play.java - 94 diagnostic/constraint sites

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
waitForScreenNavigation() Queue2Play.java:114 UI error	"Use the displayed navigation key."	Source context: } if (allowDashboard && choice.equalsIgnoreCase("M")) { requestDashboardReturn(); return; } ui.error("Use the displayed navigation key.");
start() Queue2Play.java:181 UI error	"Invalid choice. Use 1 to log in, 2 to register, or Q to exit."	Source context: if (ui.askYesNo("Exit QUEUE2PLAY?")) { ui.showGoodbye(); running = false; } } else { ui.error("Invalid choice. Use 1 to log in, 2 to register, or Q to exit.");
start() Queue2Play.java:184 UI error	e.getMessage()	Source context: } } else { ui.error("Invalid choice. Use 1 to log in, 2 to register, or Q to exit."); } } catch (StorageException e) { ui.error(e.getMessage());
paginateGames() Queue2Play.java:239 Empty-state information	"No games found. Use S to search again or X to clear search."	Source context: ui.showCatalogState(keyword, sortOption, page + 1, totalPages, cart); } else { ui.showCatalogState(keyword, sortOption, page + 1, totalPages); } if (list.isEmpty()) { ui.showEmptyState("No games found. Use S to search again or X to clear search.");
paginateGames() Queue2Play.java:266 Warning	"You are on the last page."	Source context: return null; } else if (choice.equalsIgnoreCase("N")) { if (page < totalPages - 1) { page++; } else { ui.warning("You are on the last page.");
paginateGames() Queue2Play.java:272 Warning	"You are on the first page."	Source context: } } else if (choice.equalsIgnoreCase("P")) { if (page > 0) { page--; } else { ui.warning("You are on the first page.");
paginateGames() Queue2Play.java:300 UI error	"Search text cannot be empty."	Source context: } if (searchText.equalsIgnoreCase("B") || searchText.equals("0")) { break; } if (searchText.isEmpty()) { ui.error("Search text cannot be empty.");
paginateGames() Queue2Play.java:317 UI error	error	Source context: if (cart == null) { return game; } String error = validateRentalSelection(client, cart, game); if (error != null) { ui.error(error);
paginateGames() Queue2Play.java:326 UI error	"Invalid game selection."	Source context: if (selectedQuantity[0] == 0) { continue; } return game; } ui.error("Invalid game selection.");
paginateGames() Queue2Play.java:328 UI error	"Invalid choice."	Source context: } return game; } ui.error("Invalid game selection."); } catch (NumberFormatException e) { ui.error("Invalid choice.");
paginateGames() Queue2Play.java:331 UI error	"Use S, X, T, N, P, B, or M."	Source context: ui.error("Invalid game selection."); } catch (NumberFormatException e) { ui.error("Invalid choice."); } } else { ui.error("Use S, X, T, N, P, B, or M.");
choosePlatform() Queue2Play.java:390 UI error	"Invalid choice. Please enter 1, 2, 3, 4, B, or M."	Source context: return "BACK"; } } catch (NumberFormatException e) { // handled below } ui.error("Invalid choice. Please enter 1, 2, 3, 4, B, or M.");
clientMenu() Queue2Play.java:433 UI error	"Invalid choice. Choose a displayed option."	Source context: ui.success("You have been logged out."); ui.pause(500); loggedIn = false; } } else { ui.error("Invalid choice. Choose a displayed option.");
clientMenu() Queue2Play.java:436 UI error	e.getMessage()	Source context: } } else { ui.error("Invalid choice. Choose a displayed option."); } } catch (StorageException e) { ui.error(e.getMessage());

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
browseGames() Queue2Play.java:461 UI error	"Invalid choice."	Source context: int choice; try { choice = parseMenuChoice(input); } catch (NumberFormatException e) { ui.error("Invalid choice.");
browseGames() Queue2Play.java:476 UI error	"Invalid choice. Choose a displayed option."	Source context: } else if (choice == 4) { viewGames("Nintendo"); } else if (choice == 0) { return; } else { ui.error("Invalid choice. Choose a displayed option.");
validateRentalSelection() Queue2Play.java:526 Returned validation error	"You are already renting this game."	Source context: { int inCart = cart.getOrDefault(game, 0); int queued = checkoutQueue.queuedCopies(client, game); if (rentalService.hasActiveRental(client, game)) { return " You are already renting this game.";
validateRentalSelection() Queue2Play.java:529 Returned validation error	"You already have the maximum of " + MAX_COPIES_PER_GAME + " copies of this game queued or in your cart."	Source context: int queued = checkoutQueue.queuedCopies(client, game); if (rentalService.hasActiveRental(client, game)) { return "You are already renting this game."; } if (queued + inCart >= MAX_COPIES_PER_GAME) { return "You already have the maximum of "
validateRentalSelection() Queue2Play.java:533 Returned validation error	"This game is out of stock."	Source context: if (queued + inCart >= MAX_COPIES_PER_GAME) { return "You already have the maximum of " + MAX_COPIES_PER_GAME + " copies of this game queued or in your cart."; } if (game.availableCopies <= 0) { return "This game is out of stock.";
validateRentalSelection() Queue2Play.java:536 Returned validation error	"Your cart already contains all available copies of this game."	Source context: } if (game.availableCopies <= 0) { return " This game is out of stock."; } if (inCart >= game.availableCopies) { return "Your cart already contains all available copies of this game.";
askQuantity() Queue2Play.java:553 UI error	"No more copies of this game are available to add."	Source context: int maximumToAdd = MAX_COPIES_PER_GAME - queued - inCart; if (game.availableCopies - inCart < maximumToAdd) { maximumToAdd = game.availableCopies - inCart; } if (maximumToAdd <= 0) { ui.error("No more copies of this game are available to add.");
askQuantity() Queue2Play.java:581 UI error	"Quantity must be at least 1."	Source context: } try { int quantity = Integer.parseInt(input); if (quantity < 1) { ui.error("Quantity must be at least 1.");
askQuantity() Queue2Play.java:584 UI error	"Maximum rental quantity is " + MAX_COPIES_PER_GAME + " copies per game. You have " + queued + " queued and " + inCart + " in your cart."	Source context: int quantity = Integer.parseInt(input); if (quantity < 1) { ui.error("Quantity must be at least 1."); } else if (quantity > maximumToAdd) { if (quantity > MAX_COPIES_PER_GAME - queued - inCart) { ui.error("Maximum rental quantity is " + MAX_COPIES_PER_GAME
askQuantity() Queue2Play.java:587 UI error	"Not enough stock. You may add up to " + maximumToAdd + " more copies."	Source context: } else if (quantity > maximumToAdd) { if (quantity > MAX_COPIES_PER_GAME - queued - inCart) { ui.error("Maximum rental quantity is " + MAX_COPIES_PER_GAME + " copies per game. You have " + queued + " queued and " + inCart + " in your cart."); } else { ui.error("Not enough stock. You may add up to " + maximumToAdd + " more copies.");
askQuantity() Queue2Play.java:593 UI error	"Enter a whole number, B, or M."	Source context: } } else { return quantity; } } catch (NumberFormatException e) { ui.error("Enter a whole number, B, or M.");

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
chooseRentalDays() Queue2Play.java:668 UI error	"Invalid choice."	Source context: int choice; try { choice = parseMenuChoice(input); } catch (NumberFormatException e) { ui.error("Invalid choice.");
chooseRentalDays() Queue2Play.java:708 UI error	"Rental duration must be between 1 and " + MAX_RENTAL_DAYS + " days."	Source context: break; } try { int days = Integer.parseInt(daysInput); if (days < 1 || days > MAX_RENTAL_DAYS) { ui.error("Rental duration must be between 1 and " + MAX_RENTAL_DAYS + " days.");
chooseRentalDays() Queue2Play.java:713 UI error	"Enter a whole number from 1 to " + MAX_RENTAL_DAYS + "."	Source context: ui.error("Rental duration must be between 1 and " + MAX_RENTAL_DAYS + " days."); } else { return days; } } catch (NumberFormatException e) { ui.error("Enter a whole number from 1 to " + MAX_RENTAL_DAYS + ".");
chooseRentalDays() Queue2Play.java:717 UI error	"Invalid choice. Choose a displayed option."	Source context: } catch (NumberFormatException e) { ui.error("Enter a whole number from 1 to " + MAX_RENTAL_DAYS + "."); } } } else { ui.error("Invalid choice. Choose a displayed option.");
editOrder() Queue2Play.java:764 UI error	"Invalid choice."	Source context: int choice; try { choice = parseMenuChoice(input); } catch (NumberFormatException e) { ui.error("Invalid choice.");
editOrder() Queue2Play.java:801 UI error	"Invalid line number."	Source context: if (number >= 1 && number <= cart.size()) { Game removed = new ArrayList<>(cart.keySet()).get(number - 1); cart.remove(removed); ui.success(removed.title + " removed from your cart."); } else if (number != 0) { ui.error("Invalid line number.");
editOrder() Queue2Play.java:804 UI error	"Invalid line number."	Source context: ui.success(removed.title + " removed from your cart."); } else if (number != 0) { ui.error("Invalid line number."); } } catch (NumberFormatException e) { ui.error("Invalid line number.");
editOrder() Queue2Play.java:817 UI error	"Invalid choice."	Source context: days = newDays; } } else if (choice == 0) { return days; } else { ui.error("Invalid choice.");
validateCartBeforeSubmission() Queue2Play.java:826 UI error	error	Source context: { String error = rentalService.validateCart(client, cart); if (error != null) { ui.error(error);
placeInQueue() Queue2Play.java:882 Warning	"Please edit the order before submitting."	Source context: page--; } else if ((choice.equalsIgnoreCase("B") || choice.equals("0"))) { step = 2; } else if (choice.equals("1")) { if (!validateCartBeforeSubmission(client, cart)) { ui.warning("Please edit the order before submitting.");
placeInQueue() Queue2Play.java:907 UI error	"Use 1 to confirm, E to edit, B for Rental Duration, or M for Dashboard."	Source context: if (ui.confirmDiscardCart()) { requestDashboardReturn(); return; } } else { ui.error("Use 1 to confirm, E to edit, B for Rental Duration, or M for Dashboard.");
placeInQueue() Queue2Play.java:912 Constraint (fails when false)	"Queue number range exhausted"	Must be true: (long) nextQueueNumber + cart.size() <= Integer.MAX_VALUE
placeInQueue() Queue2Play.java:916 Constraint (fails when false)	"Queue number range exhausted"	Must be true: nextQueueNumber < Integer.MAX_VALUE

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
cancelMyRequest() Queue2Play.java:947 Empty-state information	"You have no pending queue requests."	Source context: int page = 0; while (true) { ArrayList<RentalRequest> myRequests = checkoutQueue.getRequestsOf(client); if (myRequests.isEmpty()) { ui.printHeader("CLIENT > QUEUE REQUESTS", "MY QUEUE REQUESTS"); ui.showEmptyState("You have no pending queue requests.");
cancelMyRequest() Queue2Play.java:994 UI error	"Invalid choice. Use C to cancel a request or B for Dashboard."	Source context: } if ((choice.equalsIgnoreCase("B") || choice.equals("0"))) { return; } if (!choice.equalsIgnoreCase("C") && !choice.equals("1")) { ui.error("Invalid choice. Use C to cancel a request or B for Dashboard.");
cancelMyRequest() Queue2Play.java:1025 UI error	"Invalid queue number."	Source context: int number; try { number = Integer.parseInt(input); } catch (NumberFormatException e) { ui.error("Invalid queue number.");
cancelMyRequest() Queue2Play.java:1031 UI error	"Invalid queue number. You can only cancel your own requests."	Source context: continue; } RentalRequest request = checkoutQueue.findByNumber(number); if (request == null || request.client != client) { ui.error("Invalid queue number. You can only cancel your own requests.");
viewHistory() Queue2Play.java:1061 Empty-state information	"You have no rental history yet."	Source context: { if (client.rentalHistory.isEmpty()) { ui.printHeader("CLIENT > RENTAL HISTORY", "RENTAL HISTORY"); ui.showEmptyState("You have no rental history yet.");
viewHistory() Queue2Play.java:1101 UI error	"Invalid choice."	Source context: page--; } else if ((choice.equalsIgnoreCase("B") || choice.equals("0"))) { return; } else ui.error("Invalid choice.");
upgradeToVip() Queue2Play.java:1109 Empty-state information	"You are already a VIP member."	Source context: { if (client.isVip()) { ui.printHeader("CLIENT > MEMBERSHIP", "VIP MEMBERSHIP"); ui.showEmptyState("You are already a VIP member.");
upgradeToVip() Queue2Play.java:1134 UI error	"Invalid choice. Please enter 1 or C."	Source context: return; } else if (choice.equalsIgnoreCase("C")) { ui.info("VIP upgrade cancelled. No changes were made."); return; } else { ui.error("Invalid choice. Please enter 1 or C.");
adminMenu() Queue2Play.java:1168 UI error	"Invalid choice. Choose a displayed option."	Source context: ui.success("You have been logged out."); ui.pause(500); loggedIn = false; } } else { ui.error("Invalid choice. Choose a displayed option.");
adminMenu() Queue2Play.java:1171 UI error	e.getMessage()	Source context: } } else { ui.error("Invalid choice. Choose a displayed option."); } } catch (StorageException e) { ui.error(e.getMessage());
activeRentalsMenu() Queue2Play.java:1186 UI error	"Invalid choice. Please enter 1 or B."	Source context: if (input.equals("1")) { viewActiveRentals(); } else if ((input.equalsIgnoreCase("B") || input.equals("0"))) { return; } else { ui.error("Invalid choice. Please enter 1 or B.");
runCheckoutSession() Queue2Play.java:1288 Warning	"No request is ready."	Source context: } int entriesBeforeAction = session.entries.size(); if (choice.isEmpty()) { RentalRequest next = checkoutQueue.findNextEligibleRequest(); if (next == null) { ui.warning("No request is ready.");
runCheckoutSession() Queue2Play.java:1305 UI error	"Use ENTER, A, C, V, R, B or M."	Source context: ui.waitForReceiptNavigation(session, checkoutQueue, ""); } return; } } else { ui.error("Use ENTER, A, C, V, R, B or M.");

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
approveReadyBatch() Queue2Play.java:1324 Warning	"No request is ready."	Source context: { ArrayList<RentalRequest> planned = previewReadyCheckouts(); if (planned.isEmpty()) { ui.warning("No request is ready.");
approveReadyBatch() Queue2Play.java:1333 UI error	"Batch stopped at " + ui.queueLabel(request.queueNumber) + ": queue changed."	Source context: if (!ui.confirmCheckoutBatch(planned, checkoutQueue, rentalService)) { return; } for (RentalRequest request : planned) { if (checkoutQueue.findNextEligibleRequest() != request) { ui.error("Batch stopped at " + ui.queueLabel(request.queueNumber) + ": queue changed.");
approveSessionRequest() Queue2Play.java:1346 UI error	"Checkout " + ui.queueLabel(request.queueNumber) + " failed: " + error	Source context: { String error = rentalService.validateCheckout(request); if (error != null) { ui.error("Checkout " + ui.queueLabel(request.queueNumber) + " failed: " + error);
approveSessionRequest() Queue2Play.java:1358 UI error	"Checkout " + ui.queueLabel(request.queueNumber) + " failed: " + e.getMessage()	Source context: } session.addRental(rental); ui.showApprovedSlip(rental, session.entries.size()); return true; } catch (StorageException e) { ui.error("Checkout " + ui.queueLabel(request.queueNumber) + " failed: " + e.getMessage());
cancelSessionRequest() Queue2Play.java:1378 UI error	"Enter a valid queue number."	Source context: } RentalRequest request; try { request = checkoutQueue.findByNumber(Integer.parseInt(input)); } catch (NumberFormatException e) { ui.error("Enter a valid queue number.");
cancelSessionRequest() Queue2Play.java:1382 UI error	"Request not found."	Source context: } catch (NumberFormatException e) { ui.error("Enter a valid queue number."); return; } if (request == null) { ui.error("Request not found.");
cancelSessionRequest() Queue2Play.java:1393 UI error	"Cancellation " + ui.queueLabel(request.queueNumber) + " failed: " + e.getMessage()	Source context: checkoutQueue.remove(request); logAdmin("Cancelled queue request " + ui.queueLabel(request.queueNumber) + ": " + request.client.username); dataStore.saveQueue(); } catch (StorageException e) { ui.error("Cancellation " + ui.queueLabel(request.queueNumber) + " failed: " + e.getMessage());
recordAdminCancellation() Queue2Play.java:1403 Constraint (fails when false)	"History record numbers are exhausted"	Must be true: nextRentalNumber > 0 && nextRentalNumber < Integer.MAX_VALUE
approveCheckout() Queue2Play.java:1420 UI error	error	Source context: { String error = rentalService.validateCheckout(request); if (error != null) { ui.error(error);
parseReturnSelection() Queue2Play.java:1456 UI error	"Enter a row number or comma-separated numbers, such as 1,3,5."	Source context: for (String text : numbers) { int number; try { number = Integer.parseInt(text.trim()); } catch (NumberFormatException e) { ui.error("Enter a row number or comma-separated numbers, such as 1,3,5.");
parseReturnSelection() Queue2Play.java:1460 UI error	"Choose only row numbers displayed on this page."	Source context: } catch (NumberFormatException e) { ui.error("Enter a row number or comma-separated numbers, such as 1,3,5."); return null; } if (number < 1 || number > chosen.length) { ui.error("Choose only row numbers displayed on this page.");
parseReturnSelection() Queue2Play.java:1464 UI error	"Select each rental only once."	Source context: if (number < 1 || number > chosen.length) { ui.error("Choose only row numbers displayed on this page."); return null; } if (chosen[number - 1]) { ui.error("Select each rental only once.");

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
selectActiveRentalsForReturn() Queue2Play.java:1486 Empty-state information	"No active rentals."	Source context: int page = 0; String keyword = ""; while (true) { if (activeRentals.isEmpty()) { ui.printHeader("ADMIN > RETURNS", "ACTIVE RENTALS", TerminalUI.RETURN_TABLE_WIDTH); ui.showEmptyState("No active rentals.");
selectActiveRentalsForReturn() Queue2Play.java:1520 UI error	"There is no next page."	Source context: page = 0; } else if (choice.equalsIgnoreCase("N")) { if (page < totalPages - 1) { page++; } else { ui.error("There is no next page.");
selectActiveRentalsForReturn() Queue2Play.java:1526 UI error	"There is no previous page."	Source context: } } else if (choice.equalsIgnoreCase("P")) { if (page > 0) { page--; } else { ui.error("There is no previous page.");
completeSelectedReturns() Queue2Play.java:1539 Constraint (fails when false)	"Select an active rental to return."	Must be true: selected != null && !selected.isEmpty() && returnedAt != null
completeSelectedReturns() Queue2Play.java:1544 Constraint (fails when false)	"A selected rental is no longer active or was selected twice."	Must be true: rental != null && rental.status.equals("ACTIVE") && activeRentals.contains(rental) && !checked.contains(rental)
completeSelectedReturns() Queue2Play.java:1546 Constraint (fails when false)	"Invalid return date."	Must be true: !returnedAt.isBefore(rental.checkoutDate)
completeSelectedReturns() Queue2Play.java:1552 Constraint (fails when false)	"Return would exceed total stock; correct inventory first."	Must be true: restored <= MAX_GAME_STOCK
viewActiveRentals() Queue2Play.java:1592 Empty-state information	"There are no active rentals."	Source context: { ArrayList<Rental> rentals = rentalService.getRentalsByDueDate(); if (rentals.isEmpty()) { ui.printHeader("ADMIN > ACTIVE RENTALS", "ACTIVE RENTALS"); ui.showEmptyState("There are no active rentals.");
viewActiveRentals() Queue2Play.java:1630 UI error	"Invalid choice."	Source context: page--; } else if ((choice.equalsIgnoreCase("B") || choice.equals("0"))) { return; } else ui.error("Invalid choice.");
viewAuditLog() Queue2Play.java:1638 Empty-state information	"No actions have been recorded yet."	Source context: { if (auditLog.isEmpty()) { ui.printHeader("ADMIN > AUDIT LOG", "ADMIN AUDIT LOG"); ui.showEmptyState("No actions have been recorded yet.");
viewAuditLog() Queue2Play.java:1688 UI error	"Invalid choice."	Source context: page--; } else if ((choice.equalsIgnoreCase("B") || choice.equals("0"))) { return; } else ui.error("Invalid choice.");
manageInventory() Queue2Play.java:1705 UI error	"Invalid choice."	Source context: int choice; try { choice = parseMenuChoice(input); } catch (NumberFormatException e) { ui.error("Invalid choice.");
manageInventory() Queue2Play.java:1720 UI error	"Invalid choice. Please enter 1, 2, 3, 4, or B."	Source context: } else if (choice == 4) { deleteGame(); } else if (choice == 0) { return; } else { ui.error("Invalid choice. Please enter 1, 2, 3, 4, or B.");
addGame() Queue2Play.java:1759 UI error	"Choose 1, 2, 3, 4, or B for Game Summary."	Source context: step = 5; } else if (input.equals("1") || input.equals("2") || input.equals("3") || input.equals("4")) { step = Integer.parseInt(input); editing = true; } else { ui.error("Choose 1, 2, 3, 4, or B for Game Summary.");

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
addGame() Queue2Play.java:1798 UI error	"Title cannot be empty."	Source context: continue; } return; } if (input.isEmpty()) { ui.error("Title cannot be empty.");
addGame() Queue2Play.java:1802 UI error	"Use a title of 1-100 characters without control characters or the | symbol."	Source context: if (input.isEmpty()) { ui.error("Title cannot be empty."); continue; } if (!GameCatalog.validTitle(input)) { ui.error("Use a title of 1-100 characters without control characters or the | symbol.");
addGame() Queue2Play.java:1806 UI error	"That game already exists on " + platform + ". Update its stock instead."	Source context: if (!GameCatalog.validTitle(input)) { ui.error("Use a title of 1-100 characters without control characters or the | symbol."); continue; } if (editing && gameAlreadyExists(input, platform)) { ui.error("That game already exists on " + platform + ". Update its stock instead.");
addGame() Queue2Play.java:1857 UI error	"Invalid platform choice."	Source context: } else if (input.equals("3")) { selectedPlatform = "Nintendo"; } else { ui.error("Invalid platform choice.");
addGame() Queue2Play.java:1862 UI error	"That game already exists on " + selectedPlatform + ". Update its stock instead."	Source context: ui.error("Invalid platform choice."); continue; } if (gameAlreadyExists(title, selectedPlatform)) { ui.error("That game already exists on " + selectedPlatform + ". Update its stock instead.");
addGame() Queue2Play.java:1908 UI error	"Enter a valid price."	Source context: } double enteredPrice; try { enteredPrice = Double.parseDouble(input); } catch (NumberFormatException e) { ui.error("Enter a valid price.");
addGame() Queue2Play.java:1912 UI error	"Rental price must be between " + ui.money(MIN_RENTAL_PRICE) + " and " + ui.money(MAX_RENTAL_PRICE) + "."	Source context: } catch (NumberFormatException e) { ui.error("Enter a valid price."); continue; } if (!Double.isFinite(enteredPrice) || enteredPrice < MIN_RENTAL_PRICE || enteredPrice > MAX_RENTAL_PRICE) { ui.error("Rental price must be between " + ui.money(MIN_RENTAL_PRICE)
addGame() Queue2Play.java:1956 UI error	"Enter a whole number of copies."	Source context: } int enteredCopies; try { enteredCopies = Integer.parseInt(input); } catch (NumberFormatException e) { ui.error("Enter a whole number of copies.");
addGame() Queue2Play.java:1960 UI error	"Stock must be between 1 and " + MAX_GAME_STOCK + " copies."	Source context: } catch (NumberFormatException e) { ui.error("Enter a whole number of copies."); continue; } if (enteredCopies < 1 || enteredCopies > MAX_GAME_STOCK) { ui.error("Stock must be between 1 and " + MAX_GAME_STOCK + " copies.");
addGame() Queue2Play.java:1994 UI error	"Invalid choice."	Source context: if ((input.equalsIgnoreCase("B") || input.equals("0"))) { step = 4; continue; } if (!input.equals("1")) { ui.error("Invalid choice.");
deleteGame() Queue2Play.java:2038 UI error	"Cannot delete this game because it currently has an active rental."	Source context: if (game == null) { break; } if (rentalService.gameHasActiveRental(game)) { ui.error("Cannot delete this game because it currently has an active rental.");
deleteGame() Queue2Play.java:2046 Warning	"Deleting this game will also cancel its pending queue request(s)."	Source context: int pending = checkoutQueue.countRequestsForGame(game); ui.printHeader("ADMIN > MANAGE INVENTORY > DELETE GAME", "CONFIRM DELETE"); ui.showDeleteGameDetails(game, pending); if (pending > 0) { ui.warning("Deleting this game will also cancel its pending queue request(s).");

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
deleteGame() Queue2Play.java:2062 Warning	cancelled + " pending queue request(s) were cancelled."	Source context: dataStore.saveData(); ui.printHeader("ADMIN > MANAGE INVENTORY > DELETE GAME", "GAME DELETED"); ui.success(game.title + " was deleted successfully."); if (cancelled > 0) { ui.warning(cancelled + " pending queue request(s) were cancelled.");
updateGameStock() Queue2Play.java:2117 UI error	"Invalid choice."	Source context: } if ((input.equalsIgnoreCase("B") || input.equals("0"))) { break; } if (!input.equals("1") && ! input.equals("2")) { ui.error("Invalid choice.");
updateGameStock() Queue2Play.java:2151 UI error	"Quantity must be a whole number."	Source context: int quantity; try { quantity = Integer.parseInt(qtyInput); } catch (NumberFormatException e) { ui.error("Quantity must be a whole number.");
updateGameStock() Queue2Play.java:2155 UI error	"Quantity must be at least 1."	Source context: } catch (NumberFormatException e) { ui.error("Quantity must be a whole number."); continue; } if (quantity < 1) { ui.error("Quantity must be at least 1.");
updateGameStock() Queue2Play.java:2165 UI error	e.getMessage()	Source context: int newStock; try { newStock = rentalService.updatedStock(game, adding, quantity); } catch (IllegalArgumentException e) { ui.error(e.getMessage());

RentalDataStore.java - 65 diagnostic/constraint sites

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
records() RentalDataStore.java:56 Exception	"Cannot read " + path	Source context: continue; } records.add(new Record(line.split("\\|", -1), index + 1)); } } catch (IOException e) { throw new StorageException("Cannot read " + path, e);
require() RentalDataStore.java:70 Exception	message	Source context: { if (!condition) { throw new IllegalArgumentException(message);
positive() RentalDataStore.java:77 Constraint (fails when false)	"Invalid record number"	Must be true: number > 0 && number < Integer.MAX_VALUE
counter() RentalDataStore.java:84 Constraint (fails when false)	"Invalid counter"	Must be true: value > 0
serialized() RentalDataStore.java:117 Constraint (fails when false)	"Administrator password setup is required"	Must be true: user.password != null && PasswordSecurity.isHash(user.password)
validateState() RentalDataStore.java:164 Constraint (fails when false)	"Invalid account fields"	Must be true: AccountManager.validUsername(user.username) && AccountManager.validName(user.fullName) && AccountManager.validEmail(user.email)
validateState() RentalDataStore.java:166 Constraint (fails when false)	"Duplicate account identifier"	Must be true: logins.add(user.username.toLowerCase(Locale.ROOT)) && logins.add(user.email.toLowerCase(Locale.ROOT))
validateState() RentalDataStore.java:167 Constraint (fails when false)	"Invalid account role"	Must be true: user.role.equals("ADMIN") || user.role.equals("CLIENT")
validateState() RentalDataStore.java:168 Constraint (fails when false)	"Invalid membership"	Must be true: user.membership.equals("VIP") || user.membership.equals("REGULAR")
validateState() RentalDataStore.java:172 Constraint (fails when false)	"Invalid game identifier"	Must be true: GameCatalog.validId(game.gameId, game.platform)
validateState() RentalDataStore.java:173 Constraint (fails when false)	"Total game stock exceeds the limit"	Must be true: (long) game.availableCopies + system.rentalService.rentedCopies(game) <= Queue2Play.MAX_GAME_STOCK
validateState() RentalDataStore.java:178 Constraint (fails when false)	"Invalid rental number or counter"	Must be true: rental.rentalNumber > 0 && rental.rentalNumber < system.nextRentalNumber && rentals.add(rental.rentalNumber)
validateState() RentalDataStore.java:179 Constraint (fails when false)	"Unknown rental client"	Must be true: system.users.contains(rental.client) && ! rental.client.isAdmin()
validateState() RentalDataStore.java:180 Constraint (fails when false)	"Invalid rental amount or quantity"	Must be true: rental.quantity >= 1 && rental.quantity <= Queue2Play.MAX_SAVED_COPIES_PER_GAME && Double.isFinite(rental.amount) && rental.amount >= 0
validateState() RentalDataStore.java:181 Constraint (fails when false)	"Invalid rental dates"	Must be true: rental.checkoutDate != null && rental.dueDate != null && rental.dueDate.isAfter(rental.checkoutDate)

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
validateState() RentalDataStore.java:182 Constraint (fails when false)	"Invalid rental status"	Must be true: rental.status.equals("ACTIVE") || rental.status.equals("COMPLETED") || rental.status.equals("CANCELLED")
validateState() RentalDataStore.java:185 Constraint (fails when false)	"Cancelled history must have no charges or return"	Must be true: rental.amount == 0 && rental.returnDate == null && rental.daysLate == 0 && rental.lateFee == 0
validateState() RentalDataStore.java:188 Constraint (fails when false)	"Invalid late fee"	Must be true: rental.daysLate >= 0 && Double.isFinite(rental.lateFee) && rental.lateFee >= 0
validateState() RentalDataStore.java:190 Constraint (fails when false)	"Active rental has no canonical game or heap entry"	Must be true: system.games.get(rental.game.gameId) == rental.game && system.activeRentals.contains(rental)
validateState() RentalDataStore.java:191 Constraint (fails when false)	"Duplicate active rental record"	Must be true: active.add(rental)
validateState() RentalDataStore.java:192 Constraint (fails when false)	"Active client/game quantity exceeds the limit"	Must be true: system.rentalService.activeCopies(rental.client, rental.game) <= Queue2Play.MAX_SAVED_COPIES_PER_GAME
validateState() RentalDataStore.java:194 Constraint (fails when false)	"Active rental has return fields"	Must be true: rental.returnDate == null && rental.daysLate == 0 && rental.lateFee == 0
validateState() RentalDataStore.java:196 Constraint (fails when false)	"Completed rental is still active"	Must be true: !system.activeRentals.contains(rental)
validateState() RentalDataStore.java:199 Constraint (fails when false)	"Active rental heap differs from rental records"	Must be true: system.activeRentals.size() == active.size()
validateState() RentalDataStore.java:202 Constraint (fails when false)	"Invalid request or queue counter"	Must be true: CheckoutQueue.validRequest(request) && request.queueNumber < system.nextQueueNumber && numbers.add(request.queueNumber)
validateState() RentalDataStore.java:203 Constraint (fails when false)	"Unknown request client or game"	Must be true: system.games.get(request.game.gameId) == request.game && system.users.contains(request.client)
validateState() RentalDataStore.java:204 Constraint (fails when false)	"Request membership is stale"	Must be true: request.vip == request.client.isVip()
validateState() RentalDataStore.java:205 Constraint (fails when false)	"Queued and active client/game quantity exceeds the limit"	Must be true: system.checkoutQueue.queuedCopies(request.client, request.game) + system.rentalService.activeCopies(request.client, request.game) <= Queue2Play.MAX_SAVED_COPIES_PER_GAME
loadGames() RentalDataStore.java:221 Constraint (fails when false)	"Invalid game counters"	Must be true: fields.length == 4
loadGames() RentalDataStore.java:225 Constraint (fails when false)	"Invalid game counters"	Must be true: ps5Counter >= 0 && xboxCounter >= 0 && nintendoCounter >= 0 && ps5Counter < Integer.MAX_VALUE && xboxCounter < Integer.MAX_VALUE && nintendoCounter < Integer.MAX_VALUE

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
loadGames() RentalDataStore.java:233 Constraint (fails when false)	"Expected five game fields"	Must be true: fields.length == 5
loadGames() RentalDataStore.java:234 Constraint (fails when false)	"Invalid game ID or platform"	Must be true: GameCatalog.validId(fields[0], fields[2])
loadGames() RentalDataStore.java:238 Constraint (fails when false)	"Duplicate game ID"	Must be true: !system.games.containsKey(fields[0])
loadGames() RentalDataStore.java:240 Constraint (fails when false)	"Duplicate title on this platform"	Must be true: !existing.platform.equals(fields[2]) || !existing.title.equalsIgnoreCase(fields[1])
loadRentals() RentalDataStore.java:259 Constraint (fails when false)	"Invalid rental counter"	Must be true: fields.length == 2
loadRentals() RentalDataStore.java:263 Constraint (fails when false)	"Expected 9, 10 or 13 rental fields"	Must be true: fields.length == 9 || fields.length == 10 || fields.length == 13
loadRentals() RentalDataStore.java:265 Constraint (fails when false)	"Duplicate rental number"	Must be true: numbers.add(number)
loadRentals() RentalDataStore.java:267 Constraint (fails when false)	"Unknown client"	Must be true: client != null && !client.isAdmin()
loadRentals() RentalDataStore.java:268 Constraint (fails when false)	"Invalid rental status"	Must be true: fields[8].equals("ACTIVE") || fields[8].equals("COMPLETED") || fields[8].equals("CANCELLED")
loadRentals() RentalDataStore.java:272 Constraint (fails when false)	"Active rental refers to a missing game"	Must be true: !fields[8].equals("ACTIVE")
loadRentals() RentalDataStore.java:273 Constraint (fails when false)	"Invalid historical game"	Must be true: GameCatalog.validId(fields[2], fields[4])
loadRentals() RentalDataStore.java:274 Constraint (fails when false)	"Invalid historical title"	Must be true: GameCatalog.validTitle(fields[3])
loadRentals() RentalDataStore.java:279 Constraint (fails when false)	"Invalid rental dates"	Must be true: due.isAfter(checkout) && !due.isAfter(checkout.plusDays(Queue2Play.MAX_RENTAL_DAYS + 1L))
loadRentals() RentalDataStore.java:281 Constraint (fails when false)	"Invalid rental amount"	Must be true: Double.isFinite(amount) && amount >= 0
loadRentals() RentalDataStore.java:286 Constraint (fails when false)	"Invalid rental quantity"	Must be true: quantity >= 1 && quantity <= Queue2Play.MAX_SAVED_COPIES_PER_GAME
loadRentals() RentalDataStore.java:296 Constraint (fails when false)	"Invalid return record"	Must be true: rental.daysLate >= 0 && Double.isFinite(rental.lateFee) && rental.lateFee >= 0

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
loadRentals() RentalDataStore.java:297 Constraint (fails when false)	"Invalid return date"	Must be true: rental.returnDate == null || !rental.returnDate.isBefore(checkout)
loadRentals() RentalDataStore.java:298 Constraint (fails when false)	"Active rental has return details"	Must be true: !rental.status.equals("ACTIVE") || rental.returnDate == null && rental.daysLate == 0 && rental.lateFee == 0
loadRentals() RentalDataStore.java:299 Constraint (fails when false)	"Fee without a return date"	Must be true: rental.returnDate != null || rental.daysLate == 0 && rental.lateFee == 0
loadRentals() RentalDataStore.java:301 Constraint (fails when false)	"Return fee does not match the date"	Must be true: rental.status.equals("COMPLETED") && rental.daysLate == RentalManager.daysOverdue(due, rental.returnDate) && rental.lateFee == system.rentalService.calculateLateFee(rental, rental.daysLate)
loadRentals() RentalDataStore.java:307 Constraint (fails when false)	"Cancelled history must have no charges or return"	Must be true: rental.amount == 0 && rental.returnDate == null && rental.daysLate == 0 && rental.lateFee == 0
loadRentals() RentalDataStore.java:311 Constraint (fails when false)	"Active client/game quantity exceeds the limit"	Must be true: system.rentalService.activeCopies(client, game) <= Queue2Play.MAX_SAVED_COPIES_PER_GAME - quantity
loadRentals() RentalDataStore.java:324 Constraint (fails when false)	"Total inventory exceeds the limit for " + game.gameId	Must be true: game.availableCopies + system.rentalService.rentedCopies(game) <= Queue2Play.MAX_GAME_STOCK
loadQueue() RentalDataStore.java:356 Constraint (fails when false)	"Invalid queue counters"	Must be true: fields.length == 2 || fields.length == 3
loadQueue() RentalDataStore.java:360 Constraint (fails when false)	"Invalid VIP counter"	Must be true: streak >= 0 && streak <= CheckoutQueue.VIP_BURST
loadQueue() RentalDataStore.java:365 Constraint (fails when false)	"Expected 4 to 6 request fields"	Must be true: fields.length >= 4 && fields.length <= 6
loadQueue() RentalDataStore.java:367 Constraint (fails when false)	"Duplicate queue number"	Must be true: numbers.add(number)
loadQueue() RentalDataStore.java:370 Constraint (fails when false)	"Unknown request client or game"	Must be true: client != null && !client.isAdmin() && game != null
loadQueue() RentalDataStore.java:371 Constraint (fails when false)	"Invalid membership flag"	Must be true: fields[3].equals("true") || fields[3].equals("false")
loadQueue() RentalDataStore.java:379 Constraint (fails when false)	"Request duration or quantity is outside the limits"	Must be true: CheckoutQueue.validRequest(request)
loadQueue() RentalDataStore.java:386 Constraint (fails when false)	"Queued and active client/game quantity exceeds the limit"	Must be true: copies <= Queue2Play.MAX_SAVED_COPIES_PER_GAME

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
writeLogRecord() RentalDataStore.java:421 Constraint (fails when false)	"Invalid log record"	Must be true: !stamp.contains("|") && !type.contains("|") && !message.contains("\n") && !message.contains("\r")
loadLogs() RentalDataStore.java:437 Constraint (fails when false)	"Invalid log record"	Must be true: fields.length == 3
loadLogs() RentalDataStore.java:444 Exception	"Cannot load logs.txt; original file was kept."	Source context: if (fields[1].equals("ADMIN")) { system.auditLog.push(fields[0] + "|" + fields[2]); } } } catch (IOException e) { throw new StorageException("Cannot load logs.txt; original file was kept.", e);
loadLogs() RentalDataStore.java:446 Exception	"Cannot load logs.txt; original file was kept."	Source context: } } } catch (IOException e) { throw new StorageException("Cannot load logs.txt; original file was kept.", e); } catch (IllegalArgumentException e) { throw new StorageException("Cannot load logs.txt; original file was kept.", e);

RentalManager.java - 24 diagnostic/constraint sites

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
updatedStock() RentalManager.java:45 Constraint (fails when false)	"Quantity must be positive"	Must be true: quantity >= 1
updatedStock() RentalManager.java:52 Constraint (fails when false)	"Available plus rented copies must stay between zero and " + Queue2Play.MAX_GAME_STOCK	Must be true: result >= 0 && result + rentedCopies(game) <= Queue2Play.MAX_GAME_STOCK
calculateRentalCost() RentalManager.java:59 Constraint (fails when false)	"Client and game are required"	Must be true: client != null && game != null
calculateRentalCost() RentalManager.java:69 Constraint (fails when false)	"Invalid rental duration"	Must be true: days >= 1 && days <= Queue2Play.MAX_RENTAL_DAYS
calculateRentalCost() RentalManager.java:76 Constraint (fails when false)	"Invalid quantity"	Must be true: quantity >= 1 && quantity <= Queue2Play.MAX_SAVED_COPIES_PER_GAME
validateCheckout() RentalManager.java:124 Returned validation error	"This queue request no longer exists."	Source context: { if (request == null || checkoutQueue.findByNumber(request.queueNumber) != request) { return "This queue request no longer exists.";
validateCheckout() RentalManager.java:127 Returned validation error	"Invalid request client, duration or quantity."	Source context: { if (request == null || checkoutQueue.findByNumber(request.queueNumber) != request) { return "This queue request no longer exists."; } if (!CheckoutQueue.validRequest(request)) { return "Invalid request client, duration or quantity.";
validateCheckout() RentalManager.java:130 Returned validation error	"This game is no longer in the catalog."	Source context: } if (!CheckoutQueue.validRequest(request)) { return "Invalid request client, duration or quantity."; } if (catalog.get(request.game.gameId) != request.game) { return "This game is no longer in the catalog.";
validateCheckout() RentalManager.java:133 Returned validation error	"This checkout would exceed " + Queue2Play.MAX_SAVED_COPIES_PER_GAME + " active copies of this game for the client."	Source context: } if (catalog.get(request.game.gameId) != request.game) { return "This game is no longer in the catalog."; } if (activeCopies(request.client, request.game) > Queue2Play.MAX_SAVED_COPIES_PER_GAME - request.quantity) { return "This checkout would exceed " + Queue2Play.MAX_SAVED_COPIES_PER_GAME
validateCheckout() RentalManager.java:138 Returned validation error	"Invalid game price."	Source context: return "This checkout would exceed " + Queue2Play.MAX_SAVED_COPIES_PER_GAME + " active copies of this game for the client."; } if (!Double.isFinite(request.game.price) || request.game.price < Queue2Play.MIN_RENTAL_PRICE || request.game.price > Queue2Play.MAX_RENTAL_PRICE) { return "Invalid game price.";
validateCheckout() RentalManager.java:141 Returned validation error	"Not enough stock to process this checkout."	Source context: if (!Double.isFinite(request.game.price) || request.game.price < Queue2Play.MIN_RENTAL_PRICE || request.game.price > Queue2Play.MAX_RENTAL_PRICE) { return "Invalid game price."; } if (request.game.availableCopies < request.quantity) { return " Not enough stock to process this checkout.";
checkout() RentalManager.java:155 Constraint (fails when false)	"Invalid or duplicate rental number"	Must be true: rentalNumber > 0 && rentalNumber < Integer.MAX_VALUE && !duplicateNumber

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
checkout() RentalManager.java:159 Exception	error	Source context: } RentalDataStore.require(rentalNumber > 0 && rentalNumber < Integer.MAX_VALUE && ! duplicateNumber, "Invalid or duplicate rental number"); String error = validateCheckout(request); if (error != null) { throw new IllegalArgumentException(error);
calculateLateFee() RentalManager.java:177 Constraint (fails when false)	"Days late cannot be negative"	Must be true: daysLate >= 0
completeReturn() RentalManager.java:197 Constraint (fails when false)	"Invalid return date"	Must be true: rental != null && returnedAt != null && ! returnedAt.isBefore(rental.checkoutDate)
completeReturn() RentalManager.java:199 Exception	"This rental is no longer active."	Source context: { RentalDataStore.require(rental != null && returnedAt != null && !returnedAt.isBefore(rental.checkoutDate), "Invalid return date"); if (!rental.status.equals("ACTIVE") || ! activeRentals.contains(rental)) { throw new IllegalArgumentException("This rental is no longer active.");
completeReturn() RentalManager.java:202 Constraint (fails when false)	"Return would exceed total stock; correct inventory first"	Must be true: restored <= Queue2Play.MAX_GAME_STOCK
validateCart() RentalManager.java:214 Returned validation error	"A client account is required."	Source context: { if (client == null || client.isAdmin()) { return "A client account is required.";
validateCart() RentalManager.java:217 Returned validation error	"Your cart is empty. Add a game before submitting."	Source context: { if (client == null || client.isAdmin()) { return "A client account is required."; } if (cart.isEmpty()) { return "Your cart is empty. Add a game before submitting.";
validateCart() RentalManager.java:221 Returned validation error	"This game is no longer in the catalog."	Source context: if (cart.isEmpty()) { return "Your cart is empty. Add a game before submitting."; } for (Game game : cart.keySet()) { if (game == null || cart.get(game) == null || catalog.get(game.gameId) != game) { return "This game is no longer in the catalog.";
validateCart() RentalManager.java:225 Returned validation error	"Invalid quantity for " + game.title + ". Maximum is " + Queue2Play.MAX_COPIES_PER_GAME + "."	Source context: if (game == null || cart.get(game) == null || catalog.get(game.gameId) != game) { return "This game is no longer in the catalog."; } int quantity = cart.get(game); if (quantity < 1 || quantity > Queue2Play.MAX_COPIES_PER_GAME) { return "Invalid quantity for " + game.title + ". Maximum is " + Queue2Play.MAX_COPIES_PER_GAME + ".";
validateCart() RentalManager.java:228 Returned validation error	"Maximum queued quantity is " + Queue2Play.MAX_COPIES_PER_GAME + " copies for " + game.title + ", including this cart."	Source context: int quantity = cart.get(game); if (quantity < 1 || quantity > Queue2Play.MAX_COPIES_PER_GAME) { return "Invalid quantity for " + game.title + ". Maximum is " + Queue2Play.MAX_COPIES_PER_GAME + "."; } if (checkoutQueue.queuedCopies(client, game) > Queue2Play.MAX_COPIES_PER_GAME - quantity) { return " Maximum queued quantity is " + Queue2Play.MAX_COPIES_PER_GAME
validateCart() RentalManager.java:232 Returned validation error	"You are already renting " + game.title + "."	Source context: if (checkoutQueue.queuedCopies(client, game) > Queue2Play.MAX_COPIES_PER_GAME - quantity) { return "Maximum queued quantity is " + Queue2Play.MAX_COPIES_PER_GAME + " copies for " + game.title + ", including this cart."; } if (hasActiveRental(client, game)) { return "You are already renting " + game.title + ".";

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
validateCart() RentalManager.java:235 Returned validation error	"Not enough stock for " + game.title + ". Available: " + game.availableCopies + "."	Source context: } if (hasActiveRental(client, game)) { return " You are already renting " + game.title + "."; } if (quantity > game.availableCopies) { return "Not enough stock for " + game.title + ". Available: " + game.availableCopies + ".”;

TerminalUI.java - 25 diagnostic/constraint sites

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
confirmDiscardTransaction() TerminalUI.java:348 UI error	"Use Y or N."	Source context: return true; } if (answer.equalsIgnoreCase("N") || answer.equalsIgnoreCase("NO")) { return false; } error("Use Y or N.");
readText() TerminalUI.java:771 Exception		Source context: return readWindowsInput(false).trim(); } if (System.console() != null) { String line = System.console().readLine(); if (line == null) { throw new InputEndedException();
readText() TerminalUI.java:777 Exception		Source context: } return line.trim(); } if (!input.hasNextLine()) { printLine("\nInput ended. Goodbye!"); throw new InputEndedException();
readPassword() TerminalUI.java:797 Exception		Source context: if (System.console() != null) { emit(prompt); outputAtLineStart = true; char[] password = System.console().readPassword(); if (password == null) { throw new InputEndedException();
readPassword() TerminalUI.java:803 Exception	"Run QUEUE2PLAY in an interactive terminal to enter a hidden password."	Source context: } String value = new String(password); java.util.Arrays.fill(password, '\0'); return value; } throw new IllegalArgumentException("Run QUEUE2PLAY in an interactive terminal to enter a hidden password.");
readWindowsInput() TerminalUI.java:836 Exception	"Interactive input requires a Windows terminal. Use run.ps1 in the VS Code terminal."	Source context: try (java.io.BufferedReader reader = new java.io.BufferedReader(new java.io.InputStreamReader(process.getInputStream(), java.nio.charset.StandardCharsets.UTF_8))) { encoded = reader.readLine(); } if (process.waitFor() != 0 || encoded == null) { throw new IllegalArgumentException("Interactive input requires a Windows terminal. Use run.ps1 in the VS Code terminal.");
readWindowsInput() TerminalUI.java:843 Exception	"Cannot start password masking. Windows PowerShell is required."	Source context: byte[] bytes = java.util.Base64.getDecoder().decode(encoded); String password = new String(bytes, java.nio.charset.StandardCharsets.UTF_8); java.util.Arrays.fill(bytes, (byte) 0); return password; } catch (java.io.IOException e) { throw new IllegalArgumentException("Cannot start password masking. Windows PowerShell is required.", e);
readWindowsInput() TerminalUI.java:846 Exception		Source context: return password; } catch (java.io.IOException e) { throw new IllegalArgumentException("Cannot start password masking. Windows PowerShell is required.", e); } catch (InterruptedException e) { Thread.currentThread().interrupt(); throw new InputEndedException();
waitForResultNavigation() TerminalUI.java:1051 Direct message / retry	"Use the displayed navigation key."	Source context: return false; } if (!previousDestination.isEmpty() && choice.equalsIgnoreCase("B")) { return true; } clearResultRetry("Use the displayed navigation key.");
clearResultRetry() TerminalUI.java:1060 UI error	message	Source context: { int width = currentWidth; clearScreen(); currentWidth = width; error(message);
success() TerminalUI.java:1066 Direct message / retry	"SUCCESS: " + message	Source context: { showPendingMessage(currentWidth); printMessageBox("SUCCESS: " + message, currentWidth);

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
confirmRegistrationCancellation() TerminalUI.java:1627 UI error	"Use Y to discard or N to keep registering."	Source context: return true; } if (answer.equalsIgnoreCase("N") || answer.equalsIgnoreCase("NO")) { return false; } error("Use Y to discard or N to keep registering.");
showLoginCooldown() TerminalUI.java:1634 Direct message / retry	"WARNING: Three failed attempts. Wait 30 seconds before trying again."	Source context: { showPendingMessage(STANDARD_WIDTH); printMessageBox("WARNING: Three failed attempts. Wait 30 seconds before trying again.", STANDARD_WIDTH);
waitLoginSecond() TerminalUI.java:1678 Exception		Source context: { try { Thread.sleep(1000); } catch (InterruptedException e) { Thread.currentThread().interrupt(); throw new InputEndedException();
showQueue() TerminalUI.java:1766 Empty-state information	"The checkout queue is empty."	Source context: { if (queue.isEmpty()) { showEmptyState("The checkout queue is empty.");
askYesNo() TerminalUI.java:1798 UI error	"Please enter Y or N."	Source context: return true; } if (answer.equalsIgnoreCase("N") || answer.equalsIgnoreCase("NO")) { return false; } error("Please enter Y or N.");
confirmDiscardCart() TerminalUI.java:1943 UI error	"Use Y or N."	Source context: return true; } if (answer.equalsIgnoreCase("N") || answer.equalsIgnoreCase("NO")) { return false; } error("Use Y or N.");
showCheckoutQueue() TerminalUI.java:2245 Empty-state information	"The checkout queue is empty."	Source context: + (requests.isEmpty() ? 0 : start + 1) + "-" + end); printLine(border(currentWidth, '=')); flushBorder(false); printLine(); if (requests.isEmpty()) { showEmptyState("The checkout queue is empty.");
confirmCheckoutAction() TerminalUI.java:2419 UI error	"Use Y or N."	Source context: return true; } if (answer.equalsIgnoreCase("N") || answer.equalsIgnoreCase("NO")) { return false; } error("Use Y or N.");
waitForReceiptNavigation() TerminalUI.java:2682 Direct message / retry	"Use the displayed navigation keys."	Source context: if (choice.equalsIgnoreCase("N") && page < pages - 1) { page++; } else if (choice.equalsIgnoreCase("P") && page > 0) { page--; } else { clearResultRetry("Use the displayed navigation keys.");
confirmCheckoutBatch() TerminalUI.java:2757 Direct message / retry	"Use Y or N, or the displayed page keys."	Source context: if (choice.equalsIgnoreCase("F") && page < pages - 1) { page++; } else if (choice.equalsIgnoreCase("P") && page > 0) { page--; } else { clearResultRetry("Use Y or N, or the displayed page keys.");
showReturnSelectionList() TerminalUI.java:2854 Empty-state information	"No matching active rentals."	Source context: } showListStatus("Search: " + search + " · Page: " + (page + 1) + "/" + pages, "Showing " + (rentals.isEmpty() ? 0 : start + 1) + "-" + end + " of " + rentals.size(), RETURN_TABLE_WIDTH); if (rentals.isEmpty()) { showEmptyState("No matching active rentals.");
confirmSelectedReturns() TerminalUI.java:2943 Direct message / retry	"Use Y to confirm or N to return to the list."	Source context: if (answer.equalsIgnoreCase("F") && page < pages - 1) { page++; } else if (answer.equalsIgnoreCase("P") && page > 0) { page--; } else { clearResultRetry("Use Y to confirm or N to return to the list.");

SITE / EXACT LINE	EXACT MESSAGE EXPRESSION	CONDITION OR ACTUAL SOURCE CONTEXT
waitForReturnResultNavigation() TerminalUI.java:3016 Direct message / retry	"Use the displayed navigation keys."	Source context: if (choice.equalsIgnoreCase("N") && page < pages - 1) { page++; } else if (choice.equalsIgnoreCase("P") && page > 0) { page--; } else { clearResultRetry("Use the displayed navigation keys.");
showFullNameInstructions() TerminalUI.java:3199 UI error	"Use 5-60 characters with at least two letters. Only letters, spaces and dots are allowed."	Source context: { error("Use 5-60 characters with at least two letters. Only letters, spaces and dots are allowed.");

B.1 Boolean validation predicates and format checks

The following actual methods provide the conditions that can reject a value without directly printing a message. Catalog IDs require the platform prefix P/X/N, 3-10 decimal digits and an integer value above zero and below Integer.MAX_VALUE. PasswordSecurity.isHash checks only the prefix; validateHash validates the complete encoding before verification or retaining an existing hash.

Actual source: AccountManager.java:19-21

```
19  static boolean validUsername(String value) {
20      return value != null && value.matches("[A-Za-z0-9_.-]{3,32}") &&
    !value.equalsIgnoreCase("C") && !value.equals("0");
21  }
```

Actual source: AccountManager.java:24-26

```
24  static boolean validName(String value) {
25      return value != null && value.length() <= 100 && value.matches("[\\p{L}][\\p{L}\\p{M}
    .'-]{1,99}");
26  }
```

Actual source: AccountManager.java:29-31

```
29  static boolean validEmail(String value) {
30      return value != null && value.length() <= 254 &&
    value.matches("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
31  }
```

Actual source: AccountManager.java:35-49

```
35  static boolean validRegistrationName(String value) {
36      if (value == null || value.length() < 5 || value.length() > 60) {
37          return false;
38      }
39      int letters = 0;
40      for (int i = 0; i < value.length(); i++) {
41          char character = value.charAt(i);
42          if (Character.isLetter(character)) {
43              letters++;
44          } else if (character != ' ' && character != '.') {
45              return false;
46          }
47      }
48      return letters >= 2 && Character.isLetter(value.charAt(0));
49  }
```

Actual source: AccountManager.java:52-62

```
52  static boolean validRegistrationUsername(String value) {
53      if (value == null || !value.matches("[A-Za-z0-9_.-]{3,20}")) {
54          return false;
55      }
56      for (int i = 0; i < value.length(); i++) {
57          if (Character.isLetter(value.charAt(i))) {
58              return true;
59          }
60      }
61      return false;
62  }
```

Actual source: AccountManager.java:65-82

```
65  static boolean validRegistrationEmail(String value) {
66      if (!validEmail(value) || value.length() > 100) {
67          return false;
68      }
69      int at = value.indexOf('@');
70      String local = value.substring(0, at);
71      String domain = value.substring(at + 1);
72      if (local.startsWith(".") || local.endsWith(".") || value.contains("..")) {
73          return false;
74      }
75      String[] labels = domain.split("\\.", -1);
76      for (String label : labels) {
77          if (label.isEmpty() || label.length() > 63 || label.startsWith("-") ||
    label.endsWith("-")) {
78              return false;
79          }
80      }
81      return true;
82  }
```

Actual source: AccountManager.java:85-97

```
85  static boolean validRegistrationPassword(String value) {
86      if (value == null || value.length() < 8 || value.length() > 20) {
87          return false;
88      }
89      for (int i = 0; i < value.length(); i++) {
90          char character = value.charAt(i);
91          if (character == '|' || Character.isWhitespace(character)
92                  || Character.isSpaceChar(character) || Character.isISOControl(character)) {
93              return false;
94          }
95      }
96      return true;
97  }
```

Actual source: GameCatalog.java:6-9

```
6   static boolean validTitle(String title) {
7       return title != null && !title.isBlank() && title.length() <= 100 &&
    !title.contains("|")
8       && !containsControlCharacter(title);
9   }
```

Actual source: GameCatalog.java:12-19

```
12  static boolean containsControlCharacter(String value) {
13      for (int index = 0; index < value.length(); index++) {
14          if (Character.isISOControl(value.charAt(index))) {
15              return true;
16          }
17      }
18      return false;
19  }
```

Actual source: GameCatalog.java:22-42

```
22  static boolean validId(String id, String platform) {
23      String prefix;
24      if (platform.equals("PS5")) {
25          prefix = "P";
26      } else if (platform.equals("Xbox")) {
27          prefix = "X";
28      } else if (platform.equals("Nintendo")) {
29          prefix = "N";
30      } else {
31          return false;
32      }
33      if (id == null || !id.matches(prefix + "[0-9]{3,10}")) {
34          return false;
35      }
36      try {
37          int n = Integer.parseInt(id.substring(1));
38          return n > 0 && n < Integer.MAX_VALUE;
39      } catch (NumberFormatException e) {
40          return false;
41      }
42  }
```

Actual source: CheckoutQueue.java:56-60

```
56  static boolean validRequest(RentalRequest request) {
57      return request != null && request.client != null && request.game != null &&
    !request.client.isAdmin()
58          && request.queueNumber > 0 && request.quantity >= 1 && request.quantity <=
    Queue2Play.MAX_SAVED_COPIES_PER_GAME
59          && request.rentalDays >= 1 && request.rentalDays <= Queue2Play.MAX_RENTAL_DAYS;
60  }
```

Actual source: PasswordSecurity.java:12-14

```
12  static boolean isHash(String value) {
13      return value != null && value.startsWith("pbkdf2$");
14  }
```

Actual source: PasswordSecurity.java:40-50

```
40  static void validateHash(String encoded) {
41      String[] p = encoded.split("\\$", -1);
42      if (p.length != 4 || !p[0].equals("pbkdf2")) {
43          throw new IllegalArgumentException("Invalid password hash.");
44      }
45      int iterations = Integer.parseInt(p[1]);
46      if (iterations < 100000 || iterations > 1000000 ||
    Base64.getDecoder().decode(p[2]).length != 16
47          || Base64.getDecoder().decode(p[3]).length != 32) {
48          throw new IllegalArgumentException("Invalid password hash.");
49      }
50  }
```

B.2 Stored-record validation overview

DATA	CONSTRAINTS CHECKED BY LOAD / SAVE
Accounts	Six fields; valid saved identifiers/name/email; unique username/email across both identifier types; ADMIN/CLIENT role; VIP/REGULAR membership; stored password minimum length/control check; valid hash when hashing/serializing.
Games	Five fields; valid ID/platform correspondence; title/price/stock; unique ID; case-insensitive unique title on a platform; nonnegative counters below integer limit; total available+rented stock <=20.
Rentals	9/10/13 fields; positive unique numbers; known non-admin client; valid status; canonical active game or supported historical game; due after checkout and within supported old-date window; finite nonnegative amount; qty1-5; valid return date, fee and computed late days.
Active records	Heap membership matches ACTIVE allRentals entries; no duplicate active object; no return fields; active client/game quantity <=5; active canonical game; completed/cancelled records not in heap.
Cancelled history	Zero amount, no return date, zero late days/fee. It is history only and cannot be an active heap entry.
Queue	4-6 fields; positive unique numbers; known client/game; exact true/false stored flag; duration1-30; qty1-5; queued+active client/game<=5; canonical membership restored; saved streak0-3.
Logs	Valid timestamps/type layouts and safe fields for newly written records. ADMIN entries are pushed into auditLog; EVENT entries remain in the saved log list.
Counters / file state	Counter exceeds record number, positive/bounded values; current files regular; original byte equality before save; lock required; recoverable journal state.

Appendix C. Source snapshot and reference manifest

Line numbers count physical source lines, starting at 1. The table records every current application Java file analyzed, with its complete SHA-256 digest. Digests were checked again after report generation to verify that documentation work did not change application source. The current test file and launch script are supporting references, not additional application classes.

JAVA FILE	LINES	DECLARED METHODS	SHA-256
AccountManager.java	401	19	fba587ed8d18bd1388114072319885c501361ecec661577694d3ed6c1431f184
AppStorage.java	209	10	d343b33c5c0b92e717483b9acc8bbc3d394f5dc71180e971200fe5380babc12c
CheckoutQueue.java	288	22	85253ea77cd8ea1943e7911ad361f0dc1d409a7b265a66944b9ccb2b80d67442
Game.java	23	2	f974a66c22326cae4d5d89942079ba96b50d5eefedf3ce3bfc9cd92aec231e13
GameCatalog.java	186	14	7724333e79f1efe21e4bf137df7f12b341facdf2c0f67ddf8caf11b14380d42d
InputEndedException.java	4	0	2d4c0a207815b8d079142527ac33ae51fe21a6410acd8b94b05f81708ebd04c4
Main.java	17	1	64209dda1326e855a18e54157383692494af933a00a25c2c259ca4f44f3a4a37
PasswordSecurity.java	79	6	fe7e70ef15d9d3e61584643675233f1c32338d505bf822ea020a4f53da511fb2
Queue2Play.java	2200	61	50ac1007e622b77f024d305acea7ee904c5a43b357547a7c03212e513fe86d18
Rental.java	44	2	eb6708a54de62c1a498a59cfae8c1f073f98cd6a05e96c57997f083ea52bdf2e
RentalDataStore.java	561	24	049cc5dc3a3d6314bcbc1ed7462224584f926dc4dad494bb3cd0cbd92e889fa3
RentalManager.java	240	18	caf53a2a0067a93435df17b49f68d44c9e895fadecd2729a51cc2c97a904f0a8
RentalRequest.java	18	1	aab5b35db3a58f3a2067dc8bea4671d92f7af31d620a4a6e0dd523837884d481
StorageException.java	13	2	328f0e896e3cba659b01284358b4e7063683ca631769a6bb3524c46031d4bb83
TerminalUI.java	3231	227	781a9ac9694fe5c843aebff0b12c205c67478353bb0549faa408a436beffe1d9
User.java	36	3	927b1fa0237dd43ada4f63d7d026834d9ed96824799ce4301fbc079e74fd8b21

Supporting references: run.ps1; Backfiles testing/README.md; Backfiles testing/tests/README.md; Backfiles testing/tests/BehaviorRegressionTest.java. The regression class uses scripted UI fixtures and disposable data folders. Its compile/run, real-cooldown and verification scope are reported in Chapter 12. Source evidence inside this document is local project code; no web-derived feature or invented message is used.

Documentation reading order for defense: overview -> class connections -> DSA -> queue scenarios -> storage safety -> validations -> test evidence -> questions. When source changes, regenerate or recheck line references; this PDF is an accurate snapshot, not a promise that later code has the same lines.